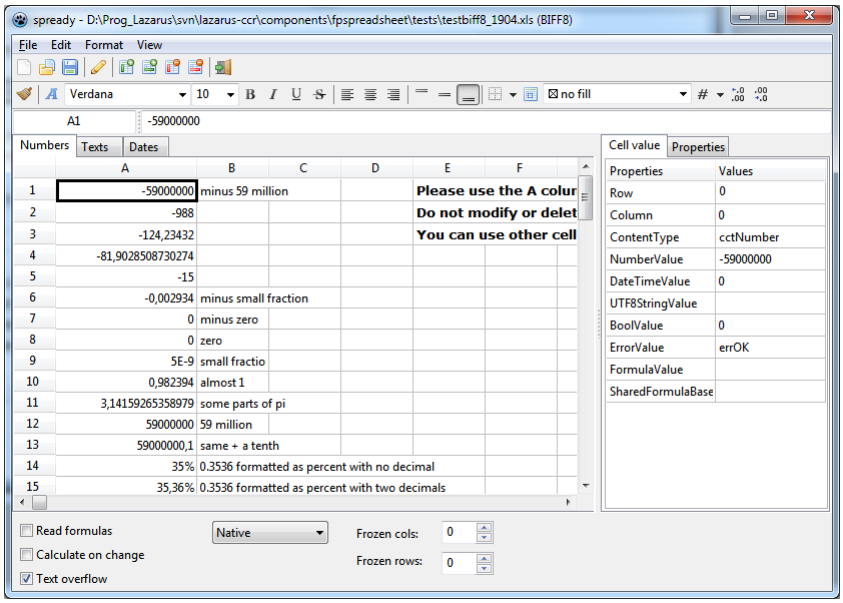
FPSpreadsheet

The fpSpreadsheet library offers a convenient way to generate and read spreadsheet documents in various formats. The library is written in a very flexible manner, capable of being extended to support any number of formats easily.

Screenshot of spready demo program provided with fpspreadsheet showing an XLS file:



Documentation

This wiki page covers the latest development/trunk version of FPSpreadsheet available via subversion. Please see section Stable releases for documentation on the stable version that you can download.

API Documentation

API Reference

A help file in CHM format can be found in the FPSpreadsheet installation folder *docs*. If you did not yet install the package follow http://lazarus-ccr.svn.sourceforge.net/viewvc/lazarus-ccr/components/fpspreadsheet/docs/fpspreadsheet-api.chm to *fpspreadsheet-api.chm*.

The second CHM file available in the folder *docs*, *fpspreadsheet-wiki.chm*, is a snapshot of the FPSpreadsheet-related wiki pages bundled into a single help file.

Basics

The smallest entities in a spreadsheet are the **cells** which contain the data. Cells can hold various data types, like strings, numbers, dates, times, boolean values, or formulas. In addition, cells can contain information on formatting, i.e. font style, background color, text alignment etc.

The cells are arranged in a grid-like structure, called **worksheet**, or **spreadsheet**, consisting of **rows** and **columns**. Each cell has a unique address given by the row and column index.

Worksheets are bound together to form a **workbook** which represents the document of the spreadsheet application. The workbook also stores information that is needed from all worksheets, i.e. font list, cell and number format lists, etc.

FPSpreadsheet follows this same structure - there is a `TCell`, a `TsWorksheet`, and a `TsWorkbook`.

Workbook

The class `TsWorkbook` is the main class visible to the user. It provides methods for reading data from and writing to files. The versatile structure of the library provides access to various popular file formats, like Excel *.xls* or *.xlsx*, or OpenOffice/LibreOffice *.ods*.

The file format is specified by the type `TsSpreadsheetFormat` defined in the unit `fpstypes`

```
type
  TsSpreadsheetFormat = (sfExcel2, sfExcel5, sfExcel8, sfExcelXML, sfOOXML,
    sfOpenDocument, sfCSV, sfHTML, sfWikiTable_Pipes, sfWikiTable_WikiMedia, sfUser);
```

where

- `sfExcel2`, `sfExcel5`, `sfExcel8` stands for versions of the binary xls format used by Excel ("BIFF" = "Binary Interchange File Format") with `sfExcel8` being the most modern one.
- `sfOOXML` corresponds to the newer xlsx format introduced by Excel2007
- `sfExcelXML` is the xml format which was introduced by Microsoft for Office XP and 2003. Not very popular.
- `sfOpenDocument` is the spreadsheet format used by OpenOffice/LibreOffice; by default, the files have the extension *.ods*.

- `sfCSV` refers to comma-delimited text files (default extension *.csv*); they can be understood by any text editor and all spreadsheet programs, but do not contain formatting information.
- `sfHTML` denotes the standard HTML format as used in web browsers.
- `sfWikiTable_Pipes` and `sfWikiTable_WikiMedia` is the format used by tables in wiki websites.
- `sfUser` is needed to register a user-defined format. There are no plans to implement "ancient" file formats like Excel3.0/4.0 or Lotus. It is possible, however, to provide your own reading and writing classes to extend the functionality of FPSpreadsheet - see the section below on Adding new file formats

When applying fpspreadsheet the first task is to create an instance of the workbook:

```
var
  MyWorkbook: TsWorkbook;
begin
  MyWorkbook := TsWorkbook.Create;
  ...
```

Reading of spreadsheet files is accomplished (among others) by the workbook methods

- `procedure ReadFromFile(AFileName: string)`:
  Reads the file with the given name and automatically determines the correct file format.
- `procedure ReadFromFile(AFileName: string; AFormat: TsSpreadsheetFormat)`:
  Reads the file, but assumes that the file format is as specified by `AFormat`.

The following workbook methods can be used for saving to file:

- `procedure WriteToFile(const AFileName: string; const AFormat: TsSpreadsheetFormat; const AOverwriteExisting: Boolean = False)`:
  Writes the workbook to the given file using the given spreadsheet format. If the file already exists it is automatically overwritten if `AOverwriteExisting` is `true`:
- `procedure WriteToFile(const AFileName: String; const AOverwriteExisting: Boolean = False)`:
  ditto, but the file format is determined from the file extension provided (in case of Excel's xls the most recent version, `sfExcel8`, is used).

After calling these methods it is advantageous to look at the workbook's property `ErrorMsg` in which messages due to errors or warnings are collected that might have occured during reading/writing. This property returns a multi-lined string which is displayed best in a memo component; if everything was fine it is an empty string.

Note: FPSpreadsheets provides specialized units for reading from and writing to each file format. These units are not available automatically, you have to add them to the uses clause explicitly. FPSpreadsheet will complain about "unsupported file format" if the requested reader/writer is not found. Here is a list of the unit names:

- *xlsbiff2*, *xlsbiff5* and *xlsbiff8* for the binary xls file formats `sfExcel2`, `sfExcel5` and `sfExcel8`, respectively,
- *xlsxOOXML* for the xlsx file format `sfOOXML` of Excel 2007 and later,
- *xlsXML* for the xml format of Excel XP and 2003,
- *fpsopendocument* for the file format `sfOpenDocument` of OpenOffice/LibreOffice,
- *fpsCSV* for text files with comma-separated values (csv),
- *fpsHTML* for HTML files,
- *wikitables* for `sfWikiTable_Pipes` and `sfWikiTable_WikiMedia`,
- or, simply add *fpsallformats* to get read/write support for all file formats supported.

Worksheet

The workbook contains a list of `TsWorksheet` instances. They correspond to the tabs that you see in Excel or Open/LibreOffice. When reading a spreadsheet file the worksheets are created automatically according to the file contents. When a spreadsheet is created manually to be stored on file a worksheet has to be created by *adding* it to the workbook:

```
var
  MyWorkbook: TsWorkbook;
  MyWorksheet: TsWorksheet;
begin
  MyWorkbook := TsWorkbook.Create;
  MyWorksheet := MyWorkbook.AddWorksheet('My_Table');
    // 'My_Table' is the "name" of the worksheet
  ...
```

Already existing worksheets can be accessed by using the `TsWorkbook` methods

- `function GetFirstWorksheet: TsWorksheet`: retrieves the first worksheet of the workbook.
- `function GetWorksheetByIndex(AIndex: Cardinal): TsWorksheet`: returns the worksheet with the given index (starting at 0).
- `function GetWorksheetByName(AName: String): TsWorksheet`: returns the worksheet with the given name which was used when the worksheet was added.

The count of already existing worksheets can be queried by calling `GetWorksheetCount`.

Cell

The worksheet, finally, gives access to the cells. A newly created worksheet, as in above example, is empty and does not contain any cells. Cells are added by assigning data or attributes to them by one of the `WriteXXXX` methods of the worksheet. As already mentioned, a cell is addressed by the index of the row and column to which it belongs. As usual, row and column indexes start at 0. Therefore, cell "A1" belongs to row 0 and column 0. It should be noted that row and column index are always specified in this order, this is *different from the convention of* `TStringGrid`. The following example creates a cell at address A1 and puts the number `1.0` in it.

```
var
  MyWorkbook: TsWorkbook;
  MyWorksheet: TsWorksheet;
begin
  MyWorkbook := TsWorkbook.Create;
  MyWorksheet := MyWorkbook.AddWorksheet('My_Table');
  MyWorksheet.WriteNumber(0, 0, 1.0);  // "A1" has row=0 and col=0
  ...
```

It is also possible to access cells directly by means of the methods `FindCell(ARow, ACol)` or `GetCell(ARow, ACol)` of the workbook. Both funtions exist also in an overloaded version to which the cell address can be passed in Excel notation: `FindCell(ACellStr: String)` or `GetCell(ACellStr: String)`. Please be aware that these functions return a `pointer` to the cell data (type `PCell`). Don't forget to dereference the pointers! The difference between `FindCell` and `GetCell` is that the former one returns `nil`, if a cell does not yet exist, while the latter one creates an empty cell in this case.

```
if MyWorksheet.FindCell('A1') = nil then
  WriteLn('Cell A1 does not exist.');
```

The basic TCell record

This is the declaration of the cell's data type:

```
type
  TCell = record
    { Location of the cell }
    Worksheet: TsWorksheet;
    Col: Cardinal; // zero-based
    Row: Cardinal; // zero-based

    { Index of format record }
    FormatIndex: Integer;

    { Status flags }
    Flags: TsCellFlags;        // (cfHasComment, cfMerged, cfHyperlink, ...)

    { Cell content }
    UTF8StringValue: String;   // strings cannot be part of a variant record
    case ContentType: TCellContentType of  // must be at the end of the declaration
      cctEmpty      : ();      // has no data at all
      cctFormula    : ();      // UTF8StringValue is outside the variant record
      cctNumber     : (Numbervalue: Double);
      cctUTF8String : ();      // FormulaValue is outside the variant record
      cctDateTime   : (DateTimevalue: TDateTime);
      cctBool       : (BoolValue: boolean);
      cctError      : (ErrorValue: TsErrorValue);
  end;
  PCell = ^TCell;
```

The field `ContentType` indicates which data type is stored in the cell:

```
type
  TCellContentType = (cctEmpty, cctFormula, cctNumber, cctUTF8String, cctDateTime, cctBool, cctError);
```

According to this field the corresponding data can be found in the fields

- `NumberValue` (for `ContentType=cctNumber`), or
- `UTF8StringValue` (for `ContentType=cctUTF8String`), or
- `DateTimeValue` (for `ContentType=cctDateTime`), or
- `BoolValue` (for `ContentType=cctBool`), i.e. `TRUE` or `FALSE`, or
- `ErrorValue` (for `ContentType=cctError`).

Due to usage of a variant record most of these values are overlapping, i.e. modification of `NumberValue` affects also the other values. Therefore, always respect the `ContentType` when accessing the `TCell` record directly (the worksheet methods discussed below consider this automatically).

The field `Flags` tells whether additional data are associated with the cell which are not included in the cell record usually to save memory:

```
type
  TsCellFlag = (cfHasComment, cfHyperlink, cfMerged, cfHasFormula, cf3dFormula);
  TsCellFlags = set of TsCellFlag;
```

- `cfHasComment`: A comment record can be found in the `Comments` of the worksheet.
- `cfHyperlink`: The cell is contains a hyperlink stored in the `Hyperlinks` of the worksheet.
- `cfMerged`: The cell belongs to a merged block and extends across several cells.
- `cfHasFormula`: The cell is associated with a formula which is stored in the `Formulas` of the worksheet.
- `cf3dFormula`: The formula associated with the cell contains elements referencing other sheets of the same workbook.

> **Note:** After calculation of a formula or after reading of a file, the `ContentType` of the formula cell is converted to that of the result. Then the presence of a formula can only be detetected by calling the function `HasFormula(cell)` for the cell to be queried (`cell: PCell`) in unit ''fpsUtils''; this function checks the presence of the element `cfHasFormula` in the cell flags.

How to add and read data to/from a cell

Adding values to a cell is most easily accompished by using one of the `WriteXXXX` methods of the worksheet. The most important ones are:

```
type
  TsWorksheet = class
    ...
    { Writing of currency values }
    function WriteCurrency(ARow, ACol: Cardinal; AValue: Double;
      ANumFormat: TsNumberFormat = nfCurrency; ADecimals: Integer = 2;
      ACurrencySymbol: String = '?'; APosCurrFormat: Integer = -1;
      ANegCurrFormat: Integer = -1): PCell; overload;
    procedure WriteCurrency(ACell: PCell; AValue: Double;
      ANumFormat: TsNumberFormat = nfCurrency; ADecimals: Integer = -1;
      ACurrencySymbol: String = '?'; APosCurrFormat: Integer = -1;
      ANegCurrFormat: Integer = -1); overload;
    function WriteCurrency(ARow, ACol: Cardinal; AValue: Double;
      ANumFormat: TsNumberFormat; ANumFormatString: String): PCell; overload;
    procedure WriteCurrency(ACell: PCell; AValue: Double;
      ANumFormat: TsNumberFormat; ANumFormatString: String); overload;

    { Writing of date/time values }
    function WriteDateTime(ARow, ACol: Cardinal; AValue: TDateTime): PCell; overload;
    procedure WriteDateTime(ACell: PCell; AValue: TDateTime); overload;
    function WriteDateTime(ARow, ACol: Cardinal; AValue: TDateTime;
      ANumFormat: TsNumberFormat; ANumFormatStr: String = ''): PCell; overload;
    procedure WriteDateTime(ACell: PCell; AValue: TDateTime;
      ANumFormat: TsNumberFormat; ANumFormatStr: String = ''); overload;
    function WriteDateTime(ARow, ACol: Cardinal; AValue: TDateTime;
      ANumFormatStr: String): PCell; overload;
    procedure WriteDateTime(ACell: PCell; AValue: TDateTime;
      ANumFormatStr: String); overload;

    { Writing of number values }
    function WriteNumber(ARow, ACol: Cardinal; ANumber: double): PCell; overload;
    procedure WriteNumber(ACell: PCell; ANumber: Double); overload;
    function WriteNumber(ARow, ACol: Cardinal; ANumber: double;
      ANumFormat: TsNumberFormat; ADecimals: Byte = 2): PCell; overload;
    procedure WriteNumber(ACell: PCell; ANumber: Double;
      ANumFormat: TsNumberFormat; ADecimals: Byte = 2); overload;
    function WriteNumber(ARow, ACol: Cardinal; ANumber: double;
      ANumFormat: TsNumberFormat; ANumFormatString: String): PCell; overload;
    procedure WriteNumber(ACell: PCell; ANumber: Double;
      ANumFormat: TsNumberFormat; ANumFormatString: String); overload;

    { Writing of string values }
    function WriteText(ARow, ACol: Cardinal; AText: ansistring;
      ARichTextParams: TsRichTextParams = nil): PCell; overload;
    procedure WriteText(ACell: PCell; AText: String;
      ARichTextparams: TsRichTextParams = nil); overload;

    // the old string methods "WriteUTF8Text" are deprecated now
    ...
```

Some of these methods exist in overloaded versions in which cell formatting parameters can be added together with the cell value. Correspondingly to writing, there is also a number of `worksheet` methods for reading the cell values:

```
type
  TsWorksheet = class
    ...
    { Reading cell content as a string }
    function  ReadAsText(ARow, ACol: Cardinal): string; overload;
    function  ReadAsText(ACell: PCell): string; overload;
    function  ReadAsText(ACell: PCell; AFormatSettings: TFormatSettings): string; overload;

    { Reading cell content as a number }
    function  ReadAsNumber(ARow, ACol: Cardinal): Double; overload;
    function  ReadAsNumber(ACell: PCell): Double; overload;
    function  ReadNumericValue(ACell: PCell; out AValue: Double): Boolean;

    { Reading cell content as a date/time value }
    function  ReadAsDateTime(ARow, ACol: Cardinal; out AResult: TDateTime): Boolean; overload;
    function  ReadAsDateTime(ACell: PCell; out AResult: TDateTime): Boolean; overload;
    ...
```

Index to Cell format

`FormatIndex` is the index of the cell format record. It describes the formatting attributes of a cell. These records are collected by an internal list of the workbook and are defined like this:

```
type
  TsCellFormat = record
    FontIndex: Integer;
    TextRotation: TsTextRotation;
    HorAlignment: TsHorAlignment;
    VertAlignment: TsVertAlignment;
    Border: TsCellBorders;
    BorderStyles: TsCellBorderStyles;
    Background: TsFillPattern;
    NumberFormatIndex: Integer;
    NumberFormat: TsNumberFormat;
    NumberFormatStr: String;
    BiDiMode: TsBiDiMode;           // bdDefault, bdLTR {left-to-right}, bdRTL {right-to-left}
    Protection: TsCellProtection;   // cpLockCell, cpHideFormulas
    UsedFormattingFields: TsUsedFormattingFields;
      //uffTextRotation, uffFont, uffBold, uffBorder, uffBackground, uffNumberFormat, uffWordWrap, uffHorAlign, uffVer
  end;
```

- `FontIndex`: text font by specifying the index in the workbook's font list
- `TextRotation`: specifies whether the cell text is written horizontally or vertically
- `HorAlignment`: left-aligned, horizontally centered, or right-aligned text
- `VertAlignment`: top, bottom or vertically centered text
- `Border`: a set of flags indicating that - if set - a border line is drawn at the left, top, right, or bottom cell edge. The lines are drawn according to the `BorderStyles` which define the linestyle and color of the border.
- `Background`: a record defining the background fill of a cell (pattern style, pattern color, and background color - see chapter on cell background below).
- `NumberFormat` and `NumberFormatStr` specify how number or date/time values are formatted (e.g., number of decimal places, long or short date format, etc.).
- Only those format attributes for which a flag is set in the `UsedFormattingFields` are considered when formatting a cell. If a flag is not included then the corresponding attribute is ignored and replaced by its default value.

For specifying a format for a given cell call the corresponding the worksheet method `WriteXXXX`, for retrieving a format call `ReadXXXX`. These methods usually get a pointer to the cell as a parameter, but there are also overloaded versions which accept the row and column index. Moreover, formatting styles can also be applied directly to the cell by using a record helper implemented in unit *fpsCell*.

See cell formatting below for a more detailed description.

Columns and rows

Column and row records are added for each column and row having a non-default properties:

```
type
  TCol = record
    Col: Cardinal;
    Width: Single;
    ColWidthType: TsColWidthType;      // = (cwtDefault, cwtCustom)
    FormatIndex: Integer;
    Options: TsColRowOptions;          // set of [croHidden, croPageBreak]
  end;
  PCol = ^TCol;

  TRow = record
    Row: Cardinal;
    Height: Single;
    RowHeightType: TsRowHeightType;    // = (rhtDefault, rhtCustom, rhtAuto)
    FormatIndex: Integer;
    Options: TsColRowOptions;          // set of [croHidden, croPageBreak]
    Hidden: Boolean;
    PageBreak: Boolean;
  end;
  PRow = ^TRow;
```

Column width / row height

Column widths and **row heights** can be specified in a variety of units defined by the type `TsSizeUnits = (suChars, suLines, suMillimeters, suCentimeters, suPoints, suInches)`. `suChars` refers to the count of `0` characters fitting into the column width - this is the way how Excel defines column widths. `suLines` is the number of lines fitting into the row height. Both units are based on the character size of the workbook's default font. The other units are conventional physical length units (1 cm = 10 mm, 1 inch = 25.4 mm = 72 pt). Fractional values are accepted. The workbook and worksheets store lengths internally in millimeters (`MyWorkbook.Units`).

The Office applications usually adjust the row heights automatically according to the font or text rotation of the cell content. This case is identified by `RowHeightType` having the value `rhtAuto`. Since the worksheet cannot calculate text size very accurately automatic row heights are not written by FPSpreadsheet; they are replaced by the **default row height**. The default row height is also used if a row is empty, i.e. does not contain any data cells. Its value can be changed by calling the worksheet's `WriteDefaultRowHeight()` or by using the worksheet property `DefaultRowHeight`. In `WriteDefaultRowHeight`, the units must be specified while in `DefaultRowHeight` they are assumed to be lines. Similarly, the **default column width** can be specified by `WriteDefaultColWidth()` or the property `DefaultColWidth` (in characters).

In order to overrun automatic and default row heights call the worksheet method `WriteRowHeight()`. These row records are identified by `RowHeightType` having the value `rhtCustom`. In the same way the width of columns can be set to a specific value by calling `WriteColWidth()`. `ColWidthType` of these columns is `cwtCustom`.

The height/width of a particular row/column can be retrieved by means of the methods `GetRowHeight` or `GetColWidth`. Note that these methods return the default row heights/column widths if there are no `TRow`/`TCol` records.

```
type TsWorksheet = class
  ...
  { Set row height }
  procedure WriteRowHeight(ARowIndex: Cardinal; AHeight: Single; AUnits: TsSizeUnits);
  { Set column width }
  procedure WriteColWidth(AColIndex: Cardinal; AWidth: Single; AUnits: TsSizeUnits);
  { Set default row height }
  procedure WriteDefaultRowHeight(AHeight: Single; AUnits: TsSizeUnits);
  {Set default cokumn width }
  procedure WriteDefaultColWidth(AWidth: Single; AUnits: TsSizeUnits);

  { Return row height }
  function GetRowHeight(ARowIndex: Cardinal; AUnits: TsSizeUnits): Single;
  { Return column width }
  function GetColWidth(AColIndex: Cardinal; AUnits: TsSizeUnits): Single;
  { Return default row height }
  function ReadDefaultRowHeight(AUnits: TsSizeUnits): Single;
  { Return default column width }
  function ReadDefaultColWidth(AUnits: TsSizeUnits): Single;

  property DefaultRowHeight: Single;  // in lines
  property DefaultColWidht: Single;   // in characters
```

There are also overloaded versions of these methods which do not require the `AUnits` parameter. In this case, row heights are defined in terms of line count, and column widths are defined in terms of character count. Note that these variants are from previos versions and are deprecated now.

Column and row formats

The `FormatIndex` element of the row and column records format applied to the entire row or column. Together with cells, these formats are stored as `TsCellFormat` records in an internal workbook list, `FormatIndex` is the index of the format properties into this list. See details in section Cell formatting. **Row and column formats** are primarily applied to empty cells, but if a new cell is added it will automatically get the format of the row or column. (If both row and column have different formats then the row format will be used).

Here is a listing of the worksheet methods available for column and row formatting:

```
type TsWorksheet = class
  // Assign a format to a column or row
  procedure WriteColFormatIndex(ACol: Cardinal; AFormatIndex: Integer);
  procedure WriteRowFormatIndex(ARow: Cardinal; AFormatIndex: Integer);

  // Query the format of a column or row
  function  GetColFormatIndex(ACol: Cardinal): Integer;
  function  GetRowFormatIndex(ARow: Cardinal): Integer;

  // Returns the font assigned to a column or row
  function  ReadColFont(ACol: PCol): TsFont;
  function  ReadRowFont(ARow: PRow): TsFont;

  // Check whether any column or row uses a special format
  function  HasRowFormats: Boolean;
  function  HasColFormats: Boolean;
```

Hidden columns / rows

Adding the flag `croHidden` to the row's or column's `Options` hides the row or column in the Office application (or in TsWorksheetGrid). The following worksheet methods are helpers to handle **row/column visibility**:

```
type TsWorksheet = class
  ...
  { Hide column/row }
  procedure HideCol(ACol: Cardinal);
  procedure HideRow(ARow: Cardinal);

  { Show a previously hidden column/row }
  procedure ShowCol(ACol: Cardinal);
  procedure ShowRow(ARow: Cardinal);

  { Check whether column/row is hidden }
  function  ColHidden(ACol: Cardinal): Boolean;
  function  RowHidden(ARow: Cardinal): Boolean;
```

Page breaks

The flag `croPageBreak` can be added to the row's or column's `Options` in order to force a **page break** before the corresponding row or column when the worksheet is printed by the Office applications. Note that FPSpreadsheet itself does not support printing. The following worksheet methods help with page breaks:

```
type TsWorksheet = class
  ...
  { Enforce a page break before the specified column/row }
  procedure AddPageBreakToCol(ACol: Cardinal);
  procedure AddPageBreakToRow(ARow: Cardinal);

  { Removes a page break previously added to a column/row }
  procedure RemovePageBreakFromCol(ACol: Cardinal);
  procedure RemovePageBreakFromRow(ARow: Cardinal);

  { Checks whether a page break is forced before the specified column/row }
  function IsPageBreakCol(ACol: Cardinal): Boolean;
  function IsPageBreakRow(ARow: Cardinal): Boolean;
```

Events

Worksheets and workbooks fire a series of events such as OnChangeCell, OnChangeFont, etc. The events usually are intended for interaction with the visual spreadsheet controls. If you need to attach your own handlers you should make sure that the original handler is called, at least in a gui program using the spreadsheet controls. Or you use the events provided by the visual controls themselves, e.g. `WorksheetGrid.OnEditingDone` instead of `WorksheetGrid.Worksheet.OnChangeCell`.

Formulas

Two kinds of formulas are supported by FPSpreadsheet:

- **String formulas**: These are written in strings just like in the office applications, for example `"=ROUND(A1+B1,0)"`. They are used internally in the files of Open/LibreOffice and Excel *.xlsx*.
- **RPN formulas** are used internally by the binary *.xls* Excel files. They are written in Reverse Polish Notation (RPN), for example: `A1, B1, Add, 0, ROUND`. If a spreadsheet containing formulas is to be saved in a binary Excel format, the RPN formulas required are generated automatically.

FPSpreadsheet can convert between string and rpn formulas. Formulas in both types can be calculated.

In older versions of FPSpreadsheet, formulas were stored directly in the cell record. This was given up to have parsed formulas available for faster calculation. The formulas are stored in the tree `Formulas` of the worksheet. The formula record contains the row and column index of the cell to which the formula belongs, the string representation of the formula, as well as the parser tree for quick evaluation.

FPSpreadsheet supports the majority of the formulas provided by the common spreadsheet applications. However, when reading a file created by these applications, there is always a chance that an unsupported formula is contained. To avoid crashing of fpspreadsheet, reading of formulas is disabled by default; the cell displays only the result of the formula written by the Office application. To activate reading of formulas add the element `boReadformulas` to the workbook's `Options` before opening the file. If an error occurs in this case the reader normally catches the exception, writes the exception message into the workbook's errorlog and continues reading. If you want reading to stop you must add the `boAbortReadingOnFormulaError` to the workbook `Options`.

Formulas can link to data in other sheets of the same workbook ("3d formulas") by applying the Excel syntax (see below). External links to spreadsheets in other files are not supported.

Calculation of formulas is normally not needed when a file is written by FPSpreadsheet for opening in an Office application because that automatically calculates the formula results. If the same file, however, is openend by an application based on FPSpreadsheet the calculated cells would be empty because the formulas are not automatically calculated by default. To activate calculation of formulas before writing a spreadsheet to file you have to add the option `boCalcBeforeSaving` to the workbook's `Options`.

If FPSpreadshet is used in an interactive application (such as the *spready* demo found in the *examples* folder of the FPSpreadsheet installation) it is desirable to calculate formulas automatically whenever formula strings or cell values are changed by the user. This can be achieved by the option `boAutoCalc` in the workbook's `Options`.

The most general setting regarding formulas, therefore, is

```
  MyWorkbook.Options := MyWorkbook.Options + [boReadFormulas, boCalcBeforeSaving, boAutoCalc];
```

Calculation of formulas can be triggered manually by calling the method `CalcFormulas` of the worksheet or workbook. The latter is absolutely required when the workbook contains 3d formulas where the result of one cell can affect cells in other sheets. If there are only within-sheet formulas then the worksheet's `CalcFormulas` is sufficient.

String formulas

String formulas are written in the same way as in the Office applications. The worksheet method for creating a string formula is `WriteFormula`:

```
var
  MyWorksheet: TsWorksheet;
//...
  MyWorksheet.WriteFormula(0, 1, '=ROUND(A1+B2+1.215,0)');
  // By default, use dot as decimal and comma as list separator!
```

A few notes on **syntax**:

- The leading = character which identifies formulas in the Office applications is not absolutely necessary here and can be dropped. The formula is stored in the cell record without it.
- The case of the formula name is ignored.
- Spaces can be added for better readability, but they will be lost when saving.
- Strings must be enclosed in double quotes.
- The corner points of a cell range must be separated by a colon (":"), e.g. `A1:C3`. Unordered ranges will be rearranged when the formula is parsed, i.e. `C3:A1` becomes `A1:C3`.
- Links to other worksheets must follow the Excel syntax which separates the sheetname and cell address by a "!". An single cell, for example, can be linked by `Sheet1!A1`. An range of sheets must be placed before the cell or cell range, e.g. `Sheet1:Sheet2!A1:C3`. Note that the Open/LibreOffice syntax with a separating point and reference to the corner points of the 3d box (i.e., `Sheet1.A1:Sheet2.C3`) is not supported. Note also that the worksheet(s) to which the formula links must exist at the time when the formula is added; otherwise the link will be replaced by the error code `#REF!`, and the formula will not be usable even if the missing sheet is added later.
- Normally, floating point numbers must be entered with a dot as decimal separator, and a comma must be used to separate function arguments.
- Setting the optional parameter `ALocalized` of the worksheet methods `WriteFormula` to `TRUE`, however, allows to use localized decimal and list separators taken from the workbook's `FormatSettings` - see *spready* demo.

```
var
  MyWorksheet: TsWorksheet;
//...
  MyWorksheet.WriteFormula(0, 1, '=ROUND(A1+B2+1,215;0)', true);
  // Because of the "true" the formula parser accepts the comma as decimal and the
  // semicolon as list separator if the workbook's FormatSettings are set up like this.
```

Use the worksheet methods `ReadFormula` or `ReadFormulaAsString` to retrieve the string formula assigned to the cell. The pointer to the cell must be given as a parameter. The latter function accepts an additional (boolean) parameter `ALocalized` to use the decimal and list separators of the workbook's `FormatSettings` for creation of the formula string.

```
var
  MyWorksheet: TsWorksheet;
  cell: PCell;
//...
  cell := MyWorksheet.FindCell(0, 1);
  WriteLn('The formula in internal format is ', MyworkSheet.ReadFormula(cell));
  WriteLn('The localized formula is ', MyWorksheet.ReadFormulaAsString(cell, true));
//-----------------------------------------------------------------
// For the previous example, this will result in the following output
The formula in internal format is ROUND(A1+B2+1.215,0)
The localized formula is ROUND(A1+B2+1,215;0)
```

RPN formulas

At application level, string formulas are mainly used, and RPN formulas are of little practical importance. Therefore, documentation of RPN formulas has been removed from this main FPSpreadsheet wiki and can be found in the article "RPN Formulas in FPSpreadsheet".

Shared formulas and array formulas

- **Shared formulas** are only supported for reading (from Excel files).
- **Array formulas** are not supported, currently.

List of built-in formulas

FPSpreadsheet supports more than 80 built-in formulas. In order not to blow up this wiki page too much documentation of these formulas has been moved to the separate document "List of formulas".

To learn more about the functions available, look at file *testcases_calcrpnformula.inc* in the *tests* folder of the FPSpreadsheet installation where every function is included with at least one sample.

Extending FPSpreadsheet by user-defined formulas

Although the built-in formulas cover most of the applications there may be a need to access a formula which is available in the Office application, but not in FPSpreadsheet. For this reason, the library supports a registration mechanism which allows to add user-defined functions to the spreadsheets. This can be done by calling the procedure `RegisterFunction` from the unit *fpsExprParser*:

```
procedure RegisterFunction(const AName: ShortString; const AResultType: Char;
  const AParamTypes: String; const AExcelCode: Integer; ACallBack: TsExprFunctionCallBack); overload;

procedure RegisterFunction(const AName: ShortString; const AResultType: Char;
  const AParamTypes: String; const AExcelCode: Integer; ACallBack: TsExprEventCallBack); overload;
```

- `AName` specifies the name under which the function will be called in the spreadsheet. It must match the name of the formula in the Office application.
- `AResultType` is a character which identifies the data type of the function result:
  - `'F'` - floating point number
  - `'I'` - integer
  - `'D'` - date/time
  - `'B'` - boolean
  - `'S'` - string
  - `'C'` - cell address, e.g. 'A1'
  - `'R'` - cell range address, e.g. 'A1:C3'
- `AParamTypes` is a string in which each character identifies the data type of the corresponding argument. In addition to the list shown above the following symbols can be used:
  - `'?'` - any type
  - `'+'` - must be the last character. It means that the preceding character is repeated indefinitely. This allows for an arbitrary argument count. Please note, however, that Excel supports only up to 30 arguments.
  - lowercase `'f', 'i', 'd', 'b', 's'` indicate optional parameters of the type explained above. Of course, uppercase symbols cannot follow lower-case symbols.
- `AExcelCode` is the identifier of the function in xls files. See "OpenOffice Documentation of the Microsoft Excel File Format", section 3.11, for a list.
- `ACallback` identifies which function is called by FPSpreadsheet for calculation of the formula. It can either be a procedure or an event handler.

```
type
  TsExprFunctionCallBack = procedure (var Result: TsExpressionResult; const Args: TsExprParameterArray);
  TsExprFunctionEvent = procedure (var Result: TsExpressionResult; const Args: TsExprParameterArray) of object;
```

The `TsExpressionResult` is a variant record containing result or argument data of several types:

```
type
  TsResultType = (rtEmpty, rtBoolean, rtInteger, rtFloat, rtDateTime, rtString,
    rtCell, rtCellRange, rtError, rtAny);

  TsExpressionResult = record
    Worksheet       : TsWorksheet;
    ResString       : String;
    case ResultType : TsResultType of
      rtEmpty       : ();
      rtError       : (ResError       : TsErrorValue);
      rtBoolean     : (ResBoolean     : Boolean);
      rtInteger     : (ResInteger     : Int64);
      rtFloat       : (ResFloat       : TsExprFloat);
      rtDateTime    : (ResDateTime    : TDatetime);
      rtCell        : (ResRow, ResCol : Cardinal);
      rtCellRange   : (ResCellRange   : TsCellRange);
      rtString      : ();
  end;

  TsExprParameterArray = array of TsExpressionResult;
```

As an example we show here the code for the `CONCATENATE()` formula which joins two or more strings:

```
const
  INT_EXCEL_SHEET_FUNC_CONCATENATE = 336;
...
  RegisterFunction('CONCATENATE', 'S', 'S+', INT_EXCEL_SHEET_FUNC_CONCATENATE, @fpsCONCATENATE);

procedure fpsCONCATENATE(var Result: TsExpressionResult; const Args: TsExprParameterArray);
// CONCATENATE( text1, text2, ... text_n )
var
  s: String;
  i: Integer;
begin
  s := '';
  for i:=0 to Length(Args)-1 do
  begin
    if Args[i].ResultType = rtError then
    begin
      Result := ErrorResult(Args[i].ResError);
      exit;
    end;
    s := s + ArgToString(Args[i]);
    // "ArgToString" simplifies getting the string from a TsExpressionResult as
    // a string may be contained in the ResString and in the ResCell fields.
    // There is such a function for each basic data type.
  end;
  Result := StringResult(s);
  // "StringResult" stores the string s in the ResString field of the
  // TsExpressionResult and sets the ResultType to rtString.
  // There is such a function for each basic data type.
end;
```

There is a worked-out example (*demo_formula_func.pas*) in the folder *examples/other* of the FPSpreadsheet installation. In this demo, four financial functions (`FV()`, `PV()`, `PMT()`, `RATE()`) are added to FPSpreadsheet.

Unsupported formulas

Sometimes it is required to create files for the Office applications with formulas not supported by fpspreadsheet. This is possible to some extent when the workbook option `boIgnoreFormulas` is active. Then any arbitrary formula can be written to a cell, and the formula is not checked and not evaluated. The workbook can be written to an .ods or .xlsx file. The old xls file format cannot be used because the formula would have to be parsed to create the rpn formula needed.

In folder *examples/other* you can find the sample project *demo_ignore_formula* which creates an ods file with references to another data file - external references normally are not supported by fpspreadsheet, and thus the ignore-formulas workaround must be used. Note that this example does not work with xlsx because Excel writes information on the external links to separate xml files within the xlsx container.

Cell formatting

Number and date/time formats

Numbers and date/time values can be displayed in a variety of formats. In FPSpreadsheet this can be achieved in two ways:

- using **built-in number formats** by specifiying a value for the `NumberFormat` of the cell
- using a custom **format string**.

Number formats can be specified by these worksheet methods:

```
type
  TsWorksheet = class
  public
    // Set number formats alone
    function WriteNumberFormat(ARow, ACol: Cardinal; ANumberFormat: TsNumberFormat;
      const AFormatString: String = ''): PCell; overload;
    procedure WriteNumberFormat(ACell: PCell; ANumberFormat: TsNumberFormat;
      const AFormatString: String = ''); overload;

    function WriteNumberFormat(ARow, ACol: Cardinal; ANumFormat: TsNumberFormat;
      ADecimals: Integer; ACurrencySymbol: String = ''; APosCurrFormat: Integer = -1;
      ANegCurrFormat: Integer = -1): PCell; overload;
    procedure WriteNumberFormat(ACell: PCell; ANumFormat: TsNumberFormat;
      ADecimals: Integer; ACurrencySymbol: String = '';
      APosCurrFormat: Integer = -1; ANegCurrFormat: Integer = -1); overload;

    function WriteFractionFormat(ARow, ACol: Cardinal; AMixedFraction: Boolean;
      ANumeratorDigits, ADenominatorDigits: Integer): PCell; overload;
    procedure WriteFractionFormat(ACell: PCell; AMixedFraction: Boolean;
      ANumeratorDigits, ADenominatorDigits: Integer); overload;

    // Set date/time formats alone
    function WriteDateTimeFormat(ARow, ACol: Cardinal; ANumberFormat: TsNumberFormat;
      const AFormatString: String = ''): PCell; overload;
    procedure WriteDateTimeFormat(ACell: PCell; ANumberFormat: TsNumberFormat;
      const AFormatString: String = ''); overload;


    // Set cell values and number formats in one call

    // number values
    function WriteNumber(ARow, ACol: Cardinal; ANumber: double;
      AFormat: TsNumberFormat = nfGeneral; ADecimals: Byte = 2;
      ACurrencySymbol: String = ''): PCell; overload;
    procedure WriteNumber(ACell: PCell; ANumber: Double; AFormat: TsNumberFormat = nfGeneral;
      ADecimals: Byte = 2; ACurrencySymbol: String = ''); overload;
    function WriteNumber(ARow, ACol: Cardinal; ANumber: double;
      AFormat: TsNumberFormat; AFormatString: String): PCell; overload;
    procedure WriteNumber(ACell: PCell; ANumber: Double;
      AFormat: TsNumberFormat; AFormatString: String); overload;

    // date/time values
    function WriteDateTime(ARow, ACol: Cardinal; AValue: TDateTime;
      AFormat: TsNumberFormat = nfShortDateTime; AFormatStr: String = ''): PCell; overload;
    procedure WriteDateTime(ACell: PCell; AValue: TDateTime;
      AFormat: TsNumberFormat = nfShortDateTime; AFormatStr: String = ''); overload;
    function WriteDateTime(ARow, ACol: Cardinal; AValue: TDateTime;
      AFormatStr: String): PCell; overload;
    procedure WriteDateTime(ACell: PCell; AValue: TDateTime;
      AFormatStr: String); overload;

    // currency values
    function WriteCurrency(ARow, ACol: Cardinal; AValue: Double;
      AFormat: TsNumberFormat = nfCurrency; ADecimals: Integer = 2;
      ACurrencySymbol: String = '?'; APosCurrFormat: Integer = -1;
      ANegCurrFormat: Integer = -1): PCell; overload;
    procedure WriteCurrency(ACell: PCell; AValue: Double;
      AFormat: TsNumberFormat = nfCurrency; ADecimals: Integer = -1;
      ACurrencySymbol: String = '?'; APosCurrFormat: Integer = -1;
      ANegCurrFormat: Integer = -1); overload;
    function WriteCurrency(ARow, ACol: Cardinal; AValue: Double;
      AFormat: TsNumberFormat; AFormatString: String): PCell; overload;
    procedure WriteCurrency(ACell: PCell; AValue: Double;
      AFormat: TsNumberFormat; AFormatString: String); overload;
  ...
```

Built-in number formats

The **built-in formats** are defined by the enumeration `TsNumberFormat`. In spite of its name, the elements cover both number and date/time values:

```
type
  TsNumberFormat = (
    // general-purpose for all numbers
    nfGeneral,
    // numbers
    nfFixed, nfFixedTh, nfExp, nfPercentage, nfFraction,
    // currency
    nfCurrency, nfCurrencyRed,
    // dates and times
    nfShortDateTime, nfShortDate, nfLongDate, nfShortTime, nfLongTime,
    nfShortTimeAM, nfLongTimeAM, nfDayMonth, nfMonthYear, nfTimeInterval,
    // other (using format string)
    nfCustom);
```

- **nfGeneral** corresponds to the default formatting showing as many decimals as possible (the number `3.141592654` would be unchanged.)
- **nfFixed** limits the decimals. The number of decimal places has to be specified in the call to `WriteNumber`. Example: with 2 decimals, the number `3.141592654` becomes `3.14`.
- **nfFixedTh**: similar to `nfFixed`, but adds a thousand separator when the number is displayed as a string: The number `3.141592654` would remain like in the previous example because it is too small to show thousand separators. But the number `314159.2654` would become `314,159.26`, for 2 decimals.

- **nfExp** selects exponential presentation, i.e. splits off the exponent. The parameter `ADecimals` in `WriteNumber` determines how many decimal places are used. (The number `3.141592654` becomes `3.14E+00` in case of two decimals).
- **nfPercentage** displays the number as a percentage. This means that the value is multiplied by 100, and a percent sign is added. Again, specify in `ADecimals` how many decimal places are to be shown. (The number `3.141592654` is displayed as `314.92%`, in case of 2 decimals).
- **nfFraction** presents a number as a fraction. Details (mixed fraction?, maximum digit count for numerator or denominator) for can be specified in the worksheet method `WriteFractionFormat`.
- **nfCurrency** displays the number together with a currency symbol, and there are special rules how to display negative values (in brackets, or minus sign before or after the number). The FormatSettings of the workbook are used to define the currency sign and the way numbers are displayed (`FormatSettings.CurrencyString` for the currency symbol, `FormatSettings.CurrencyFormat` for positive, `FormatSettings.NegCurrFormat` for negative values). These settings can be overridden by specifying them in the call to `WriteCurrency` directly.
- **nfCurrendyRed** like `nfCurrency`, in addition negative values are displayed in red.
- **nfShortDateTime** presents the DateTimeValue of the cell in "short date/time format", i.e. days + two digit months + two digit year + hours + minutes, no seconds. The order of the date parts is taken from the workbook's `FormatSettings`. This applies also to the other date/time formats.
- **nfShortDate** creates a date string showing day + two-digit month + two-digit year
- **nfShortTime** creates a time string showing hours + minutes.
- **nfLongTime**, similar, but includes seconds as well
- **nfShortTimeAM**, similar to `nfShortTime`, but uses the AM/PM time format, i.e. hours go up to 12, and AM or PM is added to specify morning or evening/afternoon.
- **nfLongTimeAM**, like `nfShortTimeAM`, but includes seconds
- **nfTimeInterval**, like `nfLongTime`, but there can be more than 24 hours. The interval can also be expressed in minutes or seconds, if the format strings `[n]:ss`, or `[s]`, respectively, are used.
- **nfCustom** allows to specify a dedicated formatting string.

As already noted the workbook has a property `FormatSettings` which provides additional information to control the resulting formatting. This is essentially a copy of the `DefaultFormatSettings` declared in the *sysutils* unit (the elements `LongDateFormat` and `ShortDateFormat` are slightly modified to better match the default settings in the main spreadsheet applications). The main purpose of the `FormatSettings` is to add a simple way of localization to the number formats.

Number format strings

In addition to these pre-defined formats, more specialized formatting can be achieved by using the format constant `nfCustom` along with a dedicated **format string**. The format string is constructed according to Excel syntax which is close to the syntax of fpc's `FormatFloat` and `FormatDateTime` commands (accepted as well, see the online-help for these functions).

Here is a basic list of the symbols used:

Symbol	Meaning	Format string: Number --> Output string
`General`	Displays all decimal places of the number	`'General'`: 1.2345678 --> `'1.2345678'`

`0`	Displays insignificant zeros if a number has less digits than there are zeros in the format. If used for decimal places then the number is rounded to as many decimal places as 0s are found.	`'000'`: 1 --> `'001'`
		`'0.0'`: 1 --> `'1.0'`
		`'0.0'`: 1.2345678 --> `'1.2'`

`*`	Like "0" above, but does not display insignificant zeros.	`'0.*'`: 1 --> `'1.'` `'0.*'`: 1.2345678 --> `'1.2'`
`?`	Like "0" above, but insignificant zeros are replaced by space characters. Good for aligning decimal points and fractions	`'??0'`: 1 --> `' 1'` `'0.0??'`: 1 --> `'1.0 '`
`.`	Decimal separator; will be replaced by the value used in the `DecimalSeparator` of the workbook's `FormatSettings`	`'0.00'`: 8.9 --> `'8.90'`
`,`	Thousand separator; will be replaced by the value used in the `ThousandSeparator` of the workbook's `FormatSettings`. If at the end of a number formatting sequence the displayed value is divided by 1000.	`'#,##0.00'`: 1200 --> `'1,200.00'` `'0.00,'`: 1200 --> `'1.20'`
`E+`, `e+`	Displays a number in exponential format. The digits used for the exponent are defined by the number of zeros added the this symbol. The sign of the exponent is shown for positive and negative exponents.	`'0.00E+00'`: 1200 --> `1.20E+03`
`E-`, `e-`	Displays a number in exponential format. The digits used for the exponent are defined by the number of zeros added the this symbol. The sign of the exponent is shown only for negative exponents.	`'0.00e-000'`: 1200 --> `1.20e003`
`%`	Displays the number as a "percentage", i.e. the number is multiplied by 100 and a `%` sign is added.	`'0.0%'`: 0.75 --> `75.0%`

`/`	This symbol has two meanings: if the cell represents a "number" then the slash indicates formatting as fraction, place holders for numerator and denominator must follow. If the cell represents a "date/time" then the slash indicates the date separator which will be replaced by the `DateSeparator` of the workbook's `FormatSettings`	`'#/#'`: 1.5 --> `'3/2'`
		`'# #/#'`: 1.5 --> `'1 1/2'`
		`'# #/16'`: 1.5 --> `'1 8/16'`
		also: see date/time examples below

`:`	Separator between hours, minutes and seconds of a date/time value. Will be replaced by the `TimeSeparator` of the workbook's `FormatSettings`.	see examples below
`yyyy`	Place holder for the year of a date/time value. The year is displayed as a four-digit number.	`'yyyy/mm/dd'`: Jan 3, 2012 --> `'2012-01-03'` In this example, the `DateSeparator` is a dash character (-).
`yy`	Place holder for the year of a date/time value. The year is displayed as a two-digit number.	`'yy/mm/dd'`: Jan 3, 2012 --> `'12-01-03'`
`m`	Place holder for the month of a date/time value. The month is shown as a number without extra digits. Please note that the `m` code can also be interpreted as the "minutes" of a time value (see below).	`'yyyy/m/dd'`: Jan 3, 2012 --> `'2012-1-03'`
`mm`	Place holder for the month of a date/time value. The month is shown as a two-digit number, i.e. a leading zero is added for January to September. Please note that the `mm` code can also be interpreted as the "minutes" of a time value (see below).	`'yyyy/mm/dd'`: Jan 3, 2012 --> `'2012-01-03'`
`mmm`	Place holder for the month of a date/time value. The month is displayed by its abbreviated name.	`'yyyy/mmm/dd'`: Jan 3, 2012 --> `'2012-Jan-03'`
`mmmm`	Place holder for the month of a date/time value. The month is displayed by its full name.	`'yyyy/mmm/dd'`: Jan 3, 2012 --> `'2012-January-03'`
`d`	Place holder for the day of a date/time value to be displayed as a number. The day is displayed as a simple number, without adding a leading zero.	`'yyyy/mm/d'`: Jan 3, 2012 --> `'2012-01-3'`
`dd`	Place holder for the day of a date/time value to be displayed as a number. `dd` adds a leading zero to single-digit day numbers.	`'yyyy/mm/dd'`: Jan 3, 2012 --> `'2012-01-03'`
`ddd`	Place holder for the day of a date/time value. The day is displayed as its abbreviated name.	`'dddd, yyyy/mm/ddd'`: Jan 03, 2012 --> `'Tue 2012-01-03'`
`dddd`	Place holder for the day of a date/time value. The day is displayed as its full name.	`'dddd, yyyy/mmmm/dd'`: Jan 03, 2012 --> `'Tuesday 2012-01-03'`
`h`	Place holder of the hour part of a date/time value. The hour is displayed as a simple number, without adding a leading zero.	`'h:mm'`: 0.25 --> `'6:00'`
`hh`	Place holder of the hour part of a date/time value. The hour is displayed with a leading zero if the hour is less than 10.	`'hh:mm'`: 0.25 --> `'06:00'`

[hh], or [h]	Displays elapsed time such that the hour part can become greater than 23	'[h]:mm': 1.25 --> '30:00'
m	Place holder of the minutes part of a date/time value. The minutes are shown as a simple number without adding a leading zero. Note that if the m codes are surrounded by date symbols (y, d) then they are interpreted as "month".	'h:m': 0.25 --> '6:0'
mm	Place holder of the minutes part of a date/time value. Single-digit minutes are displayed with a leading zero. Note that if the mm code is surrounded by date symbols (y, d) then it is interpreted as "month".	'h:mm': 0.25 --> '6:00'
[mm], or [m]	Displays elapsed time such that the minute part can become greater than 59	'[mm]:ss': 1.25 --> '1800:00'
s	Place holder of the seconds part of a date/time value. The seconds are displayed as a simple number, without adding a leading zero.	'hh:mm:s': 0.25 --> '06:00:0'
ss	Place holder of the seconds part of a date/time value. Single-digit seconds are displayed with a leading zero.	'hh:mm:ss': 0.25 --> '06:00:00'
[ss], or [s]	Displays elapsed time such that the seconds part can become greater than 59	'[ss]': 1.25 --> '108000'
AM/PM, am/pm, A/P, or a/p	Displays the time in the 12-hour format.	'hh:mm:ss AM/PM': 0.25 --> '6:00:00 AM'
"	The text enclosed by quotation marks is inserted into the formatted strings literally.	'yyyy"/"mm"/"dd': Jan 3, 2012 --> '2012/01/03' (i.e. the / is not replaced by the DateSeparator of the workbook).
\	The next character of the format string appears in the result string literally. The \ itself does not show up.	'yyyy\/mm\/dd': Jan 3, 2012 --> '2012/01/03'
;	A format string can contain up to three sections separated by the semicolon. The first section is used for positive numbers, the second section for negative numbers, and the third section for zero numbers. If the third section is missing then a zero value is formatted as specified in the first section. If the second section is missing as well then all values are formatted according to the first section.	'"#,##0"$";-#,##0"$";"-"': 1200 --> '1,200$' -1200 --> '1,200$' 0 --> '-'
(, and)	Sometimes used for currency values to indicate negative numbers, instead of minus sign	'#,##0"$";(#,##0)"$"': -1200 --> '(1200)$'
[red]	The formatted string is displayed in the specified color. Instead of [red], you can use accordingly [black], [white], [green], [blue], [magenta], [yellow], or [cyan]. Often used to highlight negative currency values.	'"$" #,##0.00;[red]("$"#,##0.00)': -1200 --> '($ 1200.00)'

A note on Excel dates

There is a discrepancy between FPSpreadsheet and Excel presentation of dates: Excel incorrectly assumes that the year 1900 was a leap year and thus allows for the date Febr 29 1900. FPSpreadsheet does the date calculation correctly. As consequence, dates before March 1 1900 are off by one! BTW, FPSpreadsheet handles this issue in the same way as LibreOffice Calc.

Colors

FPSpreadsheet supports colors for text, cell background, and cell borders. The basic EGA colors are declared in unit *fpstypes* as constants:

```
type
  TsColor = DWORD;

const
  scBlack        = $00000000;
  scWhite        = $00FFFFFF;
  scRed          = $000000FF;
  scGreen        = $0000FF00;
  scBlue         = $00FF0000;
  scYellow       = $0000FFFF;
  scMagenta      = $00FF00FF;
  scCyan         = $00FFFF00;
  scDarkRed      = $00000080;
  scDarkGreen    = $00008000;
  scDarkBlue     = $00800000;
  scOlive        = $00008080;
  scPurple       = $00800080;
  scTeal         = $00808000;
  scSilver       = $00C0C0C0;
  scGray         = $00808080;
  scGrey         = scGray; // redefine to allow different spelling

  // Identifier for undefined color
  scNotDefined   = $40000000;

  // Identifier for transparent color
  scTransparent = $20000000;
```

	A
1	black
2	white
3	red
4	green
5	blue
6	yellow
7	magenta
8	cyan
9	dark red
10	dark green
11	dark blue
12	olive
13	purple
14	teal
15	silver
16	gray

The TsColor represents the **rgb value** of a color, a single byte being used for the red, green, and blue components. The resulting number is in little endian notation, i.e. the red value comes first in memory: $00BBGGRR. (This is directly compatible with the color values as defined in the *graphics* unit.)

The high order byte is usually zero but is used internally to identify special color values, such as for undefined or transparent colors.

Note: In older versions of the library colors were defined as indexes into a color palette. '''THIS IS NO LONGER WORKING.'''

Unit *fpsutils* contains some useful functions for **modification of colors**:

- `function GetColorName(AColor: TsColor): String;`
  returns the name of the colors defined above, or a string showing the rgb components for other colors.
- `function HighContrastColor(AColor: TsColor): TsColor;`
  returns `scBlack` for a "bright", `scWhite` for a "dark" input color.
- `function TintedColor(AColor: TsColor; tint: Double): TsColor;`
  brightens or darkens a color by applying a factor `tint = -1..+1`, where `-1` means "100% darken", `+1` means "100% brighten", and `0` means "no change". The hue of the color is preserved.

Cell background

The cell background can be filled by predefined patterns which are identified by the record TsFillPattern:

```
type
  TsFillPattern = record
    Style: TsFillStyle;   // fill style pattern as defined below
    FgColor: TsColor;     // forground color of the fill pattern
    BgColor: TsColor;     // background color of the fill pattern
  end;

  TsFillStyle = (fsNoFill, fsSolidFill, fsGray75, fsGray50, fsGray25, fsGray12, fsGray6,
    fsStripeHor, fsStripeVert, fsStripeDiagUp, fsStripeDiagDown,
    fsThinStripeHor, fsThinStripeVert, fsThinStripeDiagUp, fsThinStripeDiagDown,
    fsHatchDiag, fsThinHatchDiag, fsThickHatchDiag, fsThinHatchHor);
```

- Use the worksheet method `WriteBackground` to assign a fill pattern to a specific cell. Besides the cell address, this method requires the type of the fill pattern (`TsFillStyle`), and the foreground and background colors as specified by their `TsColor` values.
- The fill pattern of a particular cell can be retrieved by calling the workbook method `ReadBackground`.
- The simplified method `WriteBackgroundColor` can be used to achieve a uniform background color.
- **Limitations:**
  - OpenDocument files support only uniform fills. The background color is a mixture of the foreground and background rgb components in a ratio defined by the fill pattern.
  - BIFF2 files support only a 12.5% black-and-white shaded pattern.

```
type
  TsWorksheet = class
  public
    function WriteBackground(ARow, ACol: Cardinal; AStyle: TsFillStyle; APatternColor, ABackgroundColor: TsColor): PCe
    procedure WriteBackground(ACell: PCell; AStyle: TsFillStyle; APatternColor, ABackgroundColor: TsColor); overload;

    function WriteBackgroundColor(ARow, ACol: Cardinal; AColor: TsColor): PCell; overload;
    procedure WriteBackgroundColor(ACell: PCell; AColor: TsColor); overload;

    function ReadBackground(ACell: PCell): TsFillPattern;
    function ReadBackgroundColor(ACell: PCell): TsColor; overload;
    // ...
  end;

var
  cell: PCell;
...
  // Example 1: Assign a pattern of thin, horizontal, yellow stripes on a blue background to empty cell A1 (row 0, col
  MyWorksheet.WriteBackground(0, 0, fsThinStripeHor, scYellow, scBlue);

  // Example 2: Uniform gray background color of cell B1 (row 0, column 1) containing the number 3.14
  cell := MyWorksheet.WriteNumber(0, 1, 3.14);
  MyWorksheet.WriteBackgroundColor(cell, clSilver);
```

Cell borders

Cells can be emphasized by drawing border lines along their edges or diagonal lines. There are four borders plus two diagonals enumerated in the data type `TsCellBorder`:

```
type
  TsCellBorder = (cbNorth, cbWest, cbEast, cbSouth, cbDiagUp, cbDiagDown);
  TsCellBorders = set of TsCellBorder;
```

In order to show a border line add the corresponding border to the cell's set `Borders` (type `TsCellBorders`, see above). In this way, each cell edge can be handled separately. Use the worksheet method `WriteBorders` for this purpose. This example adds top and bottom borders to the edges A1 and B1:

```
  MyWorksheet.WriteBorders(0, 0, [cbNorth, cbSouth]);  // A1: row 0, column 0
  MyWorksheet.WriteBorders(0, 1, [cbNorth, cbSourth]); // B1: row 0, column 1
```

Lines usually are drawn as thin, solid, black lines. But it is possible to modify **line style** and **color** of each line. For this purpose, the cell provides an array of `TsCellBorderStyle` records:

```
type
  TsLineStyle = (lsThin, lsMedium, lsDashed, lsDotted, lsThick, lsDouble, lsHair,
    lsMediumDash, lsDashDot, lsMediumDashDot, lsDashDotDot, lsMediumDashDotDot, lsSlantDashDot);

  TsCellBorderStyle = record
    LineStyle: TsLineStyle;
    Color: TsColor;
  end;

  TsCellBorderStyles = array[TsCellBorder] of TsCellBorderStyle;

  TsWorksheet = class
  public
    function WriteBorderStyle(ARow, ACol: Cardinal; ABorder: TsCellBorder; AStyle: TsCellBorderStyle): PCell; overload
    procedure WriteBorderStyle(ACell: PCell; ABorder: TsCellBorder; AStyle: TsCellBorderStyle); overload;

    function WriteBorderStyle(ARow, ACol: Cardinal; ABorder: TsCellBorder; ALineStyle: TsLineStyle; AColor: TsColor):
    procedure WriteBorderStyle(ACell: PCell; ABorder: TsCellBorder; ALineStyle: TsLineStyle; AColor: TsColor); overloa

    function WriteBorderColor(ARow, ACol: Cardinal; ABorder: TsCellBorder; AColor: TsColor): PCell; overload;
    procedure WriteBorderColor(ACell: PCell; ABorder: TsCellBorder; AColor: TsColor): PCell; overload;

    function WriteBorderLineStyle(ARow, ACol: Cardinal; ABorder: TsCellBorder; ALineStyle: TsLineStyle): PCell; overlo
    procedure WriteBorderLineStyle(ACell: PCell; ABorder: TsCellBorder; ALineStyle: TsLineStyle): PCell; overload;

    function WriteBorderStyles(ARow, ACol: Cardinal; const AStyles: TsCellBorderStyles): PCell; overload;
    procedure WriteBorderStyles(ACell: PCell; const AStyles: TsCellBorderStyles); overload;

    function WriteBorders(ARow, ACol: Cardinal; ABorders: TsCellBorders): PCell; overload
    procedure WriteBorders(ACell: PCell; ABorders: TsCellBorders); overload

    function ReadCellBorders(ACell: PCell): TsCellBorders;
    function ReadCellBorderStyle(ACell: PCell; ABorder: TsCellBorder): TsCellBorderStyle;
    function ReadCellBorderStyles(ACell: PCell): TsCellBorderStyles;
    ...
  end;
```

The style of a given cell border can be specified by the following methods provided by the worksheet:

- **WriteBorderStyle** assigns a cell border style record to one border of the cell. There are two overloaded versions of this method: one takes an entire `TsCellBorderStyle` record, the other one takes the individual record elements.
- **WriteBorderColor** changes the color of a given border without affecting the line style of this border.
- **WriteBorderLineStyle** sets the line style of the border only, but leaves the color unchanged.
- **WriteBorderStyles** sets the border style of all borders of a given cell at once. Useful for copying border styles from one cell to other cells.

This example adds a thin black border to the top, and a thick blue border to the bottom of cells A1 and B1:

```
var
  cellA1, cellB1: PCell;
...
  cellA1 := MyWorksheet.WriteBorders(0, 0, [cbNorth, cbSouth]);     // cell A1: row 0, column 0
  MyWorksheet.WriteBorderStyle(cellA1, cbNorth, lsThin, scBlack);
  MyWorksheet.WriteBorderStyle(cellA1, cbSouth, lsThick, scBlue);

  cellB1 := MyWorksheet.WriteBorders(0, 1, [cbNorth, cbSouth]);     // cell B1: row 0, column 1
  MyWorksheet.WriteBorderStyles(cellB1, cellA1^.BorderStyles);      // copy all border styles from cell A1 to B1
```

Note: Diagonal lines are not supported by ''sfExcel2'' and ''sfExcel5'' - they are just omitted. ''sfExcel8'' and ''sfOOXML'' use the same linestyle and color for both diagonals; when writing the border style of the diagonal-up line is assumed to be valid also for the diagonal-down line. Some writers do not support all the line styles of the `TsLineStyle` enumeration. In this case, appropriate replacement line styles are used.

Fonts

The cell text can displayed in various fonts. For this purpose, the workbook provides a list of `TsFont` items:

```
type
  TsFont = class
    FontName: String;
    Size: Single;
    Style: TsFontStyles;
    Color: TsColor;
    Position: TsFontPosition;
  end;
```

- The **FontName** corresponds to the name of the font as used by the operational system. In Windows, an example would be "Times New Roman".
- The **FontSize** is given in "points", i.e. units 1/72 inch which are commonly used in Office applications.
- The **FontStyle** is a set of the items `fssBold`, `fssItalic`, `fssStrikeout`, and `fssUnderline` which form the enumeration type `TsFontStyle`. The "normal" font corresponds to an empty set.
- The **Color** determines the foreground color of the text characters given in rgb presentation as discussed above.
- The **Position** is either `fpNormal`, `fpSuperscript`, or `fpSubscript` and indicates whether the font size should be decreased by about 1/3 and the characters should be displaced up (superscript) or down (subscript).

Every cell is provided with an **index into the font list**.

In order to assign a particular font to a cell, use one of the following methods of `TsSpreadsheet`:

```
type
  TsSpreadsheet = class
  public
    function  WriteFont(ARow, ACol: Cardinal; const AFontName: String;
      AFontSize: Single; AFontStyle: TsFontStyles; AFontColor: TsColor): Integer; overload;
    procedure WriteFont(ARow, ACol: Cardinal; AFontIndex: Integer); overload;
    function WriteFontColor(ARow, ACol: Cardinal; AFontColor: TsColor): Integer;
    function WriteFontSize(ARow, ACol: Cardinal; ASize: Integer): Integer;
    function WriteFontStyle(ARow, ACol: Cardinal; AStyle: TsFontStyles): Integer;
    // plus: overloaded versions accepting a pointer to a cell record instead of the row and column index as parameter
    // ...
  end;
```

- **WriteFont** assigns a font to the cell. If the font does not yet exist in the font list a new entry is created. The function returns the index of the font in the font list. In addition, there is an overloaded version which only takes the font index as a parameter.
- **WriteFontColor** replaces the color of the font that is currently assigned to the cell by a new one. Again, a new font list item is created if the font with the new color does not yet exist. The function returns the index of the font in the list.
- **WriteFontSize** replaces the size of the currently used font of the cell.
- **WriteFontStyle** replaces the style (normal, bold, italic, etc.) of the currently used cell font.

The workbook's font list contains at least one item which is the **default font** for cells with unmodified fonts. By default, this is 10-point "Arial". Use the workbook method `SetDefaultFont` to assign a different font to the first list item.

The font at a given index of the font list can be looked up by calling the workbook function `GetFont`. The count of available fonts is returned by `GetFontCount`.

Here is an **example** which decreases the size of all 10-point "Arial" fonts to 9-point:

```
var
  i: Integer;
  font: TsFont;
begin
  for i := 0 to MyWorkbook.GetFontCount-1 do
  begin
    font := MyWorkbook.GetFont(i);
    if (font.FontName = 'Arial') and (font.Size = 10.0) then
      font.Size := 9.0;
  end;
end;
```

Rich-text formatting

In addition to using a specific font for each cell it is also possible to specify particular font attributes for individual characters or groups of characters in each cell text. Following the Excel notation, we call this feature **Rich-text formatting** (although is has nothing in common with the "rich-text" file format).

For this purpose, unit *fpstypes* declares the type `TsRichTextParams` which is an array of `TsRichTextParam` records:

```
type
  TsRichTextParam = record
    FontIndex: Integer;
    FirstIndex: Integer;
  end;

  TsRichTextParams = array of TsRichTextParam;
```

`FontIndex` refers to the index of the font in the workbook's FontList to be used for formatting of the characters beginning at the index `FirstIndex`. Being a string character index the `FirstIndex` is 1-based.

There are two ways to add "rich-text" formatting to a cell text:

- Embed corresponding HTML format codes into the cell text. This can be done using the method `WriteTextAsHTML` of the worksheet. In order to add the text "Area (m^2)" to cell A1, pass the following HTML-coded string to this function

  ```
  MyWorksheet.WriteTextAsHTML(0, 0, 'Area (m2');
  ```

- Alternatively, the standard text writing method, `WriteText` can be called with an additional parameter specifiying the rich-text formatting parameters to be used directly:

  ```
  var
    richTextParams: TsRichTextParams;
    fnt: TsFont;
    cell: PCell;
  begin
    SetLength(rtp, 2);
    cell := MyWorksheet.GetFont(0, 0);
    fnt := MyWorksheet.ReadCellFont(cell);
    richTextParams[0].FirstIndex := 8; // The superscript groups begins with "2" which is the (1-based) character #8
    richTextParams[0].FontIndex := MyWorkbook.AddFont(fnt.FontName, fnt.Size, fnt.Style, fnt.Color, fpSuperscript);
    richTextParams[1].FirstIndex := 9; // Normal font again beginning with character #9.
    richTextParams[1].FontIndex := MyWorksheet.ReadCellFontIndex(0, 0);
    MyWorksheet.WriteUTF8Text(cell, 'Area (m2)', richTextParams);
  end;
  ```

Use the worksheet method `DeleteRichTextParams` to remove rich-text formatting from a previously formatted cell.

> **Note:** If the cell is supposed to have a font different from the worksheet's default font then this font must be written to the cell '''before''' writing the rich-text formatted text. Otherwise the font in the unformatted regions will not be up-to-date.

Text rotation

Usually text is displayed in the cells horizontally. However, it is also possible to rotate it by 90 degrees in clockwise or counterclockwise directions. In addition, there is also

an option to stack horizontal characters vertically above each other.

If you need this feature use the worksheet method `WriteTextRotation` and specify the text direction by an element of the enumeration type `TsTextRotation`:

```
type
  TsTextRotation = (trHorizontal, rt90DegreeClockwiseRotation,
    rt90DegreeCounterClockwiseRotation, rtStacked);

  TsWorksheet = class
  public
    function WriteTextRotation(ARow, ACol: Cardinal; ARotation: TsTextRotation): PCell; overload;
    procedure WriteTextRotation(ACell: PCell; ARotation: TsTextRotation); overload;

    function ReadTextRotation(ACell: PCell): TsTextRotation;
    // ...
  end;

  // example for counter-clockwise rotated text in cell A1:
  WriteTextRotation(0, 0, rt90DegreeCounterClockwiseRotation);
```

Warning: Finer degrees of rotation which may be supported by some spreadsheet file formats are ignored.

Text alignment

By default, cell texts are aligned to the left and bottom edges of the cell, except for numbers which are right-aligned. This behavior can be changed by using the worksheet methods `WriteHorAlignment` and `WriteVertAlignment`:

```
type
  TsHorAlignment = (haDefault, haLeft, haCenter, haRight, haJustified, haDistributed, haFilled);
  TsVertAlignment = (vaDefault, vaTop, vaCenter, vaBottom);

  TsWorksheet = class
  public
    function WriteHorAlignment(ARow, ACol: Cardinal; AValue: TsHorAlignment): PCell; overload;
    procedure WriteHorAlignment(ACell: PCell; AValue: TsHorAlignment); overload;
    function ReadHorAlignment(ACell: PCell): TsHorAlignment;

    function WriteVertAlignment(ARow, ACol: Cardinal; AValue: TsVertAlignment): PCell; overload;
    procedure WriteVertAlignment(ACell: PCell; AValue: TsVertAlignment); overload;
    function ReadVertAlignment(ACell: PCell): TsVertAlignment;
    // ...
  end;

  // Example: Center the text in cell A1 both horizontally and vertically
  MyWorksheet.WriteHorAlignment(0, 0, haCenter);
  MyWorksheet.WriteVertAlignment(0, 0, vaCenter);
```

Explanation of special horizontal alignments:

- `haDistributed`: The cell text is word-wrapped and its spaces are expanded so that the entire width of the cell is occupied. The text is both left- and right-justified.
- `haJustified`: like `haDistributed`, expect for the last line which remains left-justified.
- `haFilled`: the text is repeated to fill the entire cell.

Word wrap

Text which is longer than the width of a cell can wrap into several lines by calling the method `WriteWordwrap` of the spreadsheet:

```
type
  TsWorksheet = class
  public
    function WriteWordwrap(ARow, ACol: Cardinal; AValue: Boolean): PCell; overload;
    procedure WriteWordwrap(ACell: PCell; AValue: Boolean); overload;
    function ReadWordwrap(ACell: PCell): Boolean;
    //...
  end;

  // Example: activate wordwrap in cell A1
  MyWorksheet.WriteWordwrap(0, 0, true);
```

Merged cells

Like the Office applications, FPSpreadsheet supports also to feature of merging cells to a single large cell which is often used as a common header above simlar columns. Simply call `MergeCells` and pass a parameter to specify the cell range to be merged, either an Excel range string (such as `A1:D5`), or the first and last rows and columns:

```
  MyWorksheet.MergeCells('A1:D5');
  // or: MyWorksheet.MergeCells(0, 0, 4, 3);  // first row, first column, last row, last column
```

The content and format displayed for a merged range is taken from the upper left corner of the range, cell `A1` in above example. This cell is called the `MergeBase` in the library. Except for this corner cell, there must not be any other cells in the range. If there are their contents and format will be hidden.

In order to break up a merged range back up into individual cells, use the command `Unmerge` and pass any cell that is within the merged range:

```
  MyWorksheet.UnmergeCells('B1');
  // or: MyWorksheet.UnmergeCells(0, 1);   // row, column of any cell within the range
```

Merged cells can be read from/written to all file formats except for `sfCSV`, `sfExcel2` and `sfExcel5` which do not support this feature natively.

The information which cells are merged is stored in an internal list. Unlike in earlier versions it is no longer possible to access the `MergeBase` from the cell directly. Use the following functions to extract information on merged cells:

```
var
  cell, base: PCell;
  r1,c1,r2,c2: Cardinal;
...
  cell := MyWorksheet.FindCell('B1');
  if MyWorksheet.IsMerged(cell) then
  begin
    WriteLn('Cell B1 is merged.');

    MyWorksheet.FindMergedRange(cell, r1, c1, r2, c2);
    WriteLn('The merged range is ' + GetCellRangeString(r1, c1, r2, c2));

    base := MyWorksheet.FindMergeBase(cell);
    WriteLn('The merge base is cell ' + GetCellString(base^.Row, base^.Col));
  end;
```

Cell protection

This is described in a separate section below.

Miscellaneous

Disable printing of specific cells

Specific cells can be disabled from printing:

```
type
  TsWorksheet = class
  public
    function WriteDoNotPrintCell(ARow, ACol: Cardinal; AValue: boolean): PCell; overload;
    procedure WriteDoNotPrintCell(ACell: PCell; AValue: Boolean); overload;
    function  ReadDoNotPrintCell(ACell: PCell): Boolean;
```

Additional data

Cell comments

Comments can be attached to any cell by calling

```
  MyWorksheet.WriteComment(0, 0, 'This is a comment for cell A1');
```

They are stored in an internal list of the worksheet. Use the corresponding worksheet methods to access comments:

- If you want to know whether a particular cell contains a comment call the worksheet method `HasComment(cell)`.
- For retrieving a cell comment use the method `ReadComment(cell)`, or its overloaded companion `ReadComment(ARow, ACol)`.
- The total number of comments can be retrieved from `worksheet.Comments.Count`.

Hyperlinks

Hyperlinks can be attached to cells in order to link cells to other documents or other cells in the same workbook. The general syntax for creating hyperlinks is

```
procedure TWorksheet.WriteHyperlink(ARow, ACol: Cardinal; ATarget: String; ATooltip: String = '');
```

- The hyperlink target, passed as parameter `ATarget`, must be a fully qualified URI (Uniform resource identifier) consisting of a protocol phrase (e.g., `http://`, `file:///`, `mailto:`, etc.) followed by specific information such as web URL, filename, or e-mail address and an optional bookmark identification separated by the character `'#'`. An exception are internal hyperlinks which enable to jump to a cell in the current workbook; they consist of the optional worksheet name and the cell address separated by the character `'!'`.
- The optional `Tooltip` parameter is evaluated by Excel to display it in a hint window if the mouse if above the hyperlink.

Note: Hyperlinks can be added to empty cells or to cells with content. In the latter case, the displayed content is not changed by the hyperlink; in the former case the cell is converted to a label cell showing the hyperlink target. Be aware that OpenOffice/LibreOffice only does accept hyperlinks with non-text cells.

Examples:

```
// Open the web site www.google.com
  MyWorksheet.WriteText(0, 0, 'Open google');
  MyWorksheet.WriteHyperlink(0, 0, 'http://www.google.com');

  // Open the local file with the absolute path "C:\readme.txt" (assuming Windows)
  MyWorksheet.WriteHyperlink(1, 0, 'file:///c:\readme.txt');

  // Open the mail client to send a mail
  MyWorksheet.WriteText('Send mail');
  MyWorksheet.WriteHyperlink(3, 0, 'mailto:somebody@gmail.com?subject=Test');

  // Jump to a particular cell
  MyWorksheet.WriteText(5, 0, 'Jump to cell A10 on sheet2');
  MyWorksheet.WriteHyperlink(5, 0, '#Sheet2!A10');

  // Jump to cell A10 on the current sheet and display a popup hint
  MyWorksheet.WriteHyperlink(5, 0, '#A10', 'Go to cell A10');
```

Note: FPSpreadsheet does not "follow" the links, it only provides a mechanism to get access to the link data. TsWorksheetGrid from the ''laz_fpspreadsheet_visual'' package, however, fires the event `OnClickHyperlink` if a cell with an external hyperlink is clicked for fractions of a second. In the corresponding event handler, you can, for example, load a target spreadsheet, or open a web browser to display the linked web site. If the link is an internal link to another cell within the same workbook then the grid jumps to the related cell.

Defined Names

FPSpreadsheet allows to assign names to cells and cell ranges, simular to Excel and Calc. Since such "defined names" can be used in formulas the readability of formulas can be greatly improved.

A defined name is a simple class of type `TsDefinedName` with a string property `Name` and a record property `Range` of type `TsCellRange3D` which lists the row and column indices of the top/left and bottom/right corners of the cell range, as well as the worksheet indices of these two corner points.

```
type
  TsDefinedName = class
  ...
  public
    function RangeAsString(AWorkbook: TsBasicWorkbook): String;
    function RangeAsString_ODS(AWorkbook: TsBasicWorkbook): String;
    class function ValidName(AName: String): Boolean;
    property Name: String;
    property Range: TsCellRange3D;
  end;
```

A useful function may be the class function `ValidName` which checks the validity of a name:

- less than 256 characters long
- contains no spaces and no punctuation characters
- begins with a letter, with '_' or with '\'
- must not collide with cell references
- must be unique (case-insensitive)

The `TsDefinedName` methods `RangeAsString` and `RangeAsString_ODS` return the defined range's Range parameters as a string in Excel and ODS syntax, respectively.

The workbook and each worksheet have a list property `DefinedNames` which collects the defined names. The defined names stored in the workbook have **global** scope while those stored in worksheets are valid only **locally** within that sheet.

- There are overloaded methods `Add` for adding a single cell or a cell range in which the name as well as the cell/cell range parameters are provided. The methods return the index of the new item in the list.

```
var
  idxCell, idxRange: Integer;
...
  // Global defined name
  idxCell := workbook.DefinedNames.Add(AName: string; ASheetIndex: Integer; ARow, ACol: Cardinal);
  idxRange := workbook.DefinedNames.Add(AName: String; ASheetIndex1, ASheetIndex2, ARow1, ACol1, ARow2, ACol2: Cardina
  // Local defined name
  idxCell := worksheet.DefinedNames.Add(AName: string; ASheetIndex: Integer; ARow, ACol: Cardinal);
  idxRange := worksheet.DefinedNames.Add(AName: String; ASheetIndex1, ASheetIndex2, ARow1, ACol1, ARow2, ACol2: Cardin
```

- The default property `Items[AIndex]` allows to read a defined name from the list. The following code lists all global defined names:

```
var
  defName: TsDefinedName;
  i: Integer;
...
  for i := 0 to workbook.DefinedNames.Count-1 do
  begin
    defName := workbook.DefinedNames[i];
    WriteLn('Index ', i, ': Name="', defName.Name, '", Range=', defName.RangeAsString(workbook));
  end;
```

Note: Unlike Excel and ODS, local defined names are not allowed to have the same names as global defined names. Cross-worksheet local defined names are not allowed either.

Images

FPSpreadsheet supports embedding of images in worksheets. Use one of the worksheet methods `WriteImage()` to add an image to the worksheet:

```
  MyWorksheet.WriteImage(row, col, filename, offsetX, offsetY, scaleX, scaleY);
  MyWorksheet.WriteImage(row, col, stream, offsetX, offsetY, scaleX, scaleY);
  MyWorksheet.WriteImage(row, col, imageindex, offsetX, offsetY, scaleX, scaleY);
```

The upper/left corner of the image is placed at the upper/left corner of the cell in the specified row and column. The floating point parameters `offsetX`, `offsetY`, `scaleX` and `scaleY` are optional: they define an offset of this image anchor point from the cell corner and a magnification factor. The path to the image file is given as parameter `filename`. Alternatively, overloaded versions can be used which accept a stream in place of the file name or an image index in the workbook's EmbeddedObj list - use `MyWorkbook.FindEmbeddedObj(filename)` to get this index for a previously loaded image file.

Note that FPSpreadsheet needs to know the image type for successfull picture import. Currently, the types *png*, *jpg*, *tiff*, *bmp*, *gif*, *svg*, *wmf*, *emf*, and *pcx* are supported (Excel2007 cannot read imported *svg* and *pcx* images). Other formats can be registered by writing a function which determines the image size and pixel density, and by registering the new format using the procedure `RegisterImageType` - see unit *fpsImages* for examples:

```
type
  TsImageType = integer;
  TGetImageSizeFunc = function (AStream: TStream; out AWidth, AHeight: DWord; out dpiX, dpiY: Double): Boolean;

function RegisterImageType(AMimeType, AExtension: String; AGetImageSize: TGetImageSizeFunc): TsImageType;
```

Due to differences in row height and column width calculation between FPSpreadsheet and Office applications it is not possible to position images correctly. If exact image positions are important you should follow these rules:

- Predefine the widths of all columns at least up to the one containing the right edge of the image.
- Predefine the heights of all rows at least up to the one containing the lower edge of the image.
- If the workbook is to be saved in OpenDocument format add the image after changing column widths and row heights because ods anchors the image to the sheet, not to the cell (like Excel and FPSpreadsheet).
- If the exact size of the image is important make sure that it fits into a single cell.

Sorting

Cells in a worksheet can be sorted for a variety of criteria by calling the `Sort` method of the worksheet. This method takes a `TsSortParams` record and the edges of the cell rectangle to be sorted as parameters; in an overloaded version, the cell rectangle can also be specified by means of an Excel-type range string (e.g. `'A1:G10'`):

```
type
  TsWorksheet = class
    // ...
    procedure Sort(const ASortParams: TsSortParams;
      ARowFrom, AColFrom, ARowTo, AColTo: Cardinal); overload;
    procedure Sort(ASortParams: TsSortParams; ARange: String); overload;
    // ...
  end;
```

The sorting criteria are defined by a record of type `TsSortParams`:

```
type
  TsSortParams = record
    SortByCols: Boolean;
    Priority: TsSortPriority;  // spNumAlpha ("Numbers first"), or spAlphaNum ("Text first")
    Keys: TsSortKeys;
  end;

  TsSortKey = record
    ColRowIndex: Integer;
    Options: TsSortOptions;    // set of [spDescending, spCaseInsensitive]
  end;
```

- The boolean value `SortByCols` determines wether sorting occurs along columns (`true`) or rows (`false`). The `ColRowIndex` specified in the sorting keys, accordingly, corresponds to a column or row index, respectively (see below).
- `Priority` determines in mixed content cell ranges whether an ascending sort puts numerical values in front of text values or not. Empty cells are always moved to the end of the sorted column or row. In Excel, the priority is "numbers first" (`spNumAlpha`).
- The array `Keys` specifies multiple sorting parameters. They consist of the index of the column or row to be sorted (`ColRowIndex`) and a set of `Options` for sorting direction (`spoDescending`) and character case (`spCaseInsensitive`). If `Options` is empty, cell comparison is case-sensitive, and cells are arranged in ascending order. If two cells are found to be "equal" on the basis of the first key (`sortParams.Keys[0]`) comparison proceeds with the next conditions in the `Keys` array until a difference is found or all conditions are used up.

`InitSortParams` is a handy utility to initialize the sorting parameters:

```
function InitSortParams(ASortByCols: Boolean = true; ANumSortKeys: Integer = 1;
  ASortPriority: TsSortPriority = spNumAlpha): TsSortParams;
```

The next code fragment shows a typical sorting call:

```
var
  sortParams: TsSortParams;
begin
  sortParams := InitSortParams(true, 2);  // sorting along columns, 2 sorting keys

  // primary sorting key: column 3, ascending, case-insensitive
  sortParams.Keys[0].ColRowIndex := 3;
  sortParams.Keys[0].Options := [ssoCaseInsensitive];

  // secondary sorting key: colum 1, descending
  sortParams.Keys[1].ColRowIndex := 1;
  sortParams.Keys[1].Options := [ssoDescending];

  // The sorted block extends between cells A1 (row=0, col=0) and F10 (row=9, col=5)
  MyWorksheet.Sort(sortParams, 0, 0, 9, 5);
  // or: MyWorksheet.Sort(sortParams, 'A1:F10');
end;
```

Searching and replacing

Unit *fpsSearch* implements a **search engine** which can be used to look for specific cell content within a workbook, or to replace the found cell content by some other string.

Example:

```
uses
  fpsTypes, fpSpreadsheet, fpsUtils, fpsSearch, fpsAllFormats;
var
  MyWorkbook: TsWorkbook;
  foundWorksheet: TsWorksheet;
  foundRow, foundCol: Cardinal;
  MySearchParams: TsSearchParams;
begin
  MyWorkbook := TsWorkbook.Create;
  try
    MyWorkbook.ReadFromFile(AFileName);

    // Specify search criteria
    MySearchParams.SearchText := 'Hallo';
    MySearchParams.Options := [soEntireDocument];
    MySearchParams.Within := swWorkbook;

    // or: MySearchParaams := InitSearchParams('Hallo', [soEntireDocument], swWorkbook);

    // Create search engine and execute search
    with TsSearchEngine.Create(MyWorkbook) do begin
      if FindFirst(MySearchParams, foundWorksheet, foundRow, foundCol) then begin
        WriteLn('First "', MySearchparams.SearchText, '" found in cell ', GetCellString(foundRow, foundCol), ' of work
        while FindNext(MySeachParams, foundWorksheet, foundRow, foundCol) do
          WriteLn('Next "', MySearchParams.SearchText, '" found in cell ', GetCellString(foundRow, foundCol), ' of wor
      end;
      Free;
    end;
  finally
    MyWorkbook.Free;
  end;
end;
```

The search engine provides two methods for **searching**: `FindFirst` and `FindNext`. They are very similar, they only differ in where the search begins. In case of `FindFirst`, the starting cell is determined from the `Options` described below. In case of `FindNext` the search begins at the cell adjacent to the previously found cell. Both methods return the worksheet and row and column indexes of the cell in which the search text is found. If the search is not successful then the function result is `FALSE`, and the `foundWorksheet` is nil. It is clear that the `foundWorksheet` cannot be used for anything else while the search is running.

The record `TsSearchParams` specifies the criteria used for searching:

```
type
  TsSearchParams = record
    SearchText: String;
    Options: TsSearchOptions;
    Within: TsSearchWithin;
  end;
```

Besides the text to be searched (`SearchText`) it provides a set of options to narrow the search:

- `soCompareEntireCell`: Compares the `SearchText` with the entire cell content. If not contained in the `Options` then the cell text is compared only partially.
- `soMatchCase`: Perform a case-sensitive search
- `soRegularExpr`: The `SearchText` is considered as a regular expression
- `soAlongRows`: The search engine proceeds first along the rows. If not contained in the `Options` then the search proceeds along the columns.
- `soBackward`: The search begins at the end of the document, or runs backward from the active cell. If not contained in the `Options` then the search starts at the beginning of the document, or runs forward from the active cell.
- `soWrapDocument`: If a search has reached the end of the document the search is resumed at its beginning (or vice versa, if `soBackward` is used).
- `soEntireDocument`: Search begins at the first cell (or last cell if `soBackward` is used). If not contained in the `Options` then the search begins at the active cell of the worksheet. Ignored by `FindNext`.

The record field `Within` identifies the part of the spreadsheet to be searched:

- `swWorkbook`: The entire workbook is searched. If the search phrase is not found on the first worksheet (or last worksheet if `soBackward` is used) then the search continues with the next (previous) sheet.
- `swWorksheet`: The search is limited to the currently active worksheet
- `swColumn`: Search is restricted to the column of the active cell
- `swRow`: Search is restricted to the row of the active cell.

The search params record can be initialized by calling `InitSearchParams` (in unit *fpsutils*) with the record elements as optional parameters. Use the methods `Workbook.ActiveWorksheet` and `Worksheet.SelectCell(ARow, ACol)` to define the active worksheet and the position of the active cell, respectively, if needed by the search.

In addition to searching the search engine can also be used for **replacing** the found text by another string. Call the functions `ReplaceFirst` or `ReplaceNext` for this purpose. They act like their `FindXXXX` counterparts, therefore, they require a `TsSearchParams` record to specify the search criteria. But in addition to searching, these functions also perform the text replacement according to the specification in a `TsReplaceParams` record:

```
type
  TsReplaceParams = record
    ReplaceText: String;
    Options: TsReplaceOptions;
  end;
```

The `ReplaceText` identifies the string which will replace the found text pattern. The `Options` define a set of criteria how the replacement is done:

- `roReplaceEntirecell`: Replaces the entire cell text by the `ReplaceText`. If not contained in the `Options` then only the part matching the `SearchText` is replaced.
- `roReplaceAll`: Performs the replacement in all found cells (i.e., simply call `ReplaceFirst` to replace all automatically).
- `roConfirm`: Calls an event handler for the `OnConfirmReplacement` event in which the user must specify whether the replacement is to be performed or not. Note that this event handler is mandatory if `roConfirm` is set.

Use the function `InitReplaceParams` (in unit *fpsutils*) to initialize the replace parameters record with the provided (but optional) values.

Column and row operations

The worksheet provides these methods for inserting, deleting, hiding or unhiding columns and rows:

```
type
  TsWorksheet = class
  ...
    procedure DeleteCol(ACol: Cardinal);
    procedure DeleteRow(ARow: Cardinal);

    procedure InsertCol(ACol: Cardinal);
    procedure InsertRow(ARow: Cardinal);

    procedure RemoveCol(ACol: Cardinal);
    procedure RemoveRow(ARow: Cardinal);

    procedure RemoveAllCols;
    procedure RemoveAllRows

    procedure HideCol(ACol: Cardinal);
    procedure HideRow(ARow: Cardinal);

    procedure ShowCol(ACol: Cardinal);
    procedure ShowRow(ARow: Cardinal);

    function ColHidden(ACol: Cardinal): Boolean;
    function RowHidden(ARow: Cardinal): Boolean;
```

- When a column or row is **deleted** by `DeleteCol` or `DeleteRow`, any data assigned to this column or row are removed, i., cells, comments, hyperlinks, TCol or TRow records. Data at the right of or below the deleted column/row move to the left or up.
- `RemoveCol` and `RemoveRow`, in contract, **remove** only the column or row record, i.e. reset column width and row height to their default values. Cell, comment, and hyperlink data are not affected.
- `RemoveAllCols` **removes all** column records, i.e. resets all column widths; `RemoveAllRows` does the same with the row records and row heights.
- A column or row is **inserted** before the index specified as parameter of the `InsertXXX` method.

Page layout

General

So far, FPSpreadsheet does not support printing of worksheets, but the Office applications do, and they provide a section of information in their files for this purpose. In FPSpreadsheets this information is available in the `TsPageLayout` class which belongs to the `TsWorksheet` data structure. Its properties and methods combine the most important features from the Excel and OpenDocument worlds.

```
type
  TsPageOrientation = (spoPortrait, spoLandscape);

  TsPrintOption = (poPrintGridLines, poPrintHeaders, poPrintPagesByRows,
    poMonochrome, poDraftQuality, poPrintCellComments, poDefaultOrientation,
    poUseStartPageNumber, poCommentsAtEnd, poHorCentered, poVertCentered,
    poDifferentOddEven, poDifferentFirst, poFitPages);

  TsPrintOptions = set of TsPrintOption;

  TsHeaderFooterSectionIndex = (hfsLeft, hfsCenter, hfsRight);

  TsCellRange = record
    Row1, Col1, Row2, Col2: Cardinal;
  end;

  TsPageLayout = class
  ...
  public
    ...
    { Methods }
    // embedded header/footer images
    procedure AddHeaderImage(AHeaderIndex: Integer;
      ASection: TsHeaderFooterSectionIndex; const AFilename: String);
    procedure AddFooterImage(AFooterIndex: Integer;
      ASection: TsHeaderFooterSectionIndex; const AFilename: String);
    procedure GetImageSections(out AHeaderTags, AFooterTags: String);
    function HasHeaderFooterImages: Boolean;

    // Repeated rows and columns
    function HasRepeatedCols: Boolean;
    function HasRepeatedRows: Boolean;
    procedure SetRepeatedCols(AFirstCol, ALastCol: Cardinal);
    procedure SetRepeatedRows(AFirstRow, ALastRow: Cardinal);

    // print ranges
    function AddPrintRange(ARow1, ACol1, ARow2, ACol2: Cardinal): Integer; overload;
    function AddPrintRange(const ARange: TsCellRange): Integer; overload;
    function GetPrintRange(AIndex: Integer): TsCellRange;
    function NumPrintRanges: Integer;
    procedure RemovePrintRange(AIndex: Integer);

    { Properties }
    property Orientation: TsPageOrientation read FOrientation write FOrientation;
    property PageWidth: Double read FPageWidth write FPageWidth;
    property PageHeight: Double read FPageHeight write FPageHeight;
    property LeftMargin: Double read FLeftMargin write FLeftMargin;
    property RightMargin: Double read FRightMargin write FRightMargin;
    property TopMargin: Double read FTopMargin write FTopMargin;
    property BottomMargin: Double read FBottomMargin write FBottomMargin;
    property HeaderMargin: Double read FHeaderMargin write FHeaderMargin;
    property FooterMargin: Double read FFooterMargin write FFooterMargin;
    property StartPageNumber: Integer read FStartPageNumber write SetStartPageNumber;
    property ScalingFactor: Integer read FScalingFactor write SetScalingFactor;
    property FitHeightToPages: Integer read FFitHeightToPages write SetFitHeightToPages;
    property FitWidthToPages: Integer read FFitWidthToPages write SetFitWidthToPages;
    property Copies: Integer read FCopies write FCopies;
    property Options: TsPrintOptions read FOptions write FOptions;
    property Headers[AIndex: Integer]: String read GetHeaders write SetHeaders;
    property Footers[AIndex: Integer]: String read GetFooters write SetFooters;
    property RepeatedCols: TsRowColRange read FRepeatedCols;
    property RepeatedRows: TsRowColRange read FRepeatedRows;
    property PrintRange[AIndex: Integer]: TsCellRange read GetPrintRange;
    property FooterImages[ASection: TsHeaderFooterSectionIndex]: TsHeaderFooterImage read GetFooterImages;
    property HeaderImages[ASection: TsHeaderFooterSectionIndex]: TsHeaderFooterImage read GetHeaderImages;
  end;

  TsWorksheet = class
  ...
  public
    property PageLayout: TsPageLayout;
    ...
  end;
```

Note: The fact that the `PageLayout` belongs to the worksheet indicates that there can be several page layouts in the same workbook, one for each worksheet.

- **Orientation** defines the orientation of the printed paper, either portrait or landscape.
- **Page width** and **page height** refer to the standard orientation of the paper, usually portrait orientation.
- **Left**, **top**, **right** and **bottom margins** are self-explanatory and are given in millimeters.
- **HeaderMargin** is understood - like in Excel - as the distance between the paper top edge and the top of the header, and **TopMargin** correspondingly is the distance between the top paper edge and the top of the first table row, i.e. if the header contains several line breaks it can reach into the the table part of the print-out. This is different from OpenDocument files where the header can grow accordingly.
- **StartPageNumber** should be altered if the print-out should not begin with page 1. This setting requires to add the option `poUseStartPageNumber` to the PageLayout's `Options` - but this is normally done automatically.

- The **ScalingFactor** is given in percent and can be used to reduce the number of printed pages. Modifying this property clear the option `poFitToPages` from the PageLayout's `Options`.
- Alternatively to ScalingFactor, you can also use **FitHeightToPages** or **FitWidthToPages**. The option `poFitToPages` must be active in order to override the `ScalingFactor`setting. `FitHeightToPages` specifies the number of pages onto which the entire height of the printed worksheet should fit. Accordingly, `FitWidthToPages` can be used to define the number of pages on which the entire width of the worksheet has to fit. The value `0` has the special meaning of "use as many pages as needed". In this way, the setting "Fit all columns on one page" of Excel, for example, can be achieved by this code:

```
MyWorksheet.PageLayout.Options := MyWorksheet.PageLayout.Options + [poFitPages];
MyWorksheet.PageLayout.FitWidthToPages := 1;      // all columns on one page width
MyWorksheet.PageLayout.FitHeightToPages := 0;     // use as many pages as needed
```

- **Header rows and columns repeated on every printed page** can be defined by the `RepeatedCols` and `RepeatedRows` records; their elements `FirstIndex` and `LastIndex` refer to the indexes of the first and last column or row, respectively, to be repeated. Use the methods `SetRepeatedCols` and `SetRepeatedRows` to define these numbers. Note that the second parameter for the last index can be omitted to use only a single header row or column.
- **Print ranges** or **print areas** (using Excel terminology) can be used to restrict printing only to a range of cells. Use the methods `AddPrintRange` to define a cell range for printing: specify the indexes of the left column, top row, right column and bottom row of the range to be printed. A worksheet can contain several print ranges.
- **Copies** specifies how often the worksheet will be printed.
- The **Options** define further printing properties, their names are self-explaining. They were defined according to Excel files, some of them do not exist in ODS files and are ignored there.

Headers and footers

Header and footer texts can be composed of left-aligned, centered and right-aligned strings. Add the symbol `&L` to indicate that the following string is to be printed as the left-aligned part; use `&C` accordingly for the centered and `&R` for the right-aligned parts. There are other symbols which will be replaced by their counterparts during printing:

- `&L`: begins the **left-aligned section** of a header or a footer text definition
- `&C`: begins the **centered section** of a header or a footer text definition
- `&R`: begins the **right-aligned section** of a header or a footer text definition
- `&P`: **page number**
- `&N`: **page count**
- `&D`: current **date** of printing
- `&T`: current **time** of printing
- `&A`: **worksheet name**
- `&F`: **file name** without path
- `&Z`: **file path** without file name
- `&G`: embedded **image** - use the methods `AddHeaderImage` or `AddFooterImage` to specify the image file; this also appends the `&G` to the other codes of the current header/footer section. Note that not all image types known by the Office application may be accepted. Currently the image can be *jpeg*, *png*, *gif*, *bmp*, *tiff*, *pcx*, *svg*, *wmf* or *emf*.
- `&B`: **bold** on/off
- `&I`: **italic** on/off
- `&U`: **underlining** on/off
- `&E`: **double-underlining** on/off
- `&S`: **strike-out** on/off
- `&H`: **shadow** on/off
- `&O`: **outline** on/off
- `&X`: **superscript** on/off
- `&Y`: **subscript** on/off
- `&"font"`: begin using of the **font** with the specified name, e.g. `&"Arial"`
- `&number`: begin using of the specified **font size** (in points), e.g. `&16`
- `&Krrggbb`: switch to the **font color** specified to the binary value of the specified color, e.g. use `&KFF0000` for red.

The arrays `Headers[]`/`Footers[]` provide space for usage of three different headers or footers:

- `Headers[0]` refers to the header used on the **first page** only, similarly for `Footers[0]`. Instead of index `0` you can use the constant `HEADER_FOOTER_INDEX_FIRST`. Leave this string empty if there is no special first-page header/footer.
- `Headers[1]` refers to the header on pages with **odd page numbers**, similarly for `Footers[1]`. Instead of index `1` you may want to use the constant `HEADER_FOOTER_INDEX_ODD`.
- `Headers[2]` refers to the header on lages with **even page nubmers**, similarly for `Footers[2]`. Instead of index `2` you may want to use the constant `HEADER_FOOTER_INDEX_EVEN`. Leave the strings at index 0 and 2 empty if the print-out should always have the same header/footer. You can use the constant `HEADER_FOOTER_INDEX_ALL` for better clarity. Example:

```
MyWorksheet.PageLayout.Headers[HEADER_FOOTER_INDEX_ALL] := '&C&D &T';           // centered "date time" on all p
MyWorksheet.PageLayout.Footers[HEADER_FOOTER_INDEX_ODD] := '&RPage &P of &N';   // right-aligned "Page .. of ...
MyWorksheet.PageLayout.Footers[HEADER_FOOTER_INDEX_EVEN] := '&LPage &P of &N';  // dto, but left-aligned on even
```

Protection

In the Office applications, workbooks can be protected from unintentional changes by the user. fpspreadsheet is able to read and write the data structures related to protection, but does not enforce them. This means, for example, that cells can be modified by the user although the worksheet is specified as being locked.

Protection is handled at three levels: workbook protection, worksheet protection and cell protection.

Workbook protection

`TsWorkbook` contains a set of workbook or document protection options:

- `bpLockRevision`: specifies that the workbook is locked for revisions
- `bpLockStructure`: if this option is set then worksheets in the workbook cannot be moved, deleted, hidden, unhidden, or renamed, and new worksheets cannot be inserted.
- `bpLockWindows`: indicates that the workbook windows in the Office application are locked. Windows are the same size and position each time the workbook is opened by the Office application.

In relation to workbook protection is the worksheet option `soPanesProtection` which prevents the panes of a worksheet from being modified if the workbook is

protected.

Depending on the file format, only some of these options might be supported. In these cases, the non-supported options are commonly accepted default values.

Office applications are able to encrypt their spreadsheet files. The readers of FPSpreadsheet basically are not able to open these files. However, there exists a related package, **laz_fpspreadsheet_crypto.lpk**, which has access to decryption routines and is able to read xlsx and ods files. These readers are in the units *xlsxOOXML_crypto* and *fpsOpendocument_crypto*, respectively. Adding them to the uses clause of the application installs user-provided readers with `FormatID` `sfidOOXML_crypto` and `sfidOpenDocument_crypto`, respectively. If the password is known it can be specified as a parameter to the workbook's `LoadFromFile`/`ReadFromStream` methods. If no password is specified the workbook fires the event `OnQueryPassword` in which the user can provide the password, e.g. like this:

```
function TForm1.QueryPasswordHandler: String;
begin
  Result := InputBox('Password required', 'Password', '');
end;

procedure TForm1.LoadWorkbook(AFileName: String);
var
  workbook: TsWorkbook;
begin
  workbook := TsWorkbook.Create;
  try
    workbook.OnQueryPassword := @QueryPasswordHandler;
    workbook.ReadFromFile(AFileName);
    ...
  finally
    workbook.Free;
  end;
end;

// or

procedure TForm1.LoadWorkbookWithPassword(AFileName, APassword: STring);
var
  workbook: TsWorkbook;
begin
  workbook := TsWorkbook.Create;
  try
    workbook.ReadFromFile(AFileName, APassword);
    ...
```

Worksheet protection

`TsWorksheet` houses a similar set of protection options. Whenever an option is included in the set `Protection` of the workbook the corresponding action is not allowed and locked:

- `spCells`: the cells in the sheet are protected. It depends on the level of cell protection whether a particular cell can be changed or not. By default, no cell can be changed.
- `spDeleteColumns`: deleting of columns is not be allowed
- `spDeleteRows`: it is not possible to delete rows
- `spFormatCells`: formatting of cells is not allowed
- `spFormatColumns`: columns cannot be formatted.
- `spFormatRows`: rows cannot be formatted
- `spInsertColumns`: it is not allowed to insert columns
- `spInsertRows`: rows cannot be inserted
- `spInsertHyperlinks`: it is not possible to insert new hyperlinks
- `spSort`: the worksheet is not allowed to be sorted.
- `spSelectLockedCells`: Cells which are locked cannot be selected any more.
- `spSelectUnlockedCells`: Even cells which are unlocked cannot be selected. Together with `spSelectLockedCells` this means that the selection in the worksheet is frozen. These levels of protection become active if the option `soProtected` is added to the worksheet's `Options`, or by calling the worksheet method `Protect(true)`.

Cell protection

Cell protection becomes active when the worksheet protection is enabled. It is controlled by a set of `TsCellProtection` elements which belong to the cell format record:

- `cpLockCell`: This option determines whether cell content can be modified by the user. Since it is on by default cells of a protected worksheet normally cannot be edited. In order to unlock some cells for user input the option `cpLockCell` must be removed from the protection of these cells.
- `cpHideFormulas`: prevents formulas from being shown in the Office application.

Cell protection can be changed by calling the worksheet method `WriteCellProtection`. Conversely, `ReadCellProtection` can be used to retrieve the protection state of a particular cell:

```
// query and modifiy the protection state of cell A1 (row=0, col=0)
var
  cell: PCell;
  cellprot: TsCellProtections;
...
  // Find the cell
  cell := worksheet.FindCell(0, 0);
  // query cell protection
  cellprot := worksheet.ReadCellProtection(cell);
  // Unlock the cell for editing, don't change the visibility of formulas
  worksheet.WriteCellProtection(cell, cellprot - [cpLockCell]);
  // Hide formula of the cell and unlock the cell.
  worksheet.WriteCellProtection(cell, [cpHideFormulas]);
  // Activate protection
  worksheet.Protect(true);
```

Passwords

Workbook and worksheet protection can be secured by passwords. Note that these passwords do not encrypt the file (except for workbook protection in Excel 2007). In the Office applications the user must enter this password to turn off protection or to change protection items. The encrypted password is stored in the `CryptoInfo` record of

the wordbook and the worksheets, respectively:

```
type
  TsCryptoInfo = record
    PasswordHash: String;
    Algorithm: TsCryptoAlgorithm;  // caExcel, caSHA1, caSHA256, etc.
    SaltValue: String;
    SpinCount: Integer;
  end;
```

Warning: FPSpreadsheet does not perform any hashing calculations, the `CryptoInfo` record is just passed through from reading to writing. This causes problems when different file formats are involved in reading and writing. It is attempted to detect '''incompatible combinations'''. In these cases, the '''password protection is removed''', and an error is logged by the workbook.

Loading and saving

Adding new file formats

FPSpreadsheet is open to any spreadsheet file format. In addition to the built-in file formats which are specified by one of the `sfXXXX` declarations, it is possible to provide dedicated reader and writer units to get access to special file formats.

- Write a unit implementing a reader and a writer for the new file format. They must inherit from the basic `TsCustomSpreadReader` and `TsCustomWriter`, respectively, - both are implemented in unit *fpsReaderWriter* -, or from one of the more advanced ones belonging to the built-in file formats.
- Register the new reader/writer by calling the function `RegisterSpreadFileFormat` in the `initialization` section of this unit (implemented in unit *fpsRegFileFormats*):

```
function RegisterSpreadFormat(AFormat: TsSpreadsheetFormat; AReaderClass: TsSpreadReaderClass; AWriterClass: TsSpr
  AFormatName, ATechnicalName: String; const AFileExtensions: array of String): TsSpreadFormatID;
```

- `AFormat` must have the value `sfUser` to register an external file format.
- `AReaderClass` is the class of the reader (or nil, if reading functionality is not implemented).
- `AWriterClass` is the class of the writer (or nil, if writing functionality is not implemented).
- `AFormatName` defines the name of the format as used for example in the filter list of file-open dialogs.
- `ATechnicalName` defines a shorter format name.
- `AFileExtensions` is an array of file extensions used in the files. The first array element denotes the default extension. The extensions must begin with a period as in `.xls`.
- The registration function returns a numerical value (`TsSpreadFormatID`) which can be used as format identifier in the workbook reading and writing functions which exist in overloaded version accepting a numerical value for the format specifier. In contract to the built-in formats the FormatID is negative.
- Finally, in your application, add the new unit to the `uses` clause. This will call the registrations function when the unit is loaded and make the new file format available to FPSpreadsheet.

Stream selection

Workbooks are loaded and saved by means of the `ReadFromFile` and `WriteToFile` methods, respectively (or by their stream counterparts, `ReadFromStream` and `WriteToStream`).

By default, the data files are accessed by means of **memory streams** which yields the fastest access to the files. In case of very large files (e.g. tens of thousands of rows), however, the system may run out of memory. There are two methods to defer the memory overflow by some extent.

- Add the element `boBufStream` to the workbook's `Options`. In this case, a **"buffered" stream** is used for accessing the data. This kind of stream holds a memory buffer of a given size and swaps data to file if the buffer becomes too small.
- Add the element `boFileStream` to the workbook's `Options`. This option avoids memory streams altogether and creates temporary **files** if needed. This is, however, the slowest method of data access.
- If both options are set then `boBufStream` is ignored.
- In practice, however, the effect of the selected streams is not very large if memory is to be saved.

Virtual mode

Beyond the transient memory usage during reading/writing the main memory consumptions originates in the internal structure of FPSpreadsheet which holds all data in memory. To overcome this limitation, a "virtual mode" has been introduced. In this mode, data are received from a data source (such as a database table) and are passed through to the writer without being collected in the worksheet. It is clear that data loaded in virtual mode cannot be displayed in the visual controls. Virtual mode is good for conversion between different data formats.

These are the steps required to use this mode:

- Activate virtual mode by adding the option `boVirtualMode` to the `Options` of the workbook.
- Tell the spreadsheet writer how many rows and columns are to be written. The corresponding worksheet properties are `VirtualRowCount` and `VirtualColCount`.
- Write an event handler for the event `OnWriteCellData` of the worksheet. This handler gets the index of row and column of the cell currently being saved. You have to return the value which will be saved in this cell. You can also specify a template cell that physically exists in the workbook from which the formatting style is copied to the destination cell. Please be aware that when exporting a database, you are responsible for advancing the dataset pointer to the next database record when writing of a row is complete.
- Call the `WriteToFile` method of the workbook.

Virtual mode also works for reading spreadsheet files.

The folder *example/other* contains a worked out sample project demonstrating virtual mode using random data. More realistic database examples are in *example/db_import_export* and in the chapter on converting a large database table using virtual mode.

Dataset export

FPC contains a set of units that allow you to export datasets to various formats (XML, SQL statements, DBF files,...). There is a master package that allows you to choose an export format at design time or run time (Lazarus package lazdbexport).

FPSpreadsheet has `TFPSExport` which plugs into this system. It allows you to export the contents of a dataset to a new spreadsheet (*.xls*, *.xlsx*, *.ods*, wikitable format) file into a table on the first sheet by default. In addition, if `MultipleSheets` is set to `TRUE` it is possible to combine several sheets into individual worksheets in the same file. You can optionally include the field names as header cells on the first row using the `HeaderRow` properties in the export settings. The export component tries to find the number format of the cells according to the dataset field types.

For more complicated exports, you need to manually code a solution (see examples below) but for simple data transfer/dynamic exports at user request, this unit will probably be sufficient.

A simple example of how this works:

```
uses
...
fpsexport
...
var
  Exp: TFPSExport;
  ExpSettings: TFPSExportFormatSettings;
  TheDate: TDateTime;
begin
  FDataset.First; //assume we have a dataset called FDataset
  Exp := TFPSExport.Create(nil);
  ExpSettings := TFPSExportFormatSettings.Create(true);
  try
    ExpSettings.ExportFormat := efXLS; // choose file format
    ExpSettings.HeaderRow := true; // include header row with field names
    Exp.FormatSettings := ExpSettings; // apply settings to export object
    Exp.Dataset:=FDataset; // specify source
    Exp.FileName := 'c:\temp\datadump.xls';
    Exp.Execute; // run the export
  finally
    Exp.Free;
    ExpSettings.Free;
  end;
```

Export to DBF

You can save the information from Excel to dbf rather easily. Here is the example from forum. Please note that this example is supposed to work with Win1251 encoding by default. To get more information please refer to this thread http://forum.lazarus.freepascal.org/index.php/topic,41636.0.html

```
procedure TForm1.ExportToDBF(AWorksheet: TsWorksheet; AFileName: String);
var
  i: Integer;
  f: TField;
  r, c: Cardinal;
  cell: PCell;
begin
  DbfGlobals.DefaultCreateCodePage := 1251; //default encoding of the dbf
  DbfGlobals.DefaultOpenCodePage := 1251; //default encoding of the dbf
  if Dbf1.Active then Dbf1.Close;
  if FileExists(AFileName) then DeleteFile(AFileName);

  Dbf1.FilePathFull := ExtractFilePath(AFileName);
  Dbf1.TableName := ExtractFileName(AFileName);
  Dbf1.TableLevel := 25;  // DBase IV: 4 - most widely used; or 25 = FoxPro supports nfCurrency
  Dbf1.LanguageID := $C9; //russian language by default
  Dbf1.FieldDefs.Clear;
  //below are the fields in excel
  Dbf1.FieldDefs.Add('fam', ftString);
  //add other fields you want to save

  Dbf1.CreateTable;
  Dbf1.Open;

  for f in Dbf1.Fields do
    f.OnGetText := @DbfGetTextHandler;

  // Skip row 0 which contains the headers
  for r := 1 to AWorksheet.GetLastRowIndex do begin
    Dbf1.Append;
    for c := 0 to Dbf1.FieldDefs.Count-1 do begin
      f := Dbf1.Fields[c];
      cell := AWorksheet.FindCell(r, c);
      if cell = nil then
        f.Value := NULL
      else
        case cell^.ContentType of
          cctUTF8String: f.AsString := UTF8ToCP1251(cell^.UTF8StringValue);
          cctNumber: f.AsFloat := cell^.NumberValue;
          cctDateTime: f.AsDateTime := cell^.DateTimeValue;
          else f.AsString := UTF8ToCP1251(AWorksheet.ReadAsText(cell));
        end;
    end;
    Dbf1.Post;
  end;
end;

//This procedure is called so the text in the cells could be correctly dispayed on WorkSheetGrid on the form, otherwis
procedure TForm1.DbfGetTextHandler(Sender: TField; var AText: string; DisplayText: Boolean);
begin
  if DisplayText then
    AText := CP1251ToUTF8(Sender.AsString);
end;
```

Visual controls for FPSpreadsheet

The package **laz_fpspreadsheet_visual** implements a series of controls which simiplify creation of visual GUI applications:

- **TsWorkwookSource** links the controls to a workbook and notifies the controls of changes in the workbook.
- **TsWorksheetGrid** implements a grid control with editing and formatting capabilities; it can be applied in a similar way to TStringGrid.

- **TsWorkbookTabControl** provides a tab sheet for each worksheet of the workbook. It is the ideal container for a TsWorksheetGrid.
- **TsWorksheetIndicator**: a combobox which lists all worksheets of the workbook.
- **TsCellEdit** corresponds to the editing line in Excel or Open/LibreOffice. Direct editing in the grid, however, is possible as well.
- **TsCellIndicator** displays the name of the currently selected cell; it can be used for navigation purposes by entering a cell address string.
- **TsCellCombobox** offers to pick cell properties for a selection of formatting attributes: font name, font size, font color, background color.
- **TsSpreadsheetInspector** is a tool mainly for debugging; it displays various properties of workbook, worksheet, cell value and cell formatting, similar to the ObjectInspector of Lazarus. It is read-only, though.
- Various standard actions are provided in the unit **fpsActions**. Applied to menu or toolbar, they simplify typical formatting and editing tasks without having to write a line of code.
- **TsWorkbookChartLink** and **TsWorkbookChartSource** interface a workbook with the TAChart library. TsWorkbookChartLink reads all chart-related parameters from a workbook chart and display it in a standard Lazarus TAChart. TsWorkbookChartsource defines the cell ranges from which a chart series can get its data.

See the FPSpreadsheet tutorial: Writing a mini spreadsheet application for more information and a tutorial, and see demo projects for examples of the application of these components. Creating charts in a spreadsheet and displaying them by the TAChart library is demonstrated in the chart tutorial.

Examples

Please see FPSpreadsheet: Examples for a variety of code examples.

Download

Subversion

You can download FPSpreadsheet using the subversion software and the following command line:

```
svn checkout https://svn.code.sf.net/p/lazarus-ccr/svn/components/fpspreadsheet fpspreadsheet
```

Stable releases

The most current release is distributed by the Online-Package-Manger which is integrated in the Lazarus IDE (menu "Package" > "Online package manager"): Scroll down the list of available packages, check the entry "FPSpreadsheet" and click "Install". Confirm the prompt to rebuild the IDE. When the process is complete you find the new components in palette "FPSpreadsheet".

Older releases of FPSpreadsheet cann still be found on sourceforge.

Installation

- If you only need non-GUI components: in Lazarus: Package/Open Package File, select **laz_fpspreadsheet.lpk**, click Compile. Now the package is known to Lazarus (and should e.g. show up in Package/Package Links). Now you can add a dependency on `laz_fpspreadsheet` in your project options and `fpspreadsheet` to the uses clause of the project units that need to use it.
- If you also want GUI components (TsWorksheetGrid and TsWorksheetChartSource): Package/Open Package File, select **laz_fpspreadsheet_visual.lpk**, click Compile, then click Use, Install and follow the prompts to rebuild Lazarus with the new package. Drop needed grid/chart components on your forms as usual.
- If you want to have a GUI component for dataset export: Package/Open Package File, select **laz_fpspreadsheetexport_visual.lpk**, click Compile, then click, Use, Install and follow the prompts to rebuild Lazarus with the new package. Drop needed export components from the **Data Export** tab on your forms as usual.
- FPSpreadsheet is developed with the latest stable fpc version (currently fpc 3.0.2). We only occasionally check older versions.
- The basic spreadsheet functionality works with Lazarus versions back to version 1.0. Some visual controls or demo programs, however, require newer versions. Please update your Lazarus if you have an older version and experience problems.

Conditional defines

Here is a list of **conditional defines** which can be activated in order to tweak some operating modes of the packages and/or make it compilable with older Lazarus/FPC versions:

- `FPS_DONT_USE_CLOCALE`: In Unix systems, the unit `clocale` is automatically added to the uses clause of *fpspreadsheet.pas*. This unit sets up localization settings needed for locale-dependent number and date/time formats. However, this adds a dependence on the C library to the package http://lazarus-ccr.sourceforge.net/docs/rtl/clocale/index.html. If this is not wanted, define `FPS_DONT_USE_CLOCALE`.
- `FPS_VARISBOOL`: fpspreadsheet requires the function VarIsBool which was introduced by fpc 2.6.4. If an older FPC versions is used define `FPS_VARISBOOL`. Keep undefined for the current FPC version.
- `FPS_LAZUTF8`: fpspreadsheet requires some functions from the unit LazUTF8 which were introduced by Lazarus 1.2. If an older Lazarus version is used define `FPS_LAZUTF8`. Keep undefined for the current Lazarus version.
- `FPS_NO_GRID_MULTISELECT`: In order to allow selection of multiple ranges in the WorksheetGrid a sufficiently new version of the basic TCustomGrid is needed. The required property "RangeSelect" was introduced in Lazarus 1.4. In order to compile the package with older versions activate the define `FPS_NO_GRID_MULTISELECT`.
- `FPS_PTRINT`: In order to provide safe casting of integers to pointers new version of FPC provide the types PtrInt and IntPtr. This is not yet available in fpc 2.6.0.
- `FPS_NEED_STRINGHASHLIST`: Unit stringhashlist belongs to LCL before Lazarus 1.8. To avoid a requirement of LCL in laz_fpspreadsheet.lpk a copy in the fps directory is provided. This copy is used when the define FPS_NEED_STRINGHASHLIST is active. The define is not needed for Lazarus versions >= 1.8.
- `FPS_NO_NEW_UTF8_ROUTINES`: In Lazarus 2.0+ some UTF8 routines in unit LazUTF8 were renamed from UTF8CharacterXXX to UTF8CodePointXXX. Activate the following define when the new routines are not available, i.e. for Lazarus version < 2.0.
- `FPS_NO_LAZUTF16`: Lazarus 1.8+ has unit LazUTF16 for special access to widestring. This define must be active when this unit is not available, i.e. for Lazarus versions before 1.8.0.
- `FPS_NO_STRING_SPLIT`: Activate the following define if FPS does not have the string Split helper, e.g. before v3.0
- `FPS_CHARTS`: Activate chart support by fpspreadsheet and their reading/writing in xlsx and ods. This feature is behind this define since many spreadsheets do not use charts.

All these defines are collected in the include file *fps.inc*.

In the most recent test (Feb 2023), all fpspreadsheet packages compile fine with Laz 1.6+ / fpc 3.0+ (some defines in *fps.inc* may have to be adapted, though). With Laz 1.4 / fpc 2.6.4 or older, *fpspreadsheet_dataset.lpk* and *fpspreadsheet_crypto.lpk* cannot be used any more.

Support and Bug Reporting

The recommended place to discuss FPSpreadsheet and obtain support is asking in the Lazarus Forum.

Bug reports should be sent to the Lazarus/Free Pascal Bug Tracker; please specify the "Lazarus-CCR" project.

Current Progress

Progress by supported cell content

Format	Multiple sheets	Unicode	Reader support	Writer support	Text	Number	String Formula	RPN Formula	3D cell references	Date/Time	Comments	Hyperlinks	Images +++	Protection	Conditional formats
CSV files	No	Yes +	Working ++	Working ++	Working ++	Working ++	N/A	N/A	N/A	Working ++	N/A	N/A	N/A	N/A	N/A
Excel 2.x	No	No *	Working **	Working	Working	Working	Working	Working ***	N/A	Working ****	Working	N/A	N/A	Working	N/A
Excel 5.0 (Excel 5.0 and 95)	Yes	No *	Working **	Working	Working	Working	Working	Working ***	Working	Working ****	Working	N/A	N/A	Working	N/A
Excel 8.0 (Excel 97-2003)	Yes	Yes	Working **	Working	Working	Working	Working	Working ***	Working	Working ****	Reading only	Working	Not working	Working	Not working
Excel 2003/XML	Yes	Yes	Working **	Working	Working	Working	Working ***	Working	Working	Working ****	Working	Working	N/A	Working	Working
Excel OOXML	Yes	Yes	Working **	Working	Working	Working	Working ***	Working	Working	Working ****	Working	Working	Working	Working	Working ++++
OpenDocument	Yes	Yes	Working **	Working	Working	Working	Working ***	Working	Working	Working ****	Working	Working	Working	Working	Working
HTML	No	Yes	Working ++	Working ++	Working ++	Working ++	N/A	N/A	N/A	Working ++	N/A	Working	Not working	N/A	N/A
Wikitable files (Mediawiki)	No	Yes	planned	Working ++	Working ++	Working ++	N/A	N/A	N/A	Working ++	N/A	N/A	Not working	N/A	N/A

(+) Depends on file.
(++) No "true" number format support because the file does not containg number formatting information. But the number format currently used in the spreadsheet understood.
(+++) Only very basic image support: no transformations, no cropping, no image manipulation.
(++++) Extended OOXML formatting options not supported.
(*) In formats which don't support Unicode the data is stored by default as ISO 8859-1 (Latin 1). You can change the encoding in TsWorkbook.Encoding. Note that FPSpreadsheet offers UTF-8 read and write routines, but the data might be converted to ISO when reading or writing to the disk. Be careful that characters which don't fit selected encoding will be lost in those operations. The remarks here are only valid for formats which don't support Unicode.
(**) Some cell could be returned blank due to missing or non ready implemented number and text formats.
(***) This is the format in which the formulas are written to file (determined by design of the file format).
(****) Writing of all formats is supported. Some rare custom formats, however, may not be recognized correctly. BIFF2 supports only built-in formats by design.

Progress of the formatting options

The following formatting options are available:

Format	Text alignment	Text rotation	Font	Rich text	Border	Color support	Back ground	Word wrap	Col&Row size	Number format	Merged cells	Page layout	Print ranges	Header/footer images	Column/row format	Hide cols/rows	Page breaks
CSV files	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Excel 2.x	Working *	N/A	Working	N/A	Working	Working	Working**	N/A	Working	Working	N/A	Working	N/A	N/A	Working	N/A	Working
Excel 5.0 (Excel 5.0 and 95)	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	N/A	Working	Working	N/A	Working	Working	Working
Excel 8.0 (Excel 97 - XP)	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Not working	Working	Working	Working
Excel 2003/XML	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	N/A	Working	Wworking	Working
Excel OOXML (xlsx)	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working	Working
OpenDocument	Working	Working	Working	Working	Working	Working	Working***	Working	Working	Working	Working	Working	Working	Working	Working	Working	
HTML (+)	Working	bugs	Working	Working	bugs	Working	Working ****	Writing only	Writing only	N/A	Working	N/A	N/A	Not working	Writing only	to be done	N/A
Wikitable (Mediawiki)	Writing only	N/A	Writing only	Not working	Writing only	Writing only		Writing only	Writing only	N/A	Writing only	N/A	N/A	Not working	Writing only	to be done	N/A

(N/A) Feature is not available for this format intrinsically.
(*) BIFF2 supports only horizontal text alignment, vertical alignment is ignored.
(**) BIFF2 does not support a background color; a dotted black&white background is used instead.
(***) OpenDocument supports only uniform backgrounds; a fill color interpolated between foreground and background colors is written instead.
(****) Only uniform background color, no fill styles.
(+) HTML reader does not support styles. Since the writer does use styles these files are not read back correctly.

Progress of workbook/worksheet user-interface options

Some additional options were added to interface the file contents with the `TsWorksheetGrid`:

Format	Hide grid lines	Hide headers	Frozen Panes	Active sheet/cell	Zooming	BiDi mode	Tab color
Excel 2.x	Working	Working	not working	N/A	N/A	N/A	N/A
Excel 5.0 (Excel 5.0 and 95)	Working	Working	Working	Working	Working	N/A	N/A
Excel 8.0 (Excel 97 - XP)	Working	Working	Working	Working	Working	Working	Working
Excel2003/XML	Working	Working	Working	Working	Working	Working	N/A
Excel OOXML	Working	Working	Working	Working	Working	Working	Working
OpenDocument	Working	Working	Working	Working	Working	Working	Working
HTML	Writing only	Writing only	N/A	N/A	N/A	not working	N/A

To do list

Note: this list is provisional, maintained by developers and can change without notice. If you are interested in contributing, please feel free to get in touch and submit a patch - thanks!

- Find out why BIFF2 files are corrupt when saved with frozen rows/cols activated.
- ~~Add row and column formats~~
- Add reading support for wikitable (Mediawiki) files
- xls reader crashes for some incorrectly written xls files (which Excel can read), see http://forum.lazarus.freepascal.org/index.php/topic,25624.0.html.
- ~~Improve color support: due to palette support colors may change from file to file currently.~~
- Add embedded images.
- Fix writing of cell comments to BIFF8 files.

Long-term:

- Provide a more common user interface to fpspreadsheet (setter/getter and properties instead of Read*/Write* methods, MyWorksheet.Cells[ARow, ACol]), make TCell a class, avoid the pointers PCell.

- ~~Store formatting in a format list of the workbook, not in the cell, to reduce memory usage.~~
- Use BIFF8 file-wide string storage instead of storing strings in cells (should reduce memory usage in case of many duplicate strings)
- Redo ooxml and ods readers based upon SAX/TXmlReader instead of DOM to reduce memory usage of large files.
- ~~Add an fpspreadsheetexport unit and component on "Data Export" tab similar to~~ fpdbfexport ~~FPSpreadsheetexport demo preferably with all export formats component.~~ Find a way to register export format to all formats (look into how lazreport pdf export performs a similar procedure).

Changelog

SVN

Changes with respect to current release

- (currently none)

Incompatible changes

- (currently none)

Version 1.2

This is the lastest stable release.

Changes with respect to v1.16

- Supporting new formulas: INDEX, ROUNDDOWN, ROUNDUP, COUNTIF, SUMF, AVERAGEIF, COUNTIFS, SUMIFS, AVERAGEIFS
- Supporting horizontal alignments haDistributed and haFilled
- Supporting charts
- Supporting defined names (named cells and ranges).

Incompatible changes

- (currently none)

Version 1.16

Changes with respect to v1.14

- XLSX reader can handle embedded as well as header/footer images now.
- Less strict format check by Excel2 reader since there are BIFF2 files with BOF record of Excel8 which Excel can read.
- Reading support for the Excel3 and Excel4 formats (unit *xlsbiff34*)
- ODS reader supports the node
- ODS reader is able to open encrypted files.

Incompatible changes

- (none)

Version 1.14

Changes with respect to v1.12

- Implementation of **conditional formats** for ods, xlsx and Excel xml.
- Implementation of **meta data** for ods, xlsx and Excel xml.
- Add a worksheet-based dataset in separate package (**TsWorksheetDataset**).

Incompatible changes

- Removed the deprecated worksheet properties DefaultRowHeight and DefaultColWidth. Use the corresponding Read/Write routines instead (ReadDefaultColWidth(units), WriteDefaultColWidth(units)).

Version 1.12

Changes with respect to v1.10x

- **Hide** (and unhide) rows and columns.
- Enforce **Page breaks** before rows and columns during printing by the Office applications.
- Full support (reading and writing) of the **ExcelXML** xml format (Excel 2003 and XP).
- Support of the color of worksheet tabs (Worksheet.TabColor) in xls biff8, xlsx and ods formats. The property is ignored by the visual WorkbookTabControl, though.
- Add TsWorksheetIndicator to visual controls.

Version 1.10.1

Changes with respect to v1.8x

- **Workbook, worksheet and cell protection** (read/write in BIFF2/BIFF5/BIFF8/OOXML/ODS, write in ExcelXML).
- New package **laz_fpspreadsheet_crypto** to decipher the encryption for worksheet-protection in xls files. Requires DCPcrypt.
- TsWorksheetGrid can **display embedded images**.
- **Drag and drop** in TsWorksheetGrid
- New, **highDPI-aware** component palette icons.
- New workbook optiond **boAbortReadingOnFormulaError** and **boIgnoreFormulas**.
- Formulas with **references to other sheets**, i.e. `'=Sheet1!A1+Sheet2!A2'`

Incompatible changes

- The field **FormulaValue** has been removed from the cell record. The cell formula now can be retrieved by calling `Worksheet.ReadFormula(cell)`.

Version 1.8

Changes with respect to v1.6x

- **"Rich-text" formatting** of label cells, i.e. assignment different fonts to groups of characters within the cell text. For this purpose, HTML codes (such as `<B>...</B>`) can be embedded in the cell text to identify the parts with different font (--> `TsWorksheet.WriteTextAsHTML`).
- **Searching** for cells with specified content in worksheet or workbook.
- Support for reading and writing of **HTML format**
- Support for writing of the **ExcelXML format** (Excel XP and 2003)

- Ability to use **user-provided file reader/writer classes** to extend FPSpreadsheet to new file formats.
- Readers and writers now support all the **line styles** of Excel8 and OOXML.
- xls, xlsx and ods readers/writers now support the **active worksheet** and **selected cell**.
- Ability to write to/read from the system's **clipboard** for copy & paste using the visual controls.
- Support for **print ranges** and **repeated header rows and columns** in the Office applications.
- Support for **embedded images** (currently only writing to xlsx and ods, no reading).
- Improved compatibility of **TsWorksheetGrid** with TStringGrid (`Cells[Col,Row]` property). Standalone application as an advanced StringGrid replacement.
- Support for **Right-to-left mode** in TsWorksheetGrid. In addition to the system-wide RTL mode, there are also parameters `BiDiMode` in the Worksheet and cells allowing to controls text direction at worksheet and cell level individually, like in Excel or LibreOffice Calc.
- Support of several **units** for specification of column widths and row heights.
- The library now supports **localization** using po files. Translations are welcome.
- **Zoom factor** read and written by the worksheet, and applied by the TsWorksheetGrid.
- Support of **column and row formats**
- Support of **hidden** worksheets

Incompatible changes

- VirtualMode was changed in order to be able to treat worksheets of the same workbook differently. `VirtualRowCount` and `VirtualColCount` are now properties of the worksheet, and similarly, the event handler `OnWriteCellData`. In older versions, these properties had belonged to the workbook.
- The worksheet methods `ReadAsUTF8Text` and `WriteUTF8Text` have been renamed to `ReadAsText` and `WriteText`, respectively. The old ones are still available and marked as deprecated; they will be removed in later versions.
- The public properties of `TsWorksheetGrid` using a `TRect` as parameter were modified to use the Left, Top, Right, Bottom values separately.
- The `PageLayout` is a class now, no longer a record. As a consequence, some array properties cannot be set directly any more, use the corresponding methods instead.
- Most of the predefined color constants were marked as deprecated; only the basic EGA colors will remain.
- Unit fpsNumFormatParser is integrated in fpsNumFormat. Old code which "uses" fpsNumFormatParser must "use" fpsNumFormat now.
- The source files of the laz_fpspreadsheet, laz_fpspreadsheet_visual and laz_fpspreadsheetexport_visual packages have been moved to separate folders in order to resolve some occasional compilation issues. Projects which do not use the packages but the path to the sources must adapt the paths.

Version 1.6

Changes with respect to v1.4.x

- TsWorkbookChartSource is a new component which facilitates creation of charts from non-contiguous spreadsheet data in various worksheets. It interfaces to a workbook via tha WorkbookSource component. In the long run, it will replace the older TsWorksheetChartSource which required contiguous x/y data blocks in the same worksheet.
- Major reconstruction of the cell record resulting in strong reduction of memory consumption per cell (from about 160 bytes per cell down to about 50)
- Implementation of a record helper for the TCell which simplifies cell formatting (no need to set a bit in UsedFormattingFields any more, automatic notification of visual controls)
- Comments in cells
- Background fill patterns
- Hyperlinks
- Enumerators for worksheet's internal AVLTrees for faster iteration using a for-in loop.
- Formatting of numbers as fractions.
- Improved number format parser for better recognition of Excel-like number formats.
- Page layout (page margins, headers, footer, used for only when printing in the Office applications - no direct print support in fpspreadsheet!)
- Improved color management: no more palettes, but direct rgb colors. More pre-defined colors.
- A snapshot of the wiki documentation is added to the library as chm help file.

Incompatible changes

- All type declarations and constants are moved from fpspreadsheet.pas to the new unit **fpstypes.pas**. Therefore, most probably, this unit has to be added to the uses clause.
- Because fpspreadsheet supports now **background** fill patterns the cell property `BackgroundColor` has been replaced by `Background`. Similarly, the `UsedFormattingFields` flag `uffBackgroundColor` is called `uffBackground` now.
- Another `UsedFormattingFields` flag has been dropped: **`uffBold`** . It is from the early days of fpspreadsheet and has become obsolete since the introduction of full font support. For achieving a bold type-face, now call `MyWorksheet.WriteFont(row, col, BOLD_FONTINDEX)`, or `Myworksheet.WriteFontStyle(row, col, [fssBold])`.
- **Iteration through cells** using the worksheet methods `GetFirstCell` and `GetNextCell` has been removed - it failed if another iteration of this kind was called within the loop. Use the new `for-in` syntax instead.
- Support for **shared formulas** has been reduced. The field `SharedFormulaBase` has been deleted from the TCell record, and methods related to shared formulas have been removed from TsWorksheet. Files containing shared formulas can still be read, the shared formulas are converted to multiple normal formulas.
- The **color palettes** of previous versions have been abandoned. `TsColor` is now a `DWord` representing the rgb components of a color (just like TColor does in the *graphics* unit), it is not an index into a color palette any more. The values of pre-defined colors, therefore, have changed, their names, however, are still existing. The workbook functions for palette access have become obsolete and were removed.

Version 1.4

Changes with respect to v1.2.x

- Full support for **string formulas**; calculation of RPN and string formulas for all built-in formulas either directly or by means of registration mechanism. Calculation occurs when a workbook is saved (activate workbook option `boCalcBeforeSaving`) or when cell content changes (active workbook option `boAutoCalc`).
- **Shared formulas** (reading for sfExcel5, sfExcel8, sfOOXML; writing for sfExcel2, sfExcel5, sfExcel8).
- Significant **speed-up** of writing of large spreadsheets for the xml-based formats (ods and xlsx), speed up for biff2; **speedtest demo** program
- **VirtualMode** allowing to read and write very large spreadsheet files without loading entire document representation into memory. Formatting of cells in VirtualMode.
- Demo program for database export using virtual mode and TFPSExport.
- Added db export unit allowing programmatic exporting datasets using **TFPSExport**. Similar export units are e.g. fpdbfexport, fpXMLXSDExport.
- Reader for **xlsx** files, now fully supporting the same features as the other readers.
- Reader/writer for **CSV files** based on CsvDocument.
- **Wikitables writer** supports now most of the fpspreadsheet formatting options (background color, font style, font color, text alignment, cell borders/line styles/line colors, merged cells, column widths, row heights); new "wikitablemaker" demo
- **Insertion and deletion of rows and columns** into a worksheet containing data.
- Implementation of **sorting** of a worksheet.
- Support of **diagonal "border" lines**
- **Logging of non-fatal error messages** during reading/writing (TsWorksheet.ErrorMsg)
- **Merged** cells

- **Registration** of currency strings for automatic conversion of strings to **currency values**
- A set of **visual controls** (TsWorkbookSource, TsWorkbookTabControl, TsSpreadsheetInspector, TsCellEdit, TsCellIndicator, TsCellCombobox, in addition to the already-existing TsWorksheetGrid) and pre-defined standard actions to facilitate creation of GUI applications.
- **Overflow cells** in TsWorksheetGrid: label cells with text longer than the cell width extend into the neighboring cell(s).

Incompatible changes

- The option `soCalcBeforeSaving` now belongs to the workbook, no longer to the worksheet, and has been renamed to `boCalcBeforeSaving` (it controls automatic calculation of formulas when a workbook is saved).
- The workbook property `ReadFormulas` is replaced by the option flag `boReadFormulas`. This means that you have to add this flag to the workbook's `Options` in order to activate reading of formulas.
- With full support of string formulas some features related to RPN formulas were removed:
  - The field `RPNFormulaResult` of TCell was dropped, as well as the element `cctRPNFormula` in the `TsContentType` set.
  - Sheet function identifiers were removed from the `TsFormulaElement` set, which was truncated after `fekParen`.
  - To identify a sheet function, its name must be passed to the function `RPNFunc` (instead of using the now removed `fekXXXX` token). In the array notation of the RPN formula, a sheet function is identified by the new token `fekFunc`.
  - The calling convention for registering user-defined functions was modified. It now also requires the Excel ID of the function (see "OpenOffice Documentation of Microsoft Excel Files", section 3.11, or unit *xlsconst* containing all token up to ID 200 and some above).
  - Code related to RPN formulas was moved to a separate unit, `fpsRPN`. Add this unit to the uses clause if you need RPN features.

License

LGPL with static linking exception. This is the same license as is used in the Lazarus Component Library.

References

Documentation

FPSpreadsheet is documented in this wiki file.

Every release of FPSpreadsheet is accompanied by a snapshot of the current wiki files. There is also an api documentation created from the embedded comments in the source files. Both files are in the chm format.

This wiki page is work in progress and updated whenever a new feature is added; therefore, its state corresponds to the svn trunk version of the package. If you work with an older stable version please use these "historic" wiki versions:

- Version 1.16: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=157438
- Version 1.14: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=149828
- Version 1.12: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=136886
- Version 1.10.1: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=118771
- Version 1.10: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=118441
- Version 1.8: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=107616
- Version 1.6.2: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=100723
- Version 1.6: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=91469
- Version 1.4: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=85299
- Version 1.2 and 1.2.1: wiki.lazarus.freepascal.org/index.php?title=FPSpreadsheet&oldid=81375

Wiki links

- Office Automation
- CsvDocument
- FPSpreadsheet: Examples
- RPN formulas in FPSpreadsheet
- FPSpreadsheet: List of formulas
- FPSpreadsheet tutorial: Writing a mini spreadsheet application
- FPSpreadsheet: Chart Tutorial

External Links

- Microsoft OLE Document Format
- Excel file format description
- Excel xls and PowerPoint ppt file dumper written in Python - very handy to list all contents of BIFF files (e.g. ./xls-dump.py file.xls) - http://cgit.freedesktop.org/libreoffice/contrib/mso-dumper/. A similar application is the "BIFFExplorer" which can be found in the *applications* folder of ccr.
- Icons used in the demo programs:
  - silk icons
  - fugue icons

FPSpreadsheet: List of formulas

Introduction

This is a list of the formulas supported by FPSpreadsheet.

The arguments can be **constants** of the given type, or **cells** containing values of the given type. Similar to the Office applications, type-checking is very relaxed, and data are automatically converted to the required type if possible.

Mathematical functions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for
ABS(num)	Returns the absolute value of a number	float	1	float	
ACOS(num)	Returns the arccosine (in radians) of a number	float (>= -1 and <= +1)	1	float	
ACOSH(num)	Returns the inverse hyperbolic cosine of a number	float (>= 1)	1	float	
ASIN(num)	Returns the arcsine (in radians) of a number	float (>= -1 and <= +1)	1	float	
ASINH(num)	Returns the inverse hyperbolic sine of a number	float	1	float	
ATAN(num)	Returns the arctangent (in radians) of a number	float	1	float	
ATANH(num)	Returns the inverse hyperbolic tangent of a number	float (> -1 and < +1)	1	float	
CEILING(num,signif)	Rounds a number up to next multiple of signif	float	2	float	sfExcel2
COS(num)	Returns the cosine of an angle (in radians)	float	1	float	
COSH(num)	Returns the hyperbolic cosine of a number	float	1	float	
DEGREES(num)	Converts an angle from radians to degrees	float	1	float	sfExcel2
EVEN(num)	Rounds a pos number up, a neg number down to the next even integer	float	1	integer	sfExcel2
EXP(num)	Calculates the exponential function of a number	float	1	float	
FACT(num)	Calculates the factorial of a number	integer	1	float	
FLOOR(num,signif)	Rounds a number down to next multiple of signif	float	2	float	sfExcel2
INT(num)	Returns the integer portion of a number, rounds down	float	1	integer	
LN(num)	Calculates the natural logarithm of a number	float (> 0)	1	float	
LOG(num [, base])	Calculates the logarithm of a number to a specified base. base, if omitted, is 10.	float (> 0)	1 or 2	float	
LOG10(num)	Calculates the base-10 logarithm of a number	float (> 0)	1	float	
ODD(num)	Rounds a pos number up, a neg number down to the next odd integer	float	1	integer	sfExcel2
PI()	Returns the mathematical constant π (3.14159265358979)	none	0	float	
POWER(num, exponent)	Returns the result of a number raised to a given power	float	2	float	sfExcel2
RADIANS(num)	Converts an angle from degrees to radians	float	1 or 2	float	sfExcel2
RAND()	Returns a random number between 0 and 1	none	0	float	
ROUND(num, digits)	Returns a number rounded to a specified number of digits	float, integer	2	float	
ROUNDDOWN(num, digits)	Returns a number rounded down to a specified number of digits	float, integer	2	float	sfExcel2
ROUNDUP(num, digits)	Returns a number rounded up to a specified number of digits	float, integer	2	float	sfExcel2
SIGN(num)	Returns the sign of a number	float	1	integer	
SIN(num)	Returns the sine of an angle (in radians)	float	1	float	
SINH(num)	Returns the hyperbolic sine of a number	float	1	float	
TAN(num)	Returns the tangent of an angle (in radians)	float (<> (integer)*π/2)	1	float	
TANH(num)	Returns the hyperbolic tangent of a number	float	1	float	

Statistical functions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for
AVEDEV(num1 [, num2, ...])	Average value of absolute deviations of data from their mean.	float	> 1	float	sfExcel2
AVERAGE(num1 [, num2, ...])	Average value of a series of numbers	float	> 1	float	

AVERAGEIF(range, condition [, value_range])	Average value of data in value_range if cells in range meet condition	cell ranges (like A1:D5) condition is value, string or cell	2 or 3	float	sfExcel2 sfExcel5 sfExcel8
COUNT(value1 [, value2, ...])	Counts cells and arguments containing numbers	any	> 1	integer	
COUNTA(value1 [, value2, ...])	Counts the number of non-empty cells and arguments	any	> 1	integer	
COUNTBLANK(range)	Counts the number of empty cells in a range	cell range (like A1:D5)	1	integer	sfExcel2
COUNTIF(range, condition)	Counts the cells in range which meet condition	cell range (like A1:D5) condition is value, string or cell	2	integer	sfExcel2
MAX(num1 [, num2, ...])	Returns the largest value from the numbers provided	float	> 1	float	
MIN(num1 [, num2, ...])	Returns the smallest value from the numbers provided	float	> 1	float	
PRODUCT(num1 [, num2, ...])	Calculates the product of the numbers provided	float	> 1	float	
STDEV(num1 [, num2, ...])	Returns the standard deviation of a population based on a ample of numbers	float	> 1	float	
STDEVP(num1 [, num2, ...])	Returns the standard deviation of a population based on an entire population	float	> 1	float	
SUM(num1 [, num2, ...])	Calculates the sum of the numbers provided	float	> 1	float	
SUMIF(range, condition [, value_range])	Adds the data in value_range if cells in range meet condition	cell ranges (like A1:D5) condition is value, string or cell	2 or 3	float	sfExcel2
SUMSQ(num1 [, num2, ...])	Returns the sum of the squares of a series of numbers	float	> 1	float	sfExcel2
VAR(num1 [, num2, ...])	Returns the variance of a population based on a sample of numbers	float	>1	float	
VARP(num1 [, num2, ...])	Returns the variance of a population based on an entire population	float	> 1	float	

Date/time funtions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for
DATE(year, month, day)	Calculates a serial date number from year, month and day	integer	3	date/time	
DATEDIF(start_date, end_date, interval)	Calculates the difference between two date value based on a given interval	start_date, end_date: date/time interval is a string: Y = number of years, M = number of months, D = number of days	3	integer	sfExcel2
DATEVALUE(date_string)	Converts a (date) string to a date/time value.	string	1	date/time	
DAY(value)	Extracts the day number (1..31) of a date value.	date/time, number, string	1	integer	
HOUR(value)	Extracts the hour (0..23) of a time value.	date/time, number, string	1	integer	
MINUTE(value)	Extracts the minute (0..59) of a time value.	date/time, number, string	1	integer	
MONTH(value)	Extracts the month number (1..12) of a date value.	date/time, number, string	1	integer	
NOW()	Returns the current system date and time. Will refresh whenever the worksheet recalculates.	none	0	date/time	
SECOND(value)	Extracts the second (0..59) of a time value.	date/time, number, string	1	integer	
TIME(year, month, day)	Calculates a date/time value from hours, minutes and seconds	integer	3	date/time	
TIMEVALUE(time_string)	Converts a (time) string to a date/time value.	string	1	date/time	
TODAY()	Returns the current system date	none	0	date/time	
WEEKDAY(value [, type])	Returns a number code for the weekday of a date	value: date/time, number, string type=0: Sunday=1, Saturday=7 (default) type=1: Monday=1, Sunday=7 type=2: Monday=9, Sunday=6	1 or 2	integer	
YEAR(value)	Extracts the year of a date value.	date/time, number, string	1	integer	

String functions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for
CHAR(ascii_value)	Returns the character based on its ASCII value	integer	1	string	
CODE(text)	Returns the ASCII code of the first character of a string	string	1	integer	
CONCATENATE(text1 [, text2, ...])	Joins strings together	float	> 1	string	
EXACT(text1, text2)	Compares two strings (case-sensitive)	float	2	boolean	

LEFT(text [, count])	Returns the left-most characters of a string	text: string count: integer (default 1)	1 or 2	string	
LEN(text)	Returns the character count of a string	string	1	integer	
LOWER(text)	Converts a string to lower-case characters	string	1	string	
MID(text, start_pos, count)	Returns part of a string	text: string start_pos, count: integer	3	string	
REPLACE(text, start_pos, count, new_text)	Replaces a sequence of characters in a string with another string	text: string start_pos, count: integer new_text: string	4	string	
REPT(text, count)	Repeats a text a specified number of times	text: string count: integer	2	string	
RIGHT(text [, count])	Returns the right-most characters of a string	text: string count: integer (default 1)	1 or 2	string	
SUBSTITUTE(text, old_text, new_text [, nth_appearance])	Replaces part of a string with another string	text, old_text, new_text: string nth_appearance: integer (default: replace all)	3 or 4	string	
TRIM(text)	Removes leading and trailing spaces from a string	string	1	string	
UPPER(text)	Converts a string to upper-case characters	string	1	string	
VALUE(text)	Converts a string representing a number to a number	string	1	float	

Logical functions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for
AND(condition1 [, condition2, ...])	Calculates the logical AND of several boolean values	boolean	any	boolean	
FALSE()	Returns the boolean value FALSE	none	0	boolean	
IF(condition, value_true [, value_false])	Returns value_true if condition is true, or value_false (or false) if condition is false	condition: boolean value_true, value_false: any type	2 or 3	any type	
NOT(value)	Inverts a boolean value	boolean	1	boolean	
OR(condition1 [, condition2, ...])	Calculates the logical OR of several boolean values	boolean	any	boolean	
TRUE()	Returns the boolean value TRUE	none	0	boolean	

Info functions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for
ERROR.TYPE(value)	Returns the numeric representation of one of the errors in Excel (1 = #NULL!, 2 = #DIV/0!, 3 = #VALUE!, 4 = #REF!, 5 = #NAME?, 6 = #NUM!, #N/A else).	cell	1	integer	sfExcel2
ISBLANK(value)	Checks for blank or null values.	any, usually cell	1	boolean	
ISERR(value)	Returns TRUE if value is an error but not #N/A	any, usually cell	1	boolean	
ISERROR(value)	Returns TRUE if value is an error value (#N/A, #VALUE!, #REF!, #DIV/0!, #NUM!, #NAME? or #NULL).	any, usually cell	1	boolean	
ISLOGICAL(value)	Returns TRUE if value is boolean	any, usually cell	1	boolean	
ISNA(value)	Returns TRUE if value is a #N/A error	any, usually cell	1	boolean	
ISNONTEXT(value)	Returns TRUE if value is not a string	any, usually cell	1	boolean	
ISNUMBER(value)	Returns TRUE if value is a number	any, usually cell	1	boolean	
ISREF(value)	Returns TRUE if value is a cell reference	any, usually cell	1	boolean	
ISTEXT(value)	Returns TRUE if value is a string	any, usually cell	1	boolean	

Lookup/reference functions

Calling prototye	Meaning	Argument types	Arguments	Result type	Not for

`ADDRESS(row, col [, ref_type] [, ref_style], [sheet_name])`	Returns a text representation of a cell address `ref_type` is the type of reference to use: 1=absolute (default); 2=relative column, absolute row; 3=absolute column, relative row; 4=relative `ref_style` if true (default) means: address in A1 dialect, otherwise in R1C1 `sheet_name=name` of the worksheet	3x integer boolean string	2 (up to 5)	string	sfExcel2
`COLUMN( [reference] )`	Returns the (1-based) column number of a cell reference. `reference` is a reference to a cell or range of cells. If omitted, it is assumed that the reference is the cell address in which the COLUMN function has been entered.	string	1 (or 0)	integer	
`HYPERLINK(link [, display_name])`	Adds a hyperlink	string	1 or 2	string (hyperlink)	sfExcel2, sfExcel5

`INDEX(address, row , [col])`	Returns the cell content at the intersection of the given row and colum of the 2D cell range. row and column indices are relative to the cell range (1-based). The last parameter can be omitted when the input array is 1-dimensional (i.e., a single column or row).
	`|| 1-D or 2-D cell range`
	`integer`
	`[integer] || align="center"|2 or 3 || align="center"|(cell value)`				

`INDIRECT(address)`	Returns cell reference based on address string	string	1	cell	

`MATCH(value, array [, type])`	Searches for a value in a 1-D array and returns the relative position of that item.
	type = 1 (default) finds the largest value <= `value` (assumes an array in ascending order)
	type = -1 finds the smallest value >= `value` (assumes an array in descending order)
	type = 0 finds the first value equal to `value` (no requirement on sort order, array can contain strings with wildcard '?').
	`|| value: float or text`
	`array: 1-D cell range (e.g., A1:A9, or A1:G1)`
	`type: integer || align="center"|2 or 3 || align="center"|integer ||`				

`ROW( [reference] )`	Returns the (1-based) row number of a cell reference. `reference` is a reference to a cell or range of cells. If omitted, it is assumed that the reference is the cell address in which the ROW function has been entered.	string	1 (or 0)	integer	

Online version

FPSpreadsheet: Examples

Requirements to create an FPSpreadsheet application

To create a project which uses the fpspreadsheet library, add the **fpspreadsheet_pkg** package to it's Lazarus project, or add the base directory of fpspreadsheet to you compiler options if using another IDE.

Units

The entire FPSpreadsheet package consists of several units. A spreadsheet application typically "uses" the following units:

- *fpspreadsheet*: implements **TsWorkbook** and **TsWorksheet** and the **basic file reading/writing methods**
- *fpstypes*: declares most of the **data types** and **constants** used throughout the package. **Note:** in older versions these declarations were contained in fpspreadsheet.pas.
- the unit(s) implementing the reader/writer for a given **file format**, e.g. *xlsbiff8* for binary Excel files. If the application will be able to handle all formats "use" the unit *fpsallformats*.

The next units are required only occasionally:

- *fpsutils*: a collection of **utility functions** that are occasionally needed (e.g. conversion of col/row indexes to Excel-like cell address string).
- *fpscell*: this unit is required if you use **direct cell formatting** (e.g. `cell^.BackgroundColor := scYellow`) instead of calling the corresponding worksheet method (`MyWorksheet.WriteBackgroundColor(cell, scYellow)`, in this example).
- *fpsnumformat*: collects all utility functions related to **number formats**.

All other units are probably not needed at the application level. In case of the **visual spreadsheet controls**, the needed units usually are inserted at design-time automatically.

Sample projects in the fpspreadsheet installation folder

A bunch of sample projects accompanies the FPSpreadsheet installation. They can be found in the folder "examples". Here is a brief description of these sample projects

- **db_import_export** is an example showing how to export a large database table to a spreadsheet file using virtual mode or **TFPSExport**. It also shows importing the spreadsheet into a database using virtual mode.

- **fpsspeedtest** compares the effect of file format and various reading/writing parameters on the speed of writing and reading very large spreadsheet files. Again, please run the write test first which create the test files used for the read test.
- Folder **read_write**:
  - **excel2demo** contains command-line programs for writing and reading Excel 2.x xls files. Please run the write demo before the read demo so the required spreadsheet file is generated.
  - **excel5demo**, like excel2demo, but for Excel 5 xls files.
  - **excel8demo**, like excel2demo, but for Excel 97-2003 xls files.
  - **csvdemo**, like excel2demo, but for CSV files.
  - **htmldemo**, like excel2demo, but for HTML file (currently writing only).
  - **ooxmldemo**, like excel2demo, but for the new Excel xlsx files.
  - **opendocdemo**, like excel2demo, but for OpenOffice/LibreOffice ods files
  - **wikitabledemo**, like excel2demo, but for wiki table files. Note that the write example currently writes a format that the read example cannot understand.
- **other**: simple commandline programs showing various aspects of the fpspreadsheet package. Have a look at *readme.txt* for more details.
- Folder **visual**:
  - **fpschart** shows the application of the TsWorksheetChartSource and TsWorkbookChartSource and the interaction with the TAChart plotting package.
  - **fpsctrls** and **fpsctrls_no_install** create a GUI spreadsheet application with a minimum amount of written code; the latter demo is good for testing because it does not require installation of the FPSpreadsheet packages. Step-by-step instructions on how *fpsctrls* is made can be found in the FSpreadsheet tutorial.
  - **fpsgrid** and **fpsgrid_no_install** show the basic application of the TsWorksheetGrid without using the TsWorkbookSource component; the latter demo does not require installation of any FPSpreadsheet package.
  - **wikitablemaker** is a small application for creation of code to be used for tables on wiki pages. Type the data into a TsWorksheetGrid (or load an existing spreadsheet file), go to page "Code" to see the generated wiki code, click "Copy to clipboard" and paste the code into the wiki page.
- **spready** is an extended application of the the entire library showing spreadsheet files with formatting, editing of cells, etc. Since it is a stand-alone application it has been moved to the folder *applications/spready' of the* Lazarus Components and Code Library.

Code examples

Opening an existing spreadsheet

To open a spreadsheet while specifying a particular format to use use `ReadFromFile` with two parameters:

```
MyWorkbook.ReadFromFile(AFileName, sfExcel5);
```

It is also possible to call `ReadFromFile` with only one parameter, the filename. Then the workbook will use the extension to auto-detect the file format. In case of the ambiguous extension *.xls* (Excel 2-8) it will simply try various possibilities until one works. Although typical fingerprint byte patterns are checked now it is still possible that an exception will be raised for each incorrect format if run from the IDE at designtime; this does not occur at runtime.

```
MyWorkbook.ReadFromFile(AFileName);
```

Writing a spreadsheet to file based on extension

Similar to the `ReadFromFile` routine, there is also a `WriteToFile` procedure to determine the

spreadsheet's type based on the filename suffix. It uses the `GetFormatFromFileName` routine in the previous section's code, so the actual code is simple. However, it will always write files with a given extension using the latest format that uses that extension (e.g. Excel *.xls* files will be written as `sfExcel8`), so if you want to write them in an earlier format, you have to use the base routine.

As above, this code patches the *fpspreadsheet.pas* unit.

```
procedure TsWorkbook.WriteToFile(const AFileName: string; const AOverwrit
var SheetType: TsSpreadsheetFormat;
begin
  if getFormatFromFileName(AFileName, SheetType) then
    WriteToFile(AFileName, SheetType, AOverwriteExisting)
  else raise Exception.Create(Format(
    '[TsWorkbook.WriteToFile] Attempted to save a spreadsheet by extensi
end;
```

Converting between two spreadsheet formats

```
program ods2xls;

{$mode delphi}{$H+}

uses
  Classes, SysUtils,
  fpstypes, fpspreadsheet, fpsallformats, fpspreadsheet_pkg;

const
  INPUT_FORMAT = sfOpenDocument;
  OUTPUT_FORMAT = sfExcel8;

var
  MyWorkbook: TsWorkbook;
  MyDir: string;
begin
  // Initialization
  MyDir := ExtractFilePath(ParamStr(0));

  // Convert the spreadsheet
  MyWorkbook := TsWorkbook.Create;
  try
    MyWorkbook.ReadFromFile(MyDir + 'test.ods', INPUT_FORMAT);
    MyWorkbook.WriteToFile(MyDir + 'test.xls', OUTPUT_FORMAT);
  finally
    MyWorkbook.Free;
  end;
end.
```

Iterating through all worksheets

```
var
  MyWorkbook: TsWorkbook;
  MyWorksheet: TsWorksheet;
  i: Integer;
begin
  // Here load MyWorkbook from a file or build it

  for i := 0 to MyWorkbook.GetWorksheetCount() - 1 do
  begin
    MyWorksheet := MyWorkbook.GetWorksheetByIndex(i);
    // Do something with MyWorksheet
  end;
```

Iterating through cells

The first idea is to use a simple **for-to** loop:

```
var
  MyWorksheet: TsWorksheet;
  col, row: Cardinal;
  cell: PCell;
begin
  for row:=0 to MyWorksheet.GetLastRowIndex do
    for col := 0 to MyWorksheet.GetLastColIndex do
    begin
      cell := MyWorksheet.FindCell(row, col);
      WriteLn(MyWorksheet.ReadAsUTF8Text(cell));
    end;
end;
```

`FindCell` initiates a search for a cell independently of the previously found cell. Cells, however, are are organized interally in a sorted tree structure, and each cell "knows" its previous and next neighbors. Moreover, `FindCell` wastes time on searching for non-existing cells in case of sparsely-occupied worksheets. In general, it is more efficient to use the **for-in** syntax which takes advantage of the internal tree structure by means of special **enumerators**. Note that there are also dedicated enumerators for searching along rows, columns or in cell ranges:

```
var
  MyWorksheet: TsWorksheet;
  cell: PCell;
begin
  // Search in all cells
  for cell in Myworksheet.Cells do
    WriteLn(MyWorksheet.ReadAsText(cell));

  // Search in column 0 only
  for cell in MyWorksheet.Cells.GetColEnumerator(0) do
    WriteLn(MyWorksheet.ReadAsText(cell));

  // Search in row 2 only
  for cell in MyWorksheet.Cells.GetRowEnumerator(2) do
    WriteLn(MyWorksheet.ReadAsText(cell));

  // Search in range A1:C2 only (rows 0..1, columns 0..2)
  for cell in MyWorksheet.Cells.GetRangeEnumerator(0, 0, 1, 2) do
    WriteLn(MyWorksheet.ReadAsText(cell));
end;
```

Excel 5 example

Note: at least with fpspreadsheet from trunk (development version), this example requires (at least) Lazarus *avglvltree.pas, lazutf8.pas, asiancodepagefunctions.inc, asiancodepages.inc* and *lconvencoding.pas* (in the *$(LazarusDir)\components\lazutils* directory)

```
{
excel5demo.dpr

Demonstrates how to write an Excel 5.x file using the fpspreadsheet libr

You can change the output format by changing the OUTPUT_FORMAT constant

AUTHORS: Felipe Monteiro de Carvalho
}
program excel5demo;

{$mode delphi}{$H+}

uses
  Classes, SysUtils, fpstypes, fpspreadsheet, fpsallformats, laz_fpsprea

const
  OUTPUT_FORMAT = sfExcel5;

var
  MyWorkbook: TsWorkbook;
  MyWorksheet: TsWorksheet;
  MyFormula: TsRPNFormula;
  MyDir: string;
begin
  // Initialization
  MyDir := ExtractFilePath(ParamStr(0));

  // Create the spreadsheet
  MyWorkbook := TsWorkbook.Create;
```

```
  try
    MyWorksheet := MyWorkbook.AddWorksheet('My Worksheet');

    // Write some number cells
    MyWorksheet.WriteNumber(0, 0, 1.0);
    MyWorksheet.WriteNumber(0, 1, 2.0);
    MyWorksheet.WriteNumber(0, 2, 3.0);
    MyWorksheet.WriteNumber(0, 3, 4.0);

    // Write the formula E1 = A1 + B1
    MyWorksheet.WriteFormula(0, 4, 'A1+B1');

    // Creates a new worksheet
    MyWorksheet := MyWorkbook.AddWorksheet('My Worksheet 2');

    // Write some string cells
    MyWorksheet.WriteText(0, 0, 'First');
    MyWorksheet.WriteText(0, 1, 'Second');
    MyWorksheet.WriteText(0, 2, 'Third');
    MyWorksheet.WriteText(0, 3, 'Fourth');

    // Save the spreadsheet to a file
    MyWorkbook.WriteToFile(MyDir + 'test' + STR_EXCEL_EXTENSION, OUTPUT_F
  finally
    MyWorkbook.Free;
  end;
end.
```

Reading and writing of CSV files

CSV files (CSV = comma-separated values) are plain text files without metadata. Therefore, additional information for correct reading of writing from/to a worksheet is required. The global record `CSVParams` makes available fundamental settings for this purpose:

```
type
  CSVParams: TsCSVParams = record     // W = writing, R = reading, RW = re
    SheetIndex: Integer;              // W: Index of the sheet to be writ
    LineEnding: TsCSVLineEnding;      // W: Specification for line ending
    Delimiter: Char;                  // RW: Column delimiter
    QuoteChar: Char;                  // RW: Character for quoting texts
    Encoding: String;                 // RW: Encoding of file
    DetectContentType: Boolean;       // R: try to convert strings to con
    NumberFormat: String;             // W: if empty write numbers like i
    AutoDetectNumberFormat: Boolean;  // R: automatically detects decimal
    TrueText: String;                 // RW: String for boolean TRUE
    FalseText: String;                // RW: String for boolean FALSE
    FormatSettings: TFormatSettings; // RW: add'l parameters for convers
  end;
```

This record contains fields which are evaluated for reading only, writing only, or for both - see the attached comments.

A common situation is to read a file using a number decimal separator which is different from the system's decimal separator: Suppose you are on a European system in which the decimal separator is a comma, but the csv file to be read originates from a machine which uses a decimal point. And suppose

also, that the file contains tab characters as column separator instead of the default comma. In this case, simply set the `CSVParams.FormatSettings.DecimalSeparator` to `'.'`, and the `CSVParams.Delimiter` to `#9` (TAB character) before reading the file:

```
uses
  fpstypes, fpspreadsheet, fpscsv;
var
  MyWorkbook: TsWorkbook;
begin
  CSVParams.FormatSettings.DecimalSeparator := '.';
  CSVParams.Delimiter := #9;
  MyWorkbook := TsWorkbook.Create;
  try
    MyWorkbook.ReadFromFile('machine-data.csv', sfCSV);
  finally
    MyWorkbook.Free;
  end;
end;
```

Note: If you want to give the user the possibility to interactively modify the `CSVParams` record have a look at the unit ''scsvparamsform.pas'' of the ''spready'' demo which provides a ready-to-use dialog.

Converting a database to a spreadsheet

The easiest solution is to use the DatasetExport component.

If you need to have more control over the process, use something like:

```
program db5xls;

{$mode delphi}{$H+}

uses
  Classes, SysUtils,
  // add database units
  fpstypes, fpspreadsheet, fpsallformats;

const OUTPUT_FORMAT = sfExcel5;

var
  MyWorkbook: TsWorkbook;
  MyWorksheet: TsWorksheet;
  MyDatabase: TSdfDataset;
  MyDir: string;
  i, j: Integer;
begin
  // Initialization
  MyDir := ExtractFilePath(ParamStr(0));

  // Open the database
  MyDatabase := TSdfDataset.Create;
  MyDatabase.Filename := 'test.dat';
  // Add table description here
  MyDatabase.Active := True;

  // Create the spreadsheet
  MyWorkbook := TsWorkbook.Create;
  MyWorksheet := MyWorkbook.AddWorksheet('My Worksheet');

  // Write the field names
  for i := 0 to MyDatabase.Fields.Count - 1 do
    MyWorksheet.WriteText(0, i, MyDatabase.Fields[i].FieldName);

  // Write all cells to the worksheet
  MyDatabase.First;
  j := 0;
  while not MyDatabase.EOF do
  begin
    for i := 0 to MyDatabase.Fields.Count - 1 do
      MyWorksheet.WriteText(j + 1, i, MyDatabase.Fields[i].AsString);

    MyDatabase.Next;
    Inc(j);
  end;

  // Close the database
  MyDatabase.Active := False;
  MyDatabase.Free;

  // Save the spreadsheet to a file
  MyWorkbook.WriteToFile(MyDir + 'test' + STR_EXCEL_EXTENSION, OUTPUT_FOR
  MyWorkbook.Free;
end.
```

Converting a large database table to a spreadsheet using virtual mode

Note: The example program in ''examples\db_import_export'' shows a demonstration of using virtual mode to export datasets to spreadsheet files.

We want to write a large database table to a spreadsheet file. The first row of the spreadsheet is to show the field names in bold type face and with a gray background.

Normally, FPSpreadsheet would load the entire representation of the spreadsheet into memory, so we'll use virtual mode to minimize memory usage.

```
type
  TDataProvider = class;

var
  MyWorkbook: TsWorkbook;
  MyWorksheet: TsWorksheet;
  MyDatabase: TSdfDataset;
  MyDir: string;
  MyHeaderTemplateCell: PCell;
  DataProvider: TDataProvider;

// Implement TDataProvider here - see below...

begin
  // Initialization
  MyDir := ExtractFilePath(ParamStr(0));

  // Open the database
  MyDatabase := TSdfDataset.Create;
  try
    MyDatabase.Filename := 'test.dat';
    // Add table description here
    MyDatabase.Active := True;

    // Create the spreadsheet
    MyWorkbook := TsWorkbook.Create;
    try
      MyWorksheet := MyWorkbook.AddWorksheet('My Worksheet');

      // Create the template cell for the header line, we want the
      // header in bold type-face and gray background color
      // The template cell can be anywhere in the workbook, let's just se
      MyWorksheet.WriteFontStyle(0, 0, [fssBold]);
      MyWorksheet.WriteBackgroundColor(0, 0, scGray);
      // We'll need this cell again and again, so let's save the pointer
      MyHeaderTemplateCell := MyWorksheet.Find(0, 0);

      // Enter virtual mode
      MyWorkbook.Options := MyWorkbook.Options + [boVirtualMode];

      // Define number of columns - we want a column for each field
      MyWorksheet.VirtualColCount := MyDatabase.FieldCount;

      // Define number of rows - we want every record, plus 1 row for the
      MyWorksheet.VirtualRowCount := MyDatabase.RecordCount + 1;
```

```
      // Link the event handler which passes data from database to sprea
      MyWorksheet.OnWriteCellData := @DataProvider.WriteCellData;

      // Write all cells to an Excel8 file
      // The data to be written are specified in the OnWriteCellData eve
      MyWorkbook.WriteToFile(MyDir + 'test.xls', sfExcel8);

    finally
      // Clean-up
      MyWorkbook.Free;
    end;

  finally
    // Close the database & clean-up
    MyDatabase.Active := False;
    MyDatabase.Free;
  end;

end.
```

What is left is to write the event handler for `OnWriteCellData`. For the command-line program above we setup a particular data provider class (in a gui program the event handler can also be a method of any form):

```
type
  TDataProvider = class
    procedure WriteCellData(Sender: TsWorksheet; ARow, ACol: Cardinal; va
  end;

procedure TDataProvider.WriteCellData(Sender: TsWorksheet; ARow, ACol: Ca
begin
  // Let's handle the header row first:
  if ARow = 0 then begin
    // The value to be written to the spreadsheet is the field name.
    AValue := MyDatabase.Fields[ACol].FieldName;
    // Formatting is defined in the HeaderTemplateCell.
    AStyleCell := MyHeaderTemplateCell;
    // Move to first record
    MyDatabase.First;
  end else begin
    // The value to be written to the spreadsheet is the record value in
    // No special requirements on formatting --> leave AStyleCell at its
    AValue := MyDatabase.Fields[ACol].AsVariant;
    // Advance database cursor if last field of record has been written
    if ACol = MyDatabase.FieldCount-1 then MyDatabase.Next;
  end;
end;
```

Categories: FPSpreadsheet

Online version

FPSpreadsheet tutorial: Writing a mini spreadsheet application

Introduction

FPSpreadsheet is a powerful package for reading and writing spreadsheet files. The main intention is to provide a platform which is capable of native export/import of an application's data to/from the most important spreadsheet file formats without having these spreadsheet applications installed.

Soon, however, the wish arises to use this package also for editing of file content or formatting. For this purpose, the library contains a dedicated grid control, the FPSpreadsheetGrid, which closely resembles the features of a worksheet of a spreadsheet application. The demo "spready" which comes along with FPSpreadsheet demonstrates usage of this grid. Along with a bunch of formatting options, this demo still comes up to more than 1400 lines of code in the main form unit. Therefore, a set of visual controls was developed which greatly simplify creation of spreadsheet applications.

It is the intention of this tutorial to write a simple spreadsheet program on the basis of these controls.

Although most of the internal structure of the FPSpreadsheet library is covered by the visual controls it is recommended that you have some knowledge of FPSpreadsheet. Of course, you should not have a basic understanding of Lazarus and FPC, and you must know how to work with the object inspector of Lazarus.

Visual FPSpreadsheet Controls

FPSpreadsheet exposes non-visual classes, such as TsWorkbook, TsWorksheet etc. This keeps the library general enough for all kind of Pascal programs. For GUI programs, on the other hand, some infrastructure is needed which relates the spreadsheets to forms, grids, and other controls.

TsWorkbookSource

The heart of the visual FPSpreadsheet controls is the **TsWorkbookSource** class. This provides a link between the non-visual spreadsheet data and the visual controls on the form. Its purpose is similar to that of a TDataSource component in database applications which links database tables or queries to dedicated "data-aware" controls.

All visual FPSpreadsheet controls have a property `WorkbookSource` which links them into the information chain provided by the TsWorkbookSource. The WorkbookSource keeps a list of all controls attached. Internally, these controls are called "listeners" because they listen to information distributed by the WorkbookSource.

The workbook and worksheets use events to notify the WorkbookSource of all relevant changes: changes in cell content or formatting, selecting other cells, adding or deleting worksheet etc. Information on these changes is passed on to the listening controls, and they react in their own specialized way on these changes. If, for example, a new worksheet is added to a workbook the visual TsWorkbookTabControl creates a new tab for the new worksheet, and the TsWorksheetGrid loads the new worksheet into the grid.

TsWorkbookTabControl

This is a tabcontrol which provides a tab for each worksheet of the current workbook. The tab names are identical with the names of the worksheets. Selecting another tab is communicated to the other visual spreadsheet controls via the WorkbookSource.

TsWorksheetGrid

This is a customized DrawGrid descendant of the LCL and displays cells of the currently selected worksheet. The texts are not stored in the grid (like a StringGrid would do), but are taken from the TsWorksheet data structure. Similarly, the worksheet provides the information of how each cell is formatted. Like any LCL grid it has a bunch of properties and can be tuned for many applications by adapting its `Options`. The most important one will be described below.

Note: The `TsWorksheetGrid` can also be operated without a `TsWorkbookSource`. For this purpose it provides its own set of methods for reading and writing files.

TsCellEdit

The typical spreadsheet applications provide a line for editing formulas or cell content. This is the purpose of the **TsCellEdit**. It displays the content of the active cell of the worksheet which is the same as the active cell of the WorksheetGrid. If editing is finished (by pressing ENTER, or by selecting another cell in the WorksheetGrid) the new cell value is transferred to the worksheet. Internally, the TsCellEdit is a memo control, i.e. it is able to process multi-line text correctly. Use Ctrl + ENTER to insert a forced line-break.

TsCellIndicator

This is a TEdit control which displays the address of the currently selected cell in Excel notation, e.g. 'A1' if the active cell is in the first row and first column (row = 0, column = 0). Conversely, if a valid cell address is entered into this control the corresponding cell becomes active.

TsCellCombobox

This combobox can be used to modify various cell properties by selecting values from the dropdown list. The property affected is determined by the `CellFormatItem` of the combobox:

- `cfiFontName`: the list contains he names of all fonts available on the current system. If an item is selected the corresponding font is used to format the cell of the currently selected cells.
- `cfiFontSize`: the list contains the most typical font sizes used in spreadsheets. Selecting an item sets the font size of the currently selected cells accordingly.
- `cfiFontColor`: the list contains all colors of the workbook's palette. The selected color is assigned to the font of the selected cells.
- `cfiBackgroundColor`: like `cfiFontColor` - the selected color is used as background fill color

of the selected cells.

TsSpreadsheetInspector

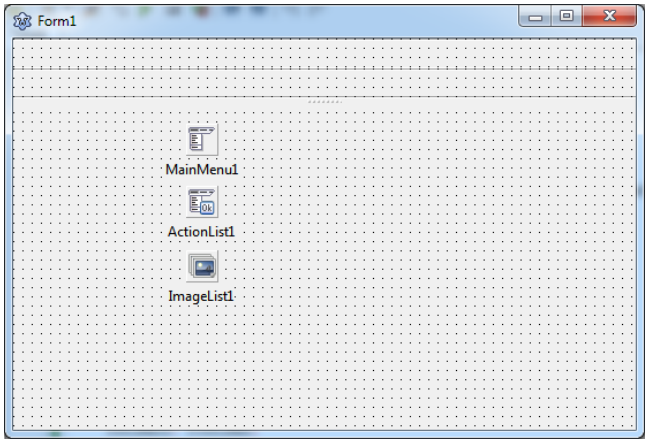
Inherits from TValueListEditor and displays name-value pairs for properties of the workbook, the selected worksheet, and the content and formatting of the active cell. It's main purpose is to help with debugging.

Writing a spreadsheet application

Enough of theory, let's get started. Let's write a small spreadsheet application. Sure - it cannot compete with the spreadsheets of the main Office applications like Excel or Open/LibreOffice, but it has all the main ingredients due to FPSpreadsheet. And using the FPSpreadsheet controls allows to achieve this with minimum lines of code.

Preparations

Create a new project and store it in a folder of your liking.



Since Office applications have a menu and a toolbar add a **TMainMenu** and a **TToolbar** component to the form. (You could even mimic the ribbon user interface of the new Microsoft applications by adding a **TSpkToolbar** from Lazarus Code and Components Repository, but be aware that this component does not yet provide all the features of a standard toolbar).

In fact, we will be needing **another toolbar** for the formula edit line. As you will see later, it will be resizable; as size control add a **TSplitter** to the form and top-align it such that it is positioned underneath the two toolbars. In order to keep a minimum size of the toolbar you should establish constraints: Look at the current height of the toolbar and enter this number into the `MinHeight` field of the `Constraints` property of the toolbar. To separate the formula toolbar from the rest of the main form, activate the option `ebBottom` of the `EdgeBorders` property of the second toolbar.

Since menu and toolbars will have to handle same user actions it is advantageous to provide a **TActionList** to store all possible actions. If assigned to the menu items and toolbuttons both will react on user interaction in the same way without any additional coding. And: The FPSpreadsheet visual controls package contains a bunch of spreadsheet-related standard actions ready to use.

The toolbar of the completed application will contain a lot of of icons. Therefore, we need a **TImageList** component which has to be linked to the `Images` property of the TMainMenu, the TToolbars, and the TActionList. Where to get icons? You can have a look in the folder `images` of your Lazarus installation where you'll find standard icons for loading and saving etc. This is a subset of the famfamfam SILK icon library. Another huge icon set is the Fugue icon collection. Both collections are licensed as "Creative commons" and are free even for commercial use, provided that appropriate reference is given in the created programs. When selecting icons prefer the png image format, and make sure to use always the same size, usually 16x16 pixels.

Setting up the visual workbook

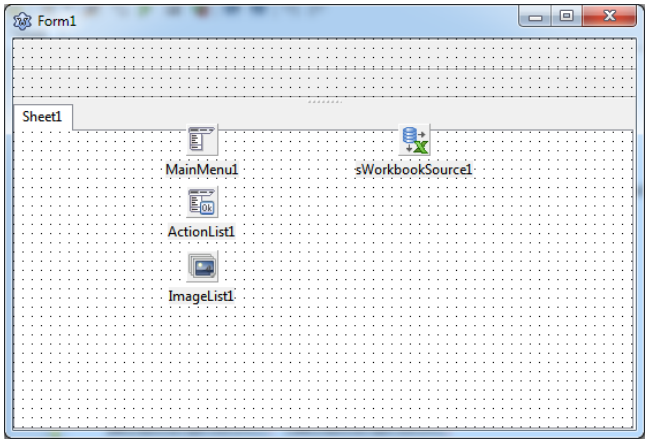
TsWorkbookSource

As described in the introductory section the **TsWorkbookSource** component is the interface between workbook and controls on the user interface. Add this component to the form and give it a decent name (we'll keep the default name `sWorkbookSource1` here, though). As you will see shortly, this component will have to be assigned to the property `WorkbookSource` of all controls of the FPSpreadsheet_visual

package.

The WorkbookSource is responsible for loading and writing data from/to file and for communicating with the workbook. Therefore, it owns a set of options that are passed to the workbook and control these processes:

```
type
  TsWorkbookOption = (boVirtual
  TsWorkbookOptions = set of Ts
```



The most important ones are

- `boAutoCalc`: activates automatic calculation of formulas whenever cell content changes.
- `boCalcBeforeSaving`: calculated formulas before a workbook is written to file
- `boReadFormulas`: if set full formulas are read from the file, otherwise only formula results.
- `boBufStream` and `boVirtualMode`: In non-visual programs, these options can help if running out of memory in case of large workbooks. `boVirtualMode`, in particular, is not usable for visual applications, though, because it avoids keeping data in the worksheet cells. See also FPSpreadsheet#Virtual_mode.

In this tutorial, it is assumed that the options `boAutoCalc` and `boReadFormulas` are activated.

TsWorkbookTabControl

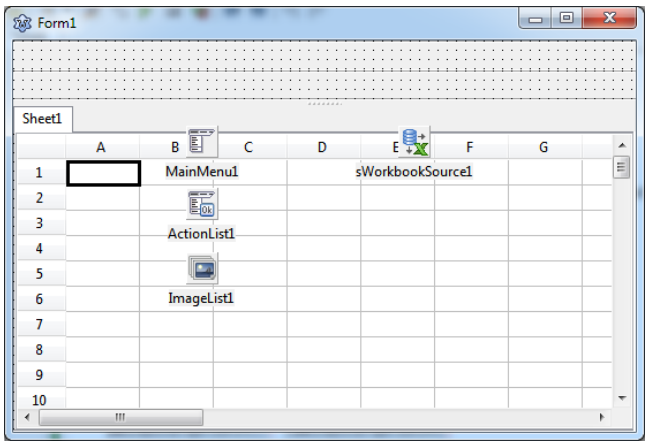
The first visual control used in the form is a **TsWorkbookTabControl** - click it onto the form (into the space not occupied by the toolbar). Client-align it within the form, this shows the TabControl as a bright rectangle only. Now link its `WorkbookSource` property to the TsWorkbookSource component that we have added just before. Now the TabControl shows a tab labelled "Sheet1". This is because the TsWorkbookSource has created a dummy workkbook containing a single worksheet "Sheet1". The WorkbookSource synchronizes this internal workbook with the TabControl (and the other visual controls to come) such that it displays this worksheet as a tab.

In Excel the worksheet tabs are at the bottom of the form - to achieve this effect you can set the property `TabPosition` of the TabControl to `tpBottom`; there are some painting issues of the LCL with this TabPosition, though, therefore, I prefer the default setting, `tpTop`.

The screenshot shows how far we've got.

TsWorksheetGrid

Now we add a **TsWorksheetGrid** control. Click it somewhere into the space occupied by the TabControl such that it becomes a child a of the TabControl. You see a standard stringgrid-like component. Link its `WorkbookSource` property to the source added at the beginning, and the grid looks more like a spreadsheet: there are the column headers labelled by letters "A", "B", etc, and the row headers labelled by numbers "1", "2", etc; the active cell, A1, is marked by a thick border.



You may want to switch the grid's `TitleStyle` to `tsNative` in order to

achieve themed painting of the row and column headers. And here is a good place to adapt the grid's `Options` in order to activate many features well-known to spreadsheets:

- `goEditing` must be active, otherwise the grid contents cannot be modified.
- `goAlwaysShowEditor` should be off because it interferes with the editing convention of spreadsheet applications.
- `goColSizing` enables changing of the column width by dragging the dividing line between adjacent column headers. Dragging occurs with the left mouse button pressed.
- `goRowSizing` does the same with the row heights.
- `goDblClickAutoResize` activates the feature that optimum column width can be set by double-clicking in the header on its dividing line to the next column. The "optimum" column width is such that no cell content is truncated and no extra space is shown in the column.
- `goHeaderHotTrack` gives visual feedback if the mouse is above a header cell.
- `goRangeSelect` (which is on by default) enables selection of a rectangular range of cells by dragging the mouse between cells at opposite corners of the rectangle. You can even select multiple rectangles by holding the CTRL key down before the next rectangle is dragged (in older Lazarus releases - before 1.4 - only a single range could be selected).
- `goThumbTracking` activates immediate scrolling of the worksheet if one of the scrollbars is dragged with the mouse. The Office applications usually scroll by lines; you can achieve this by turning off `goSmoothScroll`.

In addition to these `Options` inherited from `TCustomGrid` there are some more properties specialized for spreadsheet operation:

- `ShowGridLines`, if `false`, hides the row and column grid lines.
- `ShowHeaders` can be set to `false` if the the column and row headers are to be hidden. (The same can be achieved also by the deprecated property `DisplayFixedColRow`).
- The LCL grids normally truncate text at the cell border if it is longer than the cell width. If `TextOverflow` is set to `true` then text can overflow into adjacent empty cells.

The properties `AutoCalc` and `ReadFormulas` are meant for stand-alone usage of the WorksheetGrid (i.e. without a TsWorkbookSource). Please use the corresponding options of the WorkbookSource instead. (`AutoCalc` enables automatic calculation of formulas whenever cell content changes. `ReadFormulas` activates reading of formulas from files, otherwise the grid would display only the formula results).

Adding the reader/writer units

Unlike in earlier versions of the fpspreadsheet library the reader/writer units for the various spreadsheet file format are no longer automatically included in the "uses" clause. Therefore, the programmer must decide which file formats will be processed. Here is a list of the available reader/writer units:

- *xlsbiff2*, *xlsbiff5* and *xlsbiff8* -- the binary xls file formats for ancient Excel 2, Excel 95 and Excel 97
- *xlsOOXML* -- xlsx file format of Excel 2007 and later,
- *xlsXML* -- xml format of Excel XP and 2003 (NOTE: only for reading)
- *fpsopendocument* - file format of OpenOffice/LibreOffice's *Calc*.
- *fpsCSV* --- text files with comma-separated values (csv),
- *fpsHTML* --- HTML files,
- *wikitables* -- tables in wiki Markup (only partly supported).

For simplicity, or if you require several units, you can also simply add the unit *fpsallformats* to get read/write support for all file formats supported.

Add the required reader/writer unit(s) to the uses clause of the mainform of your application. Otherwises the program will crash with a message that the reader/writer is not found for the specific file format.

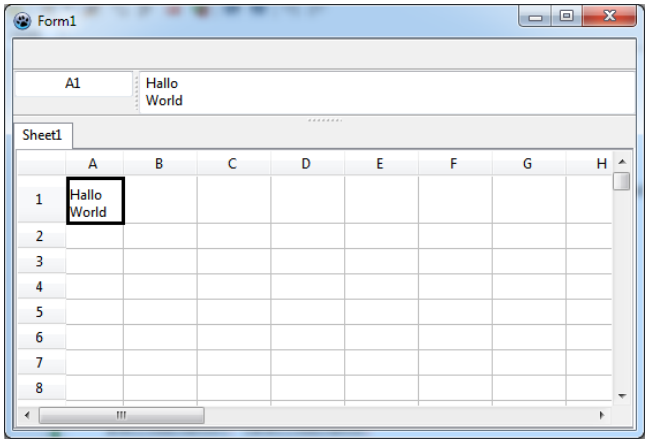
Editing of values and formulas, Navigating

When you compile and run the program you'll already be able to enter data into the grid. Just select the

cell that you want to edit by clicking or using the arrow keys - the active cell is highlighted by a thick border. Then begin typing. If the grid property `EditorLineMode` has been switched to `elmMultiLine` then manual line breaks can be entered by the key combination Ctrl + ENTER. When finished select another cell or press the ENTER key. Using ENTER automatically selects the next cell in the grid. The grid's property `AutoAdvance` defines what is understood as being the "next cell": by default, ENTER moves the active cell down (`aaDown`), but you can also move it to the right (`aaRight`), or turn this feature off (`aaNone`) - see the type `TAutoAdvance` defined in the unit `grids.pas` for even more options.

If - as assumed above - the WorkbookSource option `boAutoCalc` is enabled the worksheet automatically supports calculation of **formulas**. As an example, go to cell A1, and enter the number `10`. Then, go to cell A2 and enter the formula `=A1+10`. The formula is automatically evaluated, and its result, `20`, is displayed in cell A2.

When you navigate in the grid you may notice that cell A2 only displays the formula result, it seems that there is no way to modify the formula once it has been entered. No need to worry - press the key F2 or click into the cell a second time to enter **enhanced edit mode** in which formulas are visible in the cell.



In order to edit formulas the Office applications offer a dedicated formula editor bar. Of course, fpspreadsheet has this feature, too. It is built into the **TsCellEdit** component which is set up such as to always show the full content of a cell. You remember the second toolbar from the "Preparations" section? This will house the TsCellEdit. But wait a minute - there's more to consider: Since formulas occasionally may get rather long the control should be capable of managing serval lines. The same with multi-lined text. TsCellEdit can do this since it is inherited from TCustomMemo which is a multi-line control. You also remember that we added a splitter to the form of the second toolbar? This is for height adjustment for the case that we want to use the multi-line feature of the TsCellEdit: just drag the splitter down to show more lines, or drag it upwards to stop at the height of a single line due to the MinHeight constraints that we had assigned to the toolbar.

The TsCellEdit will cover all available space in the second toolbar. Before we add the TsCellEdit we can make life easier if we think about what else will be in the second toolbar. In Excel, there is an indicator which displays the address of the currently active cell. This is the purpose of the **TsCellIndicator**. Since its height should not change when the toolbar is dragged down we first add a **TPanel** to the second toolbar; reduce its `Width` to about 100 pixels, remove its `Caption` and set its `BevelOuter` to `bvNone`.

Add the TsCellIndicator to this panel and align it to the top of the panel. Connect its `WorkbookSource` to the TsWorkbookSource control on the form, and immediately you'll see the text "A1", the address of the currenly selected cell.

Sometimes it is desirable to change the width of this box at runtime. So, why not add a splitter to the second toolbar? Set its `Align` property to `alLeft`. The result is a bit strange: the splitter is at the very left edge of the toolbar, but you'd expect to see it at the right of the panel. This is because the panel is not aligned by default. Set the `Align` property of the panel to `alLeft` as well, and drag the splitter to the right of the panel. Now the splitter is at the correct position.

Almost done now... We finally add a TsCellEdit component to the empty space of the toolbar. Client-align it so that it fills the entire rest of the toolbar. As usual, set its `WorkbookSource` property to the instance of the TsWorkbookSource on the the form.

Compile and run. Play with the program:

- Enter some dummy data. Navigate in the worksheet. You'll see that the CellIndicator always shows the address of the active cell. The contents of the active cell is displayed in the CellEdit box. The

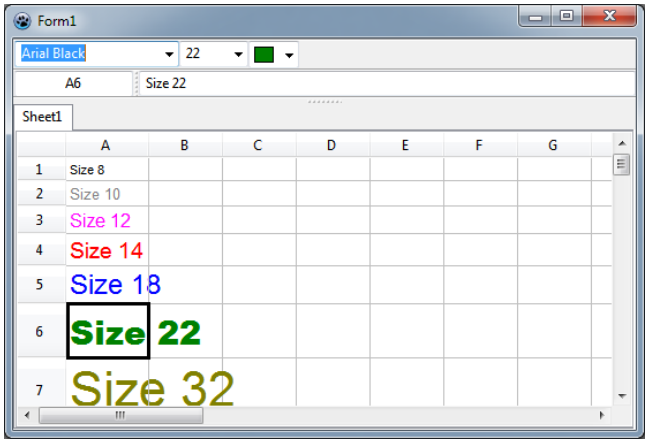
CellIndicator is not just a passive display of the current cell, it can also be edited. Type in the address of a cell which you want to become active, press ENTER, and see what happens...

- Enter a formula. Navigate back into the formula cell - the formula is displayed in the CellEdit and can be changed there readily.
- Enter multi-lined text - you can enforce a lineending in the CellEdit by holding the Ctrl key down when you press ENTER. The cell displays only one line of the text. Drag the horizontal splitter underneath the second toolbar down - the CellEdit shows all lines. Another way to see all lines of the text, is to adjust the cell height. You must have activated the grid `Option goRowSizing`. Then you can drag the lower dividing line of the row with the multi-line cell down to increase the row height - the missing lines now appear in the cell!

Formatting of cells

In addition to entering data the user usually wants to apply some formatting to the cells in order to enhance or group them. The worksheet grid is set up in such a way that its cells display the formats taken from the workbook. In addition, the visual FPSpreadsheet controls are able to store formatting attributes into the cell. Because of the notification mechanism via the WorkbookSource these formats are returned to the WorksheetGrid for display.

Adding comboboxes for font name, font size, and font color



In this section, we want to provide the possibility to modify the font of the cell texts by selecting its name, size and/or color. The visual FPSpreadsheet provide the flexible **TsCellCombobox** for this purpose. It has the property `CellFormatItem` which defines which attribute it controls:

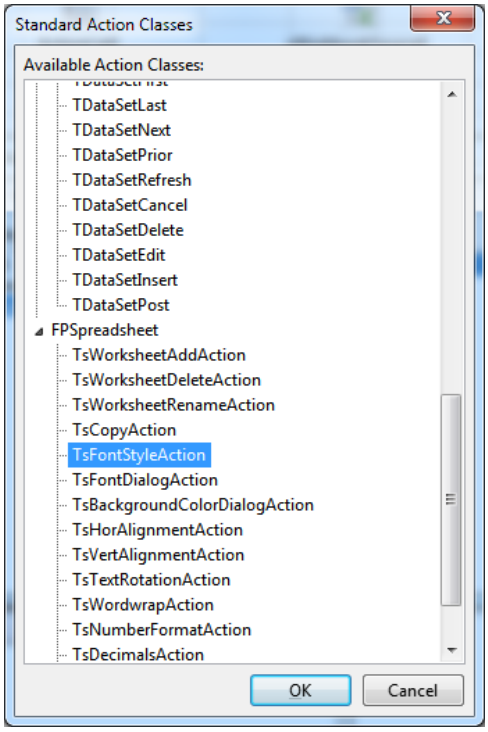
- `cfiFontName`: This option populates the combobox with all fonts found in the current system. The selected item is used for the type face in the selected cells.
- `cfiFontSize` fills the combobox with the mostly used font sizes (in points). Again, the selected item defines the font size of the selected cells.
- `cfiFontColor` adds all pre-defined colors ("palette") of the workbook to the combobox to set the text color of the selected cells. The combobox items consist of a little color box along with the color name. If the `ColorRectWidth` is set to `-1` the color name is dropped.
- `cfiBackgroundColor`, the same with the background color of the selected cells.
- `cfiCellBorderColor`, the same with the border color of the selected cells - this feature is currently not yet supported.

Add three TsCellComboboxes to the first toolbar and set their `CellFormatItem` to `cfiFontname`, `cfiFontSize`, and `cfiFontColor`, respectively. Link their `WorkbookSource` property to the TsWorkbookSource on the form. You may want to increase the width of the font name combobox such that the longest font names are not cut off; the other comboboxes may become narrower. You may also want to turn off the color names of the third combobox by setting its `ColorRectWidth` to `-1`.

That's all to modify fonts. Compile and run. Enter some text and play with these new features of the program.

Using standard actions

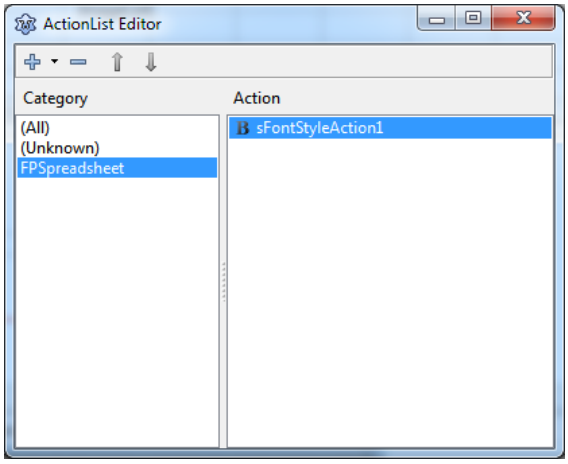
FPSpreadsheet supports a lot of formats that can be applied to cells, such as text alignment, text rotation, text font, or cell borders or background colors. Typical gui applications contain menu commands and/or toolbar buttons which are assigned to each of these properties and allow to set them by a simple mouse click. In addition, the state of these controls often reflects the properties of the active cell. For example, if there is a button for using a bold type-face this button should be drawn as being pressed if the active cell is bold, but as released if it is not. To simplify the coding of these tasks a large number of standard actions has been added to the library.



- **TsWorksheetAddAction**: adds an empty worksheet to the workbook. Specify its name in the `NameMask` property. The `NameMask` must contain the format specifier `%d` which is replaced at runtime by a number such that the worksheet name is unique.
- **TsWorksheetDeleteAction**: deletes the active worksheet from the workbook after a confirmation dialog. The last worksheet cannot be deleted.
- **TsWorksheetRenameAction**: renames the active worksheet.
- **TsCopyAction**: Copies the currently selected cells to an internal list ("CellClipboard") from where they can be pasted back into the spreadsheet to another location. The process can occur in a clipboard-manner ("copy"/"cut", then "paste") or in the way of the "copy brush" of the Office applications. The property `CopyItem` determines whether the entire cell, or only cell values, cell formulas, or cell formats are transferred.
- **TsFontStyleAction**: Modifies the font style of the selected cells. The property `FontStyle` defines whether the action makes the font bold, italic, underlined or striked-out. Normally each font style is handles by its own action. See the example below.
- **TsHorAlignmentAction**: Can be used to modify the horizontal alignment of text in the selected cells. Select `HorAlignment` to define which kind of alignment (left, center, right) is covered by the action. Like with the TsFontStyleAction, several actions should be provided to offer all available alignments. They are grouped in a mutually exclusive way like radiobuttons.
- **TsVertAlignmentAction**: Changes the vertical alignment of text in the selected cells: the kind of alignment is defined by the `VertAlignment` property. Again, these actions work like radiobuttons.
- **TsTextRotationAction**: Allows to specify the text orientation in the selected cells as defined by the property `TextRotation` in a mutially exclusive way.
- **TsWordWrapAction**: Activates the word-wrapping feature for the selected cells: if text is longer than the width of the cell (or height, if the text is rotated) then it is wrapped into multiple lines.
- **TsNumberFormatAction**: Defines the number format to be used for the selected cells. The format to be used is defined by the properties `NumberFormat` (such as `nfFixed`) for built-in formats, and `NumberFormatStr` for specialized formatting.
- **TsDecimalsAction**: Allows to increase or decrease the number of decimal places shown in the selected cells. The property `Delta` controls whether an increase (+1) or decrease (-1) is wanted.
- **TsCellBorderAction**: Allows to specify if a border will be drawn around the selected cells. The subproperties `East`, `West`, `North`, `South`, `InnerHor`, `InnerVert` of `Borders` define what the border will look like at each side of the cell range. Note that each rectangular range of cells is considere as a single block; the properties `East`, `West`, `North` and `South` are responsible for the outer borders of the entire block, inner borders are defined by `InnerHor` and `InnerVert`. Using these properties, borders can be switched on and off (`Visible`), and in addition, the line style and line color can be changed.

- **TsMergeAction**: If checked, the cells of each selected rectangular range are merged to a single block. Unchecking the action separates the block to individual cells. Note that the block's content and formatting is defined by the top-left cell of each block; content and formats of other cells will be lost.
- **TsCellProtectionAction**: comes in two flavors, one for protecting cells from modifications, and one for hiding formulas, depending on the value of the property `Protection`. Note that this action is effective only if worksheet protection has been enabled by calling `Workbooksource.Worksheet.Protect(true)`.

Adding buttons for "Bold", "Italic", and "Underline"



If you have never worked with **standard actions** before here are some detailed **step-by-step instructions**. Let us stick to above example and provide the possibility to switch the font style of the selected cells to **bold**. The standard action which is responsible for this feature is the `TsFontStyleAction`.

- At first, we add this action to the form: Double-click on the **TActionList** to open the "ActionList Editor".
- Click on the down-arrow next to the "+" button, and select the item "New standard action" from the drop-down menu.
- This opens a dialog with the list of registered "Standard Action Classes".
- Scroll down until you find a group named "FPSpreadsheet".
- In this group, select the item "TsFontStyleAction" by double-clicking.
- Now an item `sFontStyleAction1` appears in the ActionList Editor.
- It should already be selected like in the screenshot at the right. If not, select `sFontStyleAction1` in the ActionList Editor to bring it up in the Object Inspector and to set up its properties:
  - Use the text "Bold" for the `Caption` - this is the text that will be assigned to the corresponding menu item.
  - Similarly, assign "Bold font" to the `Hint` property.
  - Set the `ImageIndex` to the index of the icon in the form's ImageList that you want to see in the toolbar.
  - Make sure that the item `fssBold` is highlighted in the dropdown list of the property `FontStyle`. If not, select it. Since `TsFontStyleAction` can handle several font styles (bold, italic, underline, strikeout) we have to tell the action which font style it should be responsible of.
  - Like with the visual controls, don't forget to assign the TsWorkbookSource to the corresponding property `WorkbookSource` of the action. This activates the communication between the worksheet/workbook on the one hand, and the action and the related controls on the other hand.

Having set up the standard action we add a menu item to the form's **MainMenu**. Double-click on the TMainMenu of the form to bring up the "Menu Editor". Since the menu is empty so far there is only a dummy item, "New item1". This will become our "Format" menu. Select the item, and type "Format" into the `Caption` property field. Now the dummy item is re-labelled as "Format". Right-click on this "Format" item, and select "Create submenu" from the popup menu which brings up another new menu item, "New item2". Select it. In the dropdown list of the property `Action` of the object inspector, pick the `sFontStyle1` action - this is the action that we have just set up - and the menu item automatically shows the caption provided by the action component, "Bold".

Finally we add a **toolbar button** for the "bold" action. Right-click onto the TToolbar, and add a new toolbutton by selecting item "New button" from the popup menu. Go to the property `Action` in the

object inspector again, pick the `sFontStyle1` item, and this is enough to give the tool button the ability to set a cell font to bold!

Repeat this procedure with two other buttons. Design them to set the font style to **italic** and **underlined**.

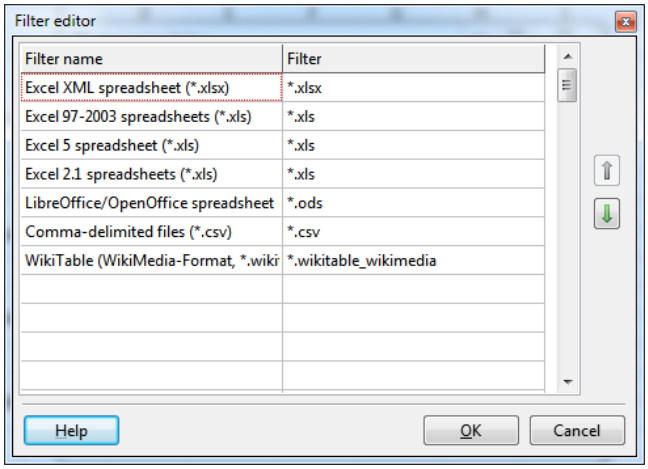
Test the program by compiling. Type some text into cells. Select one of them and click the "Bold" toolbutton - voila, the cell is in bold font. Select another cell. Note that the toolbutton is automatically drawn in the down state if the cell has bold font. Repeat with the other buttons.

Saving to file

After having entered data into the grid you will certainly want to **save the grid to a spreadsheet file**. Lazarus provides all the necessary infrastructure for saving available in the standard action `TFileSaveAs`. This action automatically opens a **FileDialog** for entering the file name.

Select the `TFileSaveAs` standard action from the list of standard action classes. Note that it cannot be found in the "FPSpreadsheet" category, but in the "File" group since it is a standard action of the LCL.

At first, let us specify the properties of the FileDialog. Select the property `Dialog` of the `TFileSaveAs` action in the object inspector. It is convenient to be able to store the workbook in various file formats; this can be prepared by providing a file format list in the `Filter` property of the dialog. Paste the following text into this property:




```
Excel XML spreadsheet (*.xlsx)|*.xlsx|Excel 97-2003 spreadsheets (*.xls)|*.xls|Excel 5 spreadsheet (*.xls)|*.xls|Excel 2.1 spreadsheets (*.xls)|*.xls|LibreOffice/OpenOffice spreadsheet (*.ods)|*.ods|Comma-delimited files (*.csv)|*.csv|WikiTable (WikiMedia-Format, *.wikitable_wikimedia)|*.wikitable_wikimedia
```

When you click on the ellipsis button next to `Filter` the file list appears in a more clearly arranged dialog shown at the right.

Make one of these file extensions, e.g. xlsx, the default of the file dialog by assigning its list index to the `FilterIndex` property. The xlsx file is the first format in the filter list. `FilterIndex`, therefore, must be set to 1.

> **Note:** The indexes in the filter list are 1-based, in contrast to the convention of Lazarus and FPC using 0-based indexes.

Next, we define what happens after a file name has been selected in the file dialog. For this purpose, the `TFileSaveAs` action provides the event `OnAccept`. This is one of the few places where we have to write code in this project... But it is short: We check which file format has been selected in the format list and write the corresponding spreadsheet file by calling the method `SaveToSpreadsheetFile` of the TWorkbookSource:

```
uses
  ..., fpstypes, ...;   // for TsSpreadsheetFormat

procedure TForm1.FileSaveAs1Accept(Sender: TObject);
var
  fmt: TsSpreadsheetFormat;
begin
  Screen.Cursor := crHourglass;
  try
    case FileSaveAs1.Dialog.FilterIndex of
      1: fmt := sfOOXML;                // Note: Indexes are 1-based here
      2: fmt := sfExcel8;
      3: fmt := sfExcel5;
      4: fmt := sfExcel2;
      5: fmt := sfOpenDocument;
      6: fmt := sfCSV;
      7: fmt := sfWikiTable_WikiMedia;
    end;
    sWorkbookSource1.SaveToSpreadsheetFile(FileSaveAs1.Dialog.FileName, f
  finally
    Screen.Cursor := crDefault;
  end;
end;
```

We will make the FileSaveAs action available in the toolbar and in the menu:

- **Toolbar**: Add a TToolButton to the first toolbar and drag it to its left edge. Assign the FileSaveAs action to its `Action` property.
- **Menu**: The "Save" command is usually in a submenu called "File". Therefore, double click on the TMainMenu, right-click on the "Format" item and insert a new item "before" the current one. Name it "File". Add a submenu to it. Click at the default menu item and assign the FileSaveAs action to its `Action` property.

Reading from file

What is left is **reading of a spreadsheet file** into our application. Of course, FPSpreadsheet is well-prepared for this task. The operations are very similar to saving. But instead of using a TFileSaveAs standard action, we use a **TFileOpen** standard action. Again, this standard action has a built-in file dialog where we have to set the `DefaultExtension` (".xls" or ".xlsx", most probably) and the format `Filter`:

```
All spreadsheet files|*.xls;*.xlsx;*.ods;*.csv|All Excel files
(*.xls, *.xlsx)|*.xls;*.xlsx|Excel XML spreadsheet
(*.xlsx)|*.xlsx|Excel 97-2003 spreadsheets (*.xls)|*.xls|Excel 5
spreadsheet (*.xls)|*.xls|Excel 2.1 spreadsheets
(*.xls)|*.xls|LibreOffice/OpenOffice spreadsheet (*.ods)|*.ods|Comma-
delimited files (*.csv)|*.csv
```

(Copy this string into the field `Filter` of the action's `Dialog`). As you may notice the `Filter` contains selections which cover various file formats, such as "All spreadsheet files", or "All Excel files". This is possible because the TsWorkbookSource has a property `AutoDetectFormat` for automatic detection of the spreadsheet file format. In the other cases, like "Libre/OpenOffice", we can specify the format, `sfOpenDocument`, explicitly. Evaluation of the correct file format and reading of the file is done in the `OnAccept` event handler of the action:

```
{ Loads the spreadsheet file selected by the FileOpen standard action }
procedure TForm1.FileOpen1Accept(Sender: TObject);
begin
  sWorkbookSource1.AutodetectFormat := false;
  case FileOpen1.Dialog.FilterIndex of
    1: sWorkbookSource1.AutoDetectFormat := true;          // All spreads
    2: sWorkbookSource1.AutoDetectFormat := true;          // All Excel f
    3: sWorkbookSource1.FileFormat := sfOOXML;             // Excel 2007+
    4: sWorkbookSource1.FileFormat := sfExcel8;            // Excel 97-20
    5: sWorkbookSource1.FileFormat := sfExcel5;            // Excel 5.0
    6: sWorkbookSource1.FileFormat := sfExcel2;            // Excel 2.1
    7: sWorkbookSource1.FileFormat := sfOpenDocument;      // Open/LibreO
    8: sWorkbookSource1.FileFormat := sfCSV;               // Text files
  end;
  sWorkbookSource1.FileName :=FileOpen1.Dialog.FileName;   // This loads
end;
```

In order to see this action in the toolbar and menu, add a TToolButton to the **toolbar** and assign the TFileOpenAction to its `Action` property. In the **menu**, add a new item before the "Save as" item, and assign its `Action` accordingly.

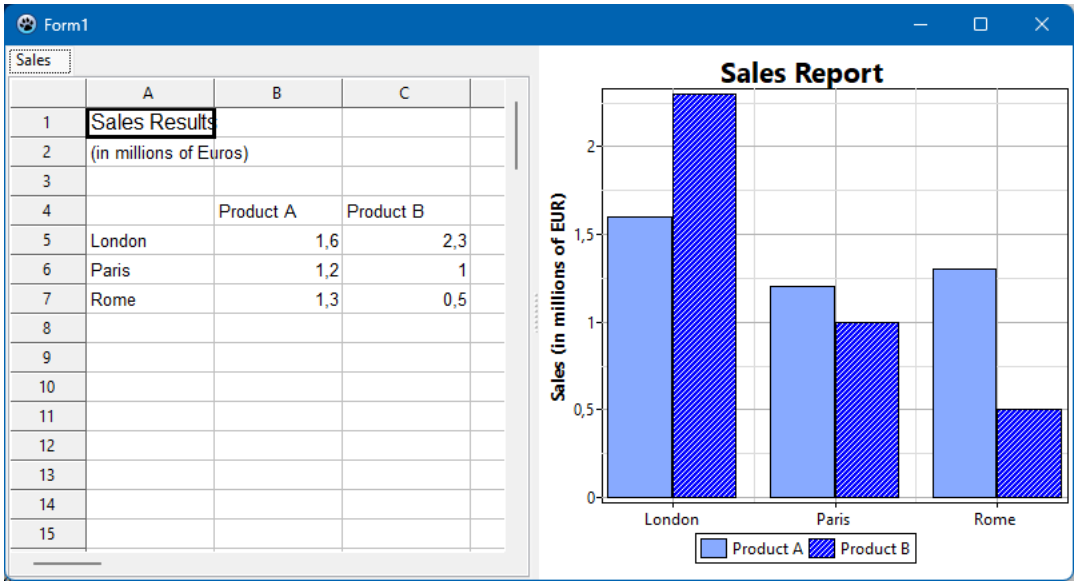
Note: You can see a spreadsheet file even at designtime if you assign its name to the `Filename` property of the TsWorkbookSource. But be aware that the file probably cannot be found at runtime if it is specified by a relative path and if the application is to run on another computer with a different directory structure!

Summary

If you followed us through the steps of this tutorial you have programmed a complex spreadsheet gui application almost without having written any line of code (with the exception of the loading and saving routines). If you did not, have a look at the demo "fps_ctrls" in the *examples* folder of the FPSpreadsheet installation; it shows the result of this tutorial with some add-ons.

Online version

FPSpreadsheet: Chart Tutorial



	A	B	C
1	Sales Results		
2	(in millions of Euros)		
3			
4		Product A	Product B
5	London	1,6	2,3
6	Paris	1,2	1
7	Rome	1,3	0,5
8			
9			
10			
11			
12			
13			
14			
15			

One of the additions to FPSpreadsheet version 1.2 is the possibility to create, read and write charts. In this tutorial we will give a short introduction how to create a chart from spreadsheet data and how to display it visually by means of the TAChart library.

Preparing data

In this chart we will show sales results of two products of an imaginary company in three European cities. At first we must create a workbook, a worksheet and add the data to be plotted:

- Create a new Lazarus LCL project.
- Add the package "laz_fpspreadsheet" to the project requirements in order to get access to the FPSpreadsheet library.
- Add `fpspreadsheet, fpstypes, fpsutils` to the `uses` clause of the project's main form - the standard for all FPSpreadsheet projects. Since we will create xlsx and ods files we also need the `xlsxooxml` and `fpsopendocument` units in the `uses` clause.
- Add a button and call a method `CreateChart` from its `OnClick` event handler. `CreateChart` our main work place during the rest of this tutorial.

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  CreateChart;
end;

procedure TForm1.CreateChart;
begin
  // to be completed...
end;
```

- In `CreateChart` we first create a `TsWorkbook` (named `wbook`) and add a `TsWorksheet` (named `wsheet`) to it. To complete the code skeleton we also add code to save the workbook to xlsx and ods files, and to destroy the workbopok at the end:

```
procedure TForm1.CreateChart;
var
  wbook: TsWorkbook;
  wsheet: TsWorksheet;
begin
  wbook := TsWorkbook.Create;
  try
    wsheet := wbook.AddWorksheet('Sales');  // 'Sales' is the caption of the worksh

    // more code to be inserted here...

    wbook.SaveToFile('chartdemo.xlsx', true);
    wbook.SaveToFile('chartdemo.ods', true);
  finally
    wbook.Free;
  end;
end;
```

- Now let's add data. All the following code will go at the place of the "`more code to be inserted here...`" comment in above snippet.
- At first, we write some kind of title in cells A1 and A2

```
    wsheet.WriteText(0, 0, 'Sales Results');              // Cell A1
    wsheet.WriteFontSize(0, 0, 12);                       // Increase the text size
    wsheet.WriteText(1, 0, '(in millions of Euros)');     // Cell A2
```

- Next, starting at row index 3 there will be three colums, the first one with the city names, the second one with the sales data for "product A", and the last one with the sales data for "product B". At first we write the product names to cells B4 and C3:

```
    wsheet.WriteText(3, 1, 'Product A');     // Cell B4
    wsheet.WriteText(3, 2, 'Product B');     // Cell C4
```

- Now we write the city names and the sales numbers:

```
    wsheet.WriteText(4, 0, 'London');  wsheet.WriteNumber(4, 1, 1.6);     wsheet.Writ
    wsheet.WriteText(5, 0, 'Paris');   wsheet.WriteNumber(5, 1, 1.2);     wsheet.Writ
    wsheet.WriteText(6, 0, 'Rome');    wsheet.WriteNumber(6, 1, 1.3);     wsheet.Writ
```

- This completes the data generation. You can run the project now, it will create files "chartdemo.xlsx" and "chartdemo.ods" which you can open in Excel or LibreOffice Calc to verify the entered data.

	A	B	C
1	Sales Results		
2	(in millions of Euros)		
3			
4		Product A	Product B
5	London	1.6	2.3
6	Paris	1.2	1
7	Rome	1.3	0.5

Preparing the Chart

All the charting types and classes and routines are contained in unit *fpschart*. Study at least the interface part of it because here we can address only a small part of it.

`TsChart` is the main class for the FPSpreadsheet charts. Do not confuse it with the chart of the TAChart library - like all (well, most...) classes in FPSpreadsheet its class name has an "s" after the "T". It has no visual representation and only collects all information related to the chart. All charts of a workbook are collected in an internal workbook list.

Creating the Chart

A chart is created similarly to the a worksheet: call the corresponding "Add..." method of the workbook, here `AddChart`; the same can be done from the worksheet. There are parameters to specify size and position of the chart within the worksheet:

```
function TsWorksheet.AddChart(AWidth, AHeight: Double; ARow, ACol: Cardinal; AOffsetX: D
function TsWorkbook.AddChart(ASheet: TsBasicWorksheet; AWidth, AHeight: Double; ARow, AC
```

`AWidth` and `AHeight` denote the dimensions of the chart, in millimeters, but note that these values are not very exact. `ARow` and `ACol` refer to the row/column indices of the cell which contains the top/left corner of the chart. And the optional `AOffsetX` and `AOffetY` specify a distance, in millimeters, by which the top/left chart corner is moved away from the top/left corner of this anchor cell.

Let's add the chart to our workbook:

```
  wchart := wbook.AddChart(wsheet, 150, 90, 0, 3, 10);
// or: wchart := wsheet.AddChart(150, 90, 0, 3, 10);
```

This means:

- The chart is in our worksheet (`wsheet`, of course...)
- The size of the chart is 150 x 90 mm.
- The top/left corner is in cell D1 and is moved to the right by 10 mm. There is no vertical offset from the cell corner (default parameter, omitted).

Adding series

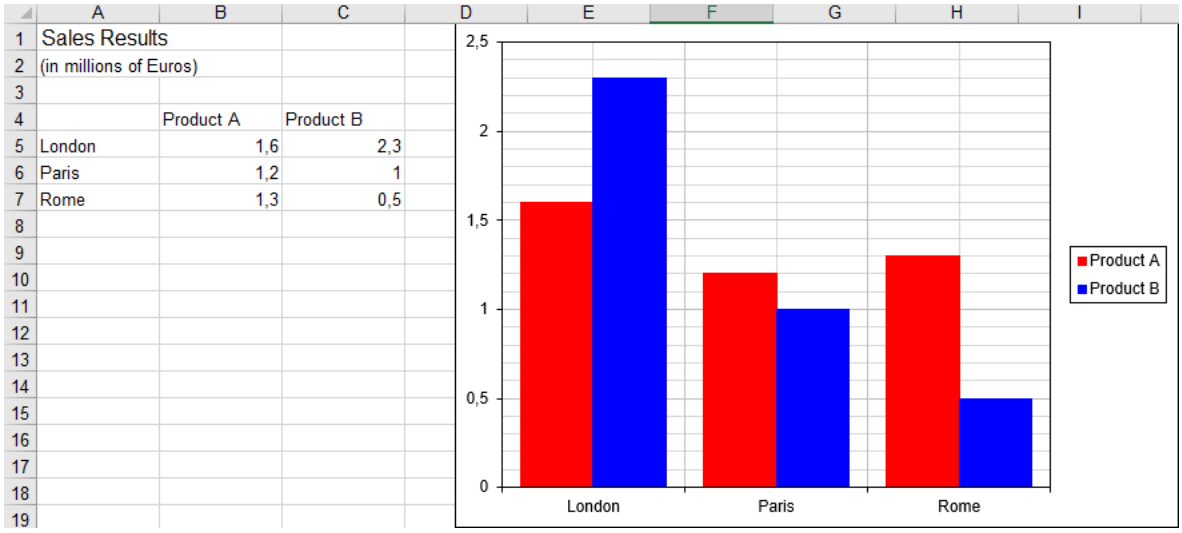
In the next step we can begin adding series to the chart. FPSpreadsheet supports a great number of series types:

- `TsBarSeries`: Draws the data as vertical or horizontal bars.
- `TsLineSeries`: Connects the data points by straight or smooth lines.
- `TsAreaSeries`: Similar to `TsLineSeries`, but the area between the series and the x axis is filled by a color, a pattern or a gradient..
- `TsScatterSeries`: Similar to `TsLineSeries`, but the x values can be placed arbitrarily (in the other series they are equidistant).
- `TsBubbleSeries`: Similar to `TsBubbleSeries`, data points are displayed as circles with varying size.
- `TsPieSeries`: Draws data as sectors of a circular shape
- `TsRadarSeries`: Spider-like, x values are interpreted as angle.
- `TsFilledRadarSeries`: like `TsRadarSeries`, but filled by a color
- `TsStockSeries`: Financial chart series type, displaying high/low/close values of shares.

Let's pick a `TsBarSeries` here. A series is created like any other object by calling its constructor `Create`. The constructor gets the chart as argument, and this automatically inserts the series into the chart. The values to be displayed on the y axis are taken from the cell range determined by the series' `SetYRange` method. As already noted, a bar chart uses equidistant x values, we must specify however which labels will be displayed at the x axis; this can be done by calling the series' `SetLabelRange` method. In case of a scatter or bubble series, however, we have to call `SetXRange` to define the x values of the data points. In any case, cell ranges are given by the row/column coordinates of the top/left and bottom/right corners. Note that ranges to be used for series can only be one column wide or one row high. Finally we should also specify the series title for the legend by calling the `SetTitleAddr` method; otherwise a generic title will be used by the Office applications.

Both series have their labels in column A, from cell A5 to A7. The first series has its y values in the range B5:B7, and the second series in range C5:C7. The axis title is in cell B4 for the first and in C4 for the second series:

```
var
  ser1, ser2: TsBarSeries
...
  // 1st bar series
  ser1 := TsBarSeries.Create(wChart);
  ser1.SetLabelRange(4, 0, 6, 0);    // A5:A7
  ser1.SetYRange(4, 1, 6, 1);        // B5:B7
  ser1.SetTitleAddr(3, 1);           // B4

  // 2nd bar series
  ser2 := TsBarSeries.Create(wChart);
  ser2.SetLabelRange(4, 0, 6, 0);    // A5:A7
  ser2.SetYRange(4, 2, 6, 2);        // C5:C7
  ser2.SetTitleAddr(3, 2);           // C4
```



Styling lines in the chart

By default, the chart is surrounded by a rectangular frame, but sometimes this is not wanted. So, let's remove it. And we also might want to give bars of the series a black outline border.

These items, like any other lines in the FPSpreadsheet charts, are implemented as instances of `TsChartLine`:

```
type
  TsChartLine = class
  ...
  public
    Color: TsChartColor;     // in hex: $00bbggrr, r=red, g=green, b=blue; contains Trans
    Width: Single;           // mm
    PatternIndex: Integer;   // Index into global RawLinePatternList
    property Style: TsChartLinePatternStyle read GetStyle write SetStyle;
  end;
```

- `Color` determines the color of the line. It is an extension of the standard FPSpreadsheet color type, `TsColor`, in order to support transparency which essentially is a floating point value between 0 (opaque) and 1 (fully transparent = inivisible). To simplify usage, a function `ChartColor()` is available which accepts the `TsColor` value, as well as, optionally, the `Transparency` single:

```
type
  TsChartColor = record
    Transparency: TsChartTransparency;   // = single in the range 0.0 (opaque) ... 1
    case Integer of
      0: (Red, Green, Blue, SystemColorIndex: Byte);
      1: (Color: TsColor);
  end;

function ChartColor(AColor: TsColor; ATransparency: Single = 0.0): TsChartColor;
begin
  Result.Color := AColor;
  Result.Transparency := ATransparency;
end;
```

- `Width` denotes the width of the line, given in millimeters.
- `Style` and `PatternIndex` identify the line pattern to be used for drawing. Both essentially are indices into a global pattern array implemented in unit *fpsPatterns*. The first patterns are pre-defined and accessibile via the `Style` enumeration `TsChartLinePatternStyle = (clsSolid, clsNoLine, clsFineDot, clsDot, clsDash, clsDashDot, clsLongDash, clsLongDashDot, clsLongDashDotDot)`. Additionally, users can register their own special patterns by calling `RegisterRawLinePattern()` where they have to specify the length and count of stroke elements, as well as their distance, given in either millimeters (`AUnit = cluMillimeters`) or as precentage of the line width (`AUnit = cluPercentage`):

```
function RegisterRawLinePattern(AName: String; AElementLength: Single; ADistanceLeng
function RegisterRawLinePattern(AName: String; AElement1Length: Single; AElement1Cou
```

OK. Now we have everything to remove the chart's border and to draw the series bar borders in black:

```
  wChart.Border.Style := clsNoLine;
  ser1.Line.Color := ChartColor(scBlack);  // This automatically selects the solid "patt
  ser2.Line.Color := ChartColor(scBlack);
```

Filling chart elements

In the next chapter we want to show how areas can be filled by color or patterns. Class `TsChartFill` gives access to all related properties: fill style, fill color, fill pattern, pattern parameters, gradient parameters etc.

```
type
  TsChartFill = class
  public
    Style: TsChartFillStyle;
    Color: TsChartColor;
    Gradient: Integer;      // Index into chart's Gradients list
    Pattern: Integer;       // Index into chart's FillPattern list
    Image: Integer;         // Index into chart's Images list
    [...]
  end;
```

- `Style` of type `TsChartFillStyle = (cfsNoFill, cfsSolid, cfsGradient, cfsHatched, cfsSolidHatched, cfsImage)` classifies to the type of the fill: no fill at all, solid fill by uniform color, fill by a gradient, fill by a pattern or by an image.
- `Color` specifies the color of the solid fill-
- `Gradient`, `Pattern` and `Image` are indices into chart-internal lists containing the corresponding items - see below.

Changing the series fill color

At first we change the color of the 1st series bars to some brighter blue, and its border to black. The related `TsChartFill` property is named `Fill` here:

```
  ser1.Fill.Color := ChartColor($FFAA88);   // bright blue fill of the bars
  ser1.Fill.Style := cfsSolidFill;          // not absolutely necessary, just for better
```

Changing the series fill pattern

The 2nd bar series is supposed to be hatched by a pattern of diagonal-up strokes. The library provides a lot of pre-defined fill patterns which can be identified by the `TsChartFillPatternStyle` enumeration (declared in unit *fpsTypes*, and implemented in unit *fpsPatterns*):

```
type
  TsChartFillPatternStyle = (
    fpsGray05, fpsGray10, fpsGray20, fpsGray25, fpsGray30, fpsGray40,
    fpsGray50, fpsGray60, fpsGray70, fpsGray75, fpsGray80, fpsGray90,
    fpsHorThick, fpsVertThick, fpsDiagUpThick, fpsDiagDownThick, fpsHatchThick, fpsCross
    fpsHorThin, fpsVertThin, fpsDiagUpThin, fpsDiagDownThin, fpsHatchThin, fpsCrossThin,
    fpsHorNarrow, fpsVertNarrow, fpsDiagUpNarrow, fpsDiagDownNarrow, fpsHatchNarrow, fps
    fpsHorDash, fpsVertDash, fpsDiagUpDash, fpsDiagDownDash, fpsHatchDot, fpsCrossDot,
    fpsBrickDiag, fpsBrickHor, fpsCheckerBoardLarge, fpsCheckerBoardSmall, fpsConfettiLa
    fpsDiamond, fpsDivot, fpsPlaid, fpsShingle, fpsSphere, fpsTrellis, fpsWave, fpsWeave
  );
```

There are several "diagonal-up" patterns, a nice one is `fpsDiagUpNarrow`. These patterns do not carry any color, they are just black and white. In order to fill an area, we must add the combination of fill pattern style, foreground and background color to the chart's `FillPatterns` list (`Chart.FillPatterns.AddPattern(name, style, forgroundColor, backgroundColor)`. This process returns the index of this colored pattern in `FillPatterns` which is needed as the `Pattern` index of the `TsChartFill` class. If this index is already known from a previous fill, of course, you can reuse it instead of calling `AddPattern`.

```
  ser2.Fill.Pattern := wchart.FillPatterns.AddPattern('white-on-blue diag-up', fpsDiagUp
    // Drop the last argument if you need a pattern without background
  ser2.Fill.Style := cfsPattern;                // Selector for the pattern fill
  ser2.Line.Color := ChartColor(scBlack);
```

Alternatively you can also fill the area by any bitmap loaded from file: Add the image file to the workbook's embedded object list (`workbook.AddEmbeddedObj(filename)`) and then use the returned index in the method `AddImage` of the chart's `Images` list:

```
  ser2.Fill.Image := wchart.Images.AddImage('FillImg', wbook.AddEmbeddedObj('bgimage.jpg
  ser2.Fill.Style := cfsImage;                  // Selector for the image fill
  ser2.Line.Color := ChartColor(scBlack);
```

Drawing a gradient background

Spreadsheet charts provide various gradient types:

- axial
- elliptic
- linear
- radial
- rectangular
- square

Note that it is not guaranteed that every gradient type is supported by both XLSX and ODS files.

Let's fill the chart background by a linear gradient. The related property is accessible as - well - `Background.Fill`. So, we must set the chart's `Background.Fill.Style` to `cfsGradient`, add the specific gradient with its parameters to the chart's `Gradients` list and put the returned index into the `Background.Gradient` property:

```
  wchart.Background.Style := cfsGradient;     // Selector for the gradient fill
  wChart.Background.Gradient := wchart.Gradients.AddLinearGradient('bkGr', ChartColor(sc
```

- The first argument of the `AddLinearGradient` method is the name of the gradient needed for identification.
- The second argument is the color at which the gradient starts, the third argument the color at which it ends.
- The last argument is the drawing direction of the gradient: normally the gradient runs horizontally, i.e. the startcolor is at the left, and the endcolor is at the right. The angle is measured in degress and oriented in counter-clockwise direction. The value "90" means that "white" is at the bottom and "light blue" is at the top of the chart.
- The function returns the index of the gradient in the chart's `Gradients` list which must be assigned to the

`Fill.Gradient` property.

If you know that this specific gradient already exists you can find its index by calling `Gradients.IndexOf('bkGr')` and assign it to the `Fill.Gradient` property of another chart element for re-using the gradient.

Showing the chart legend

In order to display the chart legend set the `Visible` property of the `Legend` to `true` (this, however, is the default setting already). You can reposition the legend by applying one of the `TsChartLegendPosition = (legRight, legTop, legBottom, legLeft)` values to `Legend.Position`:

```
wChart.Legend.Visible := true;
wChart.Legend.Position := legBottom;
```

Adding titles

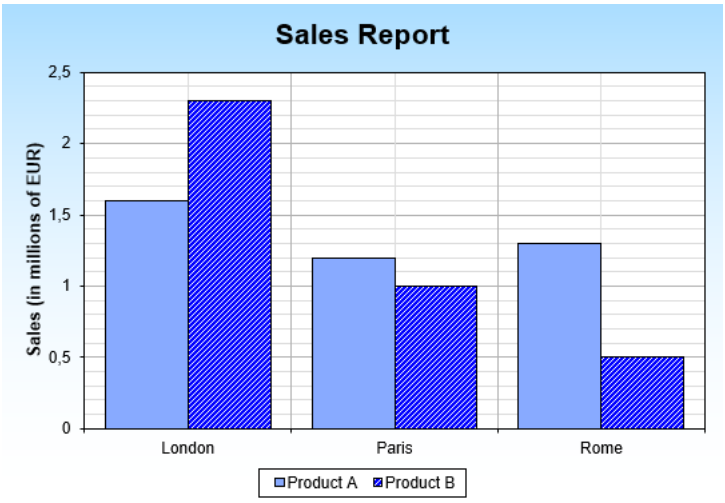
The `Title.Caption` property of the chart defines the text to be shown above the chart as a title. You can change its `Font` property to emphasize it:

```
wChart.Title.Caption := 'Sales Report';
wChart.Title.Font.Size := 16;
wChart.Title.Font.Style := [fssBold];
```

Likewise, there is an axis `Title` which can be drawn along the axis line:

```
wChart.yAxis.Title.Caption := 'Sales (in millions of EUR)';
```

In total, the chart will look in Excel as follows:



Displaying the chart in a Lazarus form

So far, a spreadsheet chart cannot be displayed embedded in a worksheet grid, like in the Office applications. But it is possible to use a standard `TAChart` component to render it. Cooperation between TAChart and workbook/worksheet is done by the `TsWorkbookSource` and `TsWorkbookChartLink` components: The `TsWorkbookSource` gives access to the workbook, its worksheets and the contained data, and the `TsWorkbookChartlink` passes the chart-related data and properties on to a `TAChart` instance and its subcomponents.

- Drop a `TsWorkbookSource` component on the form, set its `FileName` property to point to the file containing the chart. Make sure that the spreadsheet reader unit for the file format is listed in the *uses* clause, e.g. *xlsxooxml* for an xlsx file.
- Drop a `TAChart` component on the form. Position and size it as needed.
- Drop a `TsWorkbookChartLink` on the form. Set its `WorkbookSource` and `Chart` properties to the components that you just had added. When the workbook contains several charts, enter its index in `WorkbookChartIndex` - charts are numbered internally from the first chart per worksheet to the last and from the first worksheet to the last worksheet.
- Voila - here you see the chart!
- For completeness, you might also want to drop a `TsWorkbookTabControl` and - into it - a `TsWorksheetGrid`. This way you can easily switch between the indidvidual worksheets and see their contents in a grid. See

FPSpreadsheet_tutorial:_Writing_a_mini_spreadsheet_application of further instructions how to set up a visual spreadsheet application.

The overall result can be seen in the screenshot at the top of this wiki page.

We should note that the chart displayed this way is interactive: If you change cell content which is needed by the chart, the change will be updated immediately in the chart. And you may notice that there are some differences to the Office application charts, here and there. Gradients, for example, are not supported, and some specific line pattern are not either. But the same may happen also within *Excel* (or *Calc*) when an ods (or xlsx) file is imported. So, FSpreadsheet is in good company here...

Appendix

Source code

The source code developed during this tutorial can be found in folder *examples/visual/fpschart/fpschart_tutoral* of the FPSpreadsheet installation.

Wiki links

- FPSpreadsheet
- FPSpreadsheet tutorial: Writing a mini spreadsheet application
- TAChart documentation

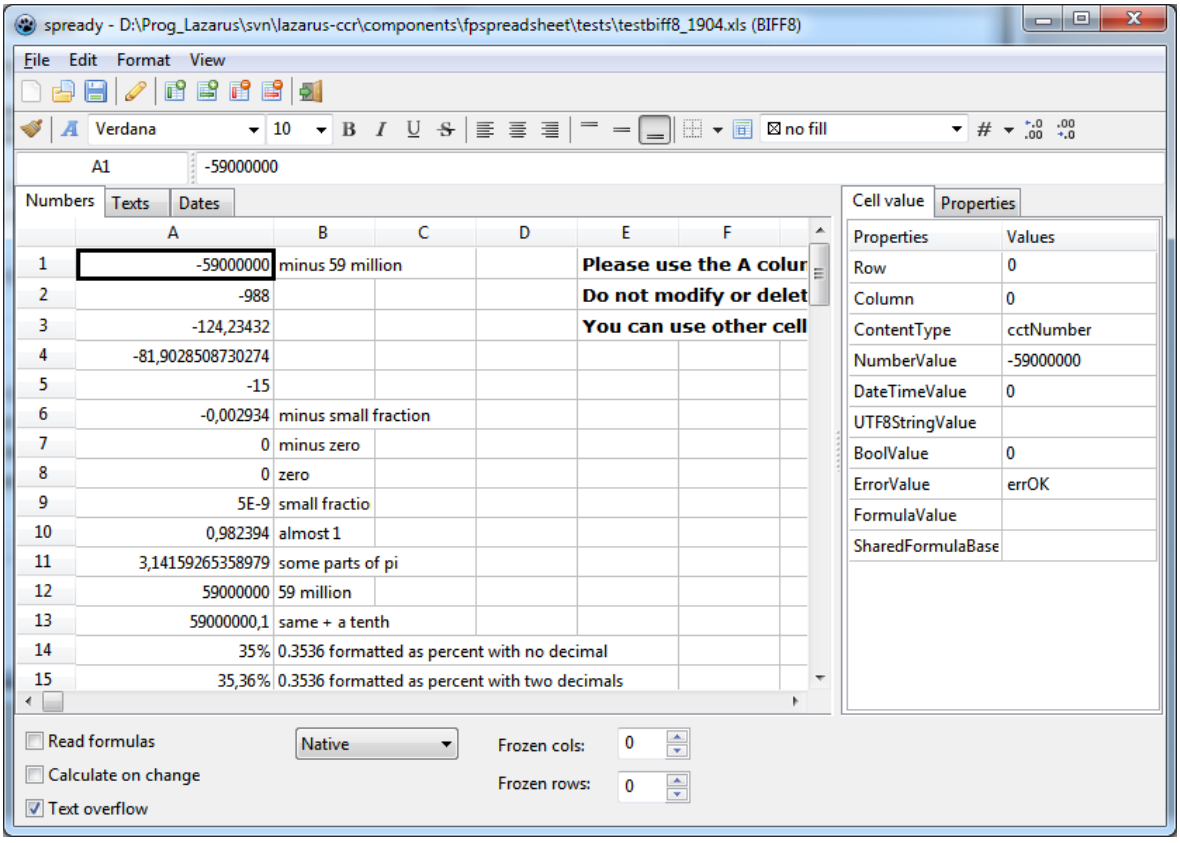
Categories: Components | Data import and export | Grids | Lazarus-CCR | FPSpreadsheet | TAChart

Online version

TsWorksheetGrid

Overview

`TsWorksheetGrid` is a specialized grid component which interfaces to a `TsWorksheet` from the FPSpreadsheet library and displays spreadsheet data files like in a conventional `TStringGrid`.



Programming interface

`TsWorksheetGrid` inherits from TCustomGrid and behaves much like a standard `TStringGrid`. The main difference is that the grid itself does not store data, but data are stored in a `TWorksheet` of FPSpreadsheet. The worksheet can be accessed by using the property `Worksheet` of the grid. Similarly, `Workbook` allows to access the entire workbook to

which the worksheet belongs.

A variety of properties are available to access cells, their values and attributes, in a StringGrid-like way, such as writing a cell value by means of `Grid.Cells[ACol, ARow] := 1.234`. The cells are identified in these properties by means of the cell's column and row indexes. Please note that the indexes are passed in the order "column first / row last", and they include the indexes occupied by the fixed cells, i.e. the top/left data cell has the indexes col=1 and row=1. This is different from fpspreadsheat where the indexes always start at 0 and are passed to functions in the opposite order "row first / column last".

Access to cell values

The property `Cells[ACol, ARow]` provides access to the data in a cell given by its column and row indexes. This is similar to `TStringGrid`. Unlike `TStringGrid` which works with strings only, `TsWorksheetGrid`, however, accepts numbers, dates, booleans, and formulas as well. For this reason, the data type of `Cells` is a `variant`.

Writing of cell values

Use this code to write data to a cell in a `TsWorksheetGrid` named `Grid` for simplicity:

```
// Write a string
  Grid.Cells[1, 1] := 'This is a string';
  // Write a string with embedded HTML codes
  Grid.Cells[1, 2] := 'Chemical formula of water: H2O';
  // Write a number
  Grid.Cells[1, 3] := 1.2345;
  // Write a date
  Grid.Cells[1, 4] := EncodeDate(2016, 1, 18);
  // Write a formula
  Grid.Cells[1, 5] := '=A3+2';
  // etc.
```

Note: A text assigned to a cell can contain embedded simple HTML codes to change the font for particular words or characters.

Reading of cell values

In the same way, cell values can be read into variables of decidated types:

```
var
  str: String;
  x: Double;
begin
  str := Grid.Cells[1, 1];  // Variable "str" will contain the value "This is a str
  x := Grid.Cells[1, 2];    // x will be 1.2345
  str := Grid.Cells[1, 2];  // Although the cell contains a number it is converted
end;
```

Comparing cell values

Since the Cells property is a variant it is a bit more difficult to compare string values. Use the string-to-variant conversion function `VarToStr()` provided by unit `variants`.

```
// This straightforward comparison will fail:
  // if Grid.Cells[1,1] = 'This is a string' then ...

  // Use this instead:
  if VarToStr(Grid.Cells[1,1]) = 'This is a string' then ...
```

Numerical values often can be compared without an explicit conversion:

```
if Grid.Cells[1,2] = 1.2345 then ...
```

Formulas in cells

Since the `TsWorksheetGreed` works on top of a spreadsheet all formulas supported by FPSpreadsheet can be entered. See this document for a complete list of supported formulas.

The formula string must begin with the character "=". Cell references must follow Excel's "A1" notation, i.e. column index must be characters "A".."Z" , and row index is a 1-based integer. If there are more than 26 columns then two (or three) characters can be used.

In case of a cell range, the coordinates of the top-left and bottom-right corners of the cell rectangle must be separated by a colon, e.g. '=SUM(A1:C10)' calculates the sum of the values in the cell rectangle extending from A1 to C10.

Formulas referring to cells in other sheets can be used by following the Excel syntax: specify the sheet name followed by a "!" in front of the cell addresses, e.g. '=Sheet3!A1' or '=SUM(Sheet3:Sheet4!A1:C10)'.

Note that formulas are not automatically calculated by default. In order to activate automatic calculation of formulas set the grid property `AutoCalc` to true, or set the option `boAutoCalc` of the workbook used by the grid. "Automatic calculation" means that all formulas in the entire worksheet are recalculated whenever the content of any cell changes. Therefore, it is not required that the cells addressed by the formula already have values when the formula is entered.

```
// Enable automatic calculation of formulas
Grid.AutoCalc := true;
// or: Grid.Workbook.Options := Grid.Workbook.Options + [boAutoCalc];

// Enter cells
Grid.Cells[1,1] := 1;          // this is cell A1
Grid.Cells[1,2] := 2;          // this is cell A2

// Enter formula
Grid.Cells[1,3] := '=A1+A2';  // Calculates the sum of the values in A1 and A2

// Another formula in Cell B3
Grid.Cells[2,3] := '=B1*B2';  // It does not matter that the cells B1 and B2 do not

// Enter cells needed by the formula
Grid.Cells[2,1] := '=A3';      // Use the result of the previous formula
Grid.Cells[2,2] := 10;
```

Cell formatting

Cell attributes can be attached to each cell in a similar way as the cell values. There is a set of grid properties representing each attribute:

- `BackgroundColor[ACol, ARow: Integer]: TsColor` - specifies the background color of the cell. `TsColor` is an integer containing the rgb components of the color. Unit `fpsTypes` provides a list of constants for a large number of predefined colors; the colors defined by the standard unit `Graphics` can be used as well (except for the system color values).
- `CellBorder[ACol, ARow: Integer]: TsCellBorders` - specifies which cell edges will be decorated by a border. Use the set values `[cbEast, cbWest, cbNorth, cbSouth]` for the right, left, top and border edges, respectively.
- `CellBorderStyle[ACol, ARow: Integer; ABorder: TsCellBorder]: TsCellBorderStyle` - specifies the style to be used for the cell border line given in the parameter `ABorder`. The `TsCellBorderStyle` is a record containing information on the line style and line color. Note that the set `CellBorder[ACol, ARow]` must contain the element `ABorder` in order to activate this border line.
- `CellFont[ACol, ARow: Integer]: TFont` - describes the font used when painting the cell text. Elements of the font can be changed separately by these properties:
  - `CellFontColor[ACol, ARow: Integer]: TsColor` - identifies the text color. See "BackgroundColor" above for a description of the TsColor type.
  - `CellFontName[ACol, ARow: Integer]: String` - is the name of the font.
  - `CellFontSize[ACol, ARow: Integer]: Single` - is the point size of the font (1 pt = 1/72 inch).
  - `CellFontStyle[ACol, ARow: Integer]: TsFontStyles` - is a set containing elements for using a bold, italic, underlined, or striked-out font.
- `HorAlignment[ACol, ARow: Integer]: TsHorAlignment` - allows to modify the horizontal alignment of the cell text (`haLeft`, `haCenter`, or `haRight`).
- `NumberFormat[ACol, ARow: Integer]: String` - is an Excel-compatible number format string, e.g. `'0.000'` for displaying a number value with 3 decimal places. The number format is important if numbers are to be displayed as date or time values.
- `TextRotation[ACol, ARow: Integer]: TsTextRotation` - must be used for rotating the text within

the cell. The type `TsTextRotation` provides `trHorizontal`, `rt90DegreeClockwiseRotation`, `rt90DegreeCounterClockwiseRotation`, and `rtStacked`.

- `VertAlignment[ACol, ARow: Integer]: TsVertAlignment` - allows to modify the horizontal alignment of the cell text (`vaTop`, `vaCenter`, or `vaBottom`).
- `Wordwrap[ACol, ARow: Integer]: Boolean` - activates word-wrapping of text which is longer than the width of a cell (or the height if rotated text is used).

These properties can also be accessed for a range of cells specified by the indexes of the left column, top row, right column and bottom row of the cell block. Since these properties are related to several cells they are spelled in plural form (with appended "s"):

- `BackgroundColors[ALeft, ATop, ARight, ABottom: Integer]: TsColor`
- `CellBorders[ALeft, ATop, ARight, ABottom: Integer]: TsCellBorders`
- `CellBorderStyles[ALeft, ATop, ARight, ABottom: Integer; ABorder: TsCellBorder]: TsCellBorderStyle`
- `CellFonts[ALeft, ATop, ARight, ABottom: Integer]: TFont`
- `CellFontColors[ALeft, ATop, ARight, ABottom: Integer]: TsColor`
- `CellFontNames[ALeft, ATop, ARight, ABottom: Integer]: String`
- `CellFontStyles[ALeft, ATop, ARight, ABottom: Integer]: TsFontStyles`
- `CellFontSizes[ALeft, ATop, ARight, ABottom: Integer]: Single`
- `HorAlignments[ALeft, ATop, ARight, ABottom: Integer]: TsHorAlignment`
- `NumberFormats[ALeft, ATop, ARight, ABottom: Integer]: String`
- `TextRotations[ALeft, ATop, ARight, ABottom: Integer]: TsTextRotation`
- `VertAlignments[ALeft, ATop, ARight, ABottom: Integer]: TsVertAlignment`
- `Wordwraps[ALeft, ATop, ARight, ABottom: Integer]: Boolean`

If properties to be read do not have identical values within the cell block then a neutral or default value is returned.

Example

This example adds a formula for today's date to cell A1, formats the cells to display the number as a date and selects a white, italic font on lightgray background. A red dotted border is drawn around the cell.

```
const
  RED_DOTTED_BORDER: TsBorderStyle = (LineStyle: lsDotted; Color: scRed);

// Set cell content
Grid.Cells[1,1] := '=TODAY()';

// Format as date
Grid.Numberformat[1,1] := 'yyyy/mm/dd';

// Select background color
Grid.BackgroundColor[1,1] := scSilver;

// Select format
Grid.FontColor[1,1] := scWhite;
Grid.FontStyle[1,1] := [fssItalic];

// Activate cell borders
Grid.Border[1,1] := [cbEast, cbWest, cbNorth, cbSouth];

// Determine how cell borders will be drawn
Grid.BorderStyle[1,1, cbEast] := RED_DOTTED_BORDER;
Grid.BorderStyle[1,1, cbWest] := RED_DOTTED_BORDER;
Grid.BorderStyle[1,1, cbNorth] := RED_DOTTED_BORDER;
Grid.BorderStyle[1,1, cbSouth] := RED_DOTTED_BORDER;
```

Defining cell borders for a larger cell block by using these properties is a bit cumbersome because different styles may have to be used for the corder, border and inner cells. To simplyfy this task, the grid provides a method `ShowCellBorders` which gets the coordinates of the cell block and the styles of the left, top, right, bottom outer, and horizontal and vertical inner border lines. Use the constant `NO_CELL_BORDER` of no border line should be drawn at the specific location:

```
const
  THICK_CELL_BORDER: TsCellBorderStyle = (LineStyle: lsThick; Color: clBlack);
  DOTTED_CELL_BORDER: TsCellBorderStyle = (LineStyle: lsDotted; Color: clSilver);

// Draw a thick border around the block A1:C3, no inner border
Grid.ShowCellBorders(1,1, 3,3,      // left, top, right, bottom coordinates of cell
  THICK_CELL_BORDER, THICK_CELL_BORDER, THICK_CELL_BORDER, THICK_CELL_BORDER,    //
  NO_CELL_BORDER, NO_CELL_BORDER                                                 //
);

// Draw a thick border around the block A1:C3, dotted gray horizontal inner lines
Grid.SetCellBorders(1,1, 3,3,
  THICK_CELL_BORDER, THICK_CELL_BORDER, THICK_CELL_BORDER, THICK_CELL_BORDER,
  DOTTED_CELL_BORDER, NO_CELL_BORDER
);
```

Cell comments

Comments can be added to each cell by using the grid's property `CellComment[ACol, ARow: Integer]: String`. Cells containing a comment are marked with a red triangle in the upper right corner of a cell. If the mouse is moved into a cell with a comment a popup window appears to display the comment.

Note: For the popup window to show up it is required to add the flag `goCellHints` to the grid's `Options`, and the standard grid property `ShowHint` must be true. Otherwise the popup windows will not appear.

Example:

```
Grid.Cells[1,1] := '=pi()';
Grid.CellComment[1,1] := 'The number pi is needed to calculate the area and circumf
```

Hyperlinks in cells

Cells with attached hyperlinks allow the user to navigate to other cells or other documents by clicking on the cell. Hyperlinks can be accessed by using the property `Hyperlink[ACol, ARow: Integer: String`. The hyperlink string contains the hyperlink target and an optional tooltip text which is separated by means of a bar character ("|"). Internal targets are already handled by the grid, but for navitation to the hyperlink an event handler for `OnClickHyperlink` must be provided. To distinguish normal cell clicks from hyperlink clicks the mouse must be held down for fractions of a second before the hyperlink is executed.

Example:

```
// Example for adding an external hyperlink
Grid.Cells[1,1] := 'Lazarus';
Grid.Hyperlink[1,1] := 'www.lazarus-ide.org|Open Lazarus web site';

// Example for adding an internal hyperlink
Grid.Cells[2,2] := 'Summary';
Grid.Hyperlink[2,2] := '#Sheet2!B10|Go to the summary starting at cell B10 of sheet

// Example for an OnClickHyperlink event handler needed for external hyperlinks
uses
  ..., uriparser;

procedure TForm1.GridOnClickHyperlink(Sender: TObject; const AHyperlink: TsHyperlin
begin
var
  uri: TUri;
begin
  uri := ParseURI(AHyperlink.Target);
  case Lowercase(uri.Protocol) of
    'http', 'https', 'ftp', 'mailto', 'file':
      OpenUrl(AHyperlink.Target);
    else
      ShowMessage('Hyperlink ' + AHyperlink.Target + ' clicked');
  end;
end;
```

Note: Follow the instructions [[#Cell_comments|above]] to show the tooltip text as a popup window.

Merged cells

A rectangular group of cells can be merged to a single block. The content of the top/left cell of this block is displayed across all the combined cells, the content of the other cells is ignored.

Use the method `MergeCells` to perform this operation. As parameters, it requires the left, top, right and bottom coordinates of the cell range to be merged. They also can be combined into a a `TRect` record. The method `Unmerge` splits a previously merged block again in the individual cells, here the coordinates of a single cell from the merged range is sufficient.

```
// Example for merging
  Grid.MergeCells(1,1, 3,1);          // Combine the first 3 cells of the first row
  Grid.Cells[1,1] := 'Summary';       // Write the text "Summary" across the 3 cell
  Grid.HorAlignment[1,1] := haCenter; // and center it.

  // Example for unmerging
  Grid.UnmergeCells(1,1);             // Splits the merged block (1,1..3,1)
```

Column widths and row heights

Column widths and row heights can be changed for all cells by setting the `DefaultColWidth` and `DefaultRowHeight`. This works also in design mode. In addition, the widths and heights of particular columns and rows can be modified by the properties `ColWidths[ACol]` and `RowHeights[ARow]`. These values must be given in pixels.

```
  Grid.DefaultColWidth := 80;  // This is the width of all columns
  Grid.ColWidths[10] := 10;    // except for column #10 which is only 10 pixels wid

  Grid.DefaultRowHeight := 20; // All rows are 20 pixels high
  Grid.RowHeights[2] := 4;     // Row #2 serves as a spacer and is only 4 pixels hi
```

Worksheet and cell protection

The worksheet grid supports protection at the worksheet and cell level. This way, for example, an entire worksheet can be protected from changes by the user except for some specific cells. For this purpose you must follow the same inverted logic used also by Excel and LibreOffice Calc: Protect the entire worksheet, and then remove the protection from the writeable cells. In the following code snipped editing is allowed only in cells A1 and C2:C5:

```
// Protect the worksheet
  Grid.Worksheet.Protection := DEFAULT_SHEET_PROTECTION;
  Grid.Worksheet.Protect(Checkbox1.Checked);

  // Unprotect the writeable cells A1 and C2:C5
  Grid.CellProtection[1, 1] := [];
  Grid.CellProtections[3, 2, 3, 5] := [];
```

For further details on all protection options see the main FPSpreadsheet documentation.

New grid properties

In addition to the properties inherited from its ancestors `TCustomDrawGrid` and `TCustomGrid`, the `TsWorksheetGrid` introduces the following new **published properties**:

- **AllowDragAndDrop** (boolean): Cells can be dragged to a new location if this option is active. Move the mouse cursor to the border of the cells to be dragged until the mouse cursor becomes a four-sided arrow, and then begin dragging. If the dragged cell is referenced by a formula the formula is NOT updated - this behavior is different from Excel but agrees with Open/LibreOffice Calc.
- **AutoCalc** (boolean): Formulas in the grid are automatically recalculated whenever cell content changes.
- **AutoExpand** (set of `aeData, aeNavigation, aeDefault`):
  - If the option `aeData` is contained in the set `AutoExpand` then the grid automatically expands if cells are written outside the predefined range.
  - If `AutoExpand` contains the option `aeNavigation` then the user can *navigate* outside the predefined cell range; new rows and columns are automatically added to the grid (but not to the underlying worksheet). If a file is loaded into the grid then grid dimensions are automatically expanded to the range needed for the worksheet.
  - The option `aeDefault` comes into play if a grid is smaller than the DEFAULT_COL_COUNT and DEFAULT_ROW_COUNT default values. If this option is included the grid is automatically expanded to this default size. On the other hand, if ony a given number of rows and columns should be contained in the grid the option `aeDefault` must be **removed before** setting `RowCount`/`ColCount`.
- **EditorLineMode** determines whether the grid's cell editor supports only single lines (`elmSingleLine`) or multiple lines (`elmMultiLine`). In the latter case, you can press CTRL+ENTER in order to begin a new line during editing a cell.
- **FrozenBorderPen**: if the grid has frozen panes (see `FrozenCols` and `FrozenRows`) a separating line is drawn out the edge of the last frozen row and column. The property `FrozenBorderPen` determines how this line is drawn. Set `FrozenBorderPen.Style = psClear` to hide these lines.
- **FrozenCols** and **FrozenRows** (integer): determines the number of non-scrolling columns (rows) at the left (top) of the grid. Technically these are custom-drawn fixed columns (rows) of the ancestor. Note that the user cannot navigate or edit cells within this range.
- **ReadFormulas** (boolean): Reads formulas from the input files. Since fpspreadsheet does not support all formulas available in the Office spreadsheet applications there is a chance that reading of a file may crash due to formulas; in this case, reading of formulas can be disabled.
- **SelectionPen** (TPen): determines how the border of the selected cell is painted. By default, the selected cell is outlined by a 3-pixel-wide black line.
- **ShowGridLines** (boolean): allows to turn off the grid lines
- **ShowHeaders** (boolean): can be used to turn off the column and row headers ('A', 'B', 'C', ..., '1', '2', '3'). The property **DisplayFixedColRow** has the same effect, but is deprecated now.
- **TextOverflow** (boolean): If this property is on then long text content is allowed to extend into adjacent cells if these are empty. Note that numerical data cells are rounded such that the cell does not overflow.
- **WorkbookSource** (TsWorkbookSource): links to the workbook source which provides the data. If empty, the grid uses an internal workbook source.

Events

- **OnClickHyperlink**: This event fires whenever the user clicks a cell with an embedded hyperlink. Since clicking a cell normally would bring a cell into edit mode it is necessary to hold the mouse key down for about half a second to trigger the hyperlink event.

Recommended property values

In order to set up the grid to behave similar to the well-known Office applications we recommend the following grid property settings. It should be emphasized, though, that differences in usage do exist and cannot be removed without a major re-write of the inherited grid infrastructure:

- `AutoAdvance = aaDown`: The ENTER key advances the selected cell to the next lower cell.
- `AutoCalc = true`: Automatically calculate formulas
- `AutoEdit = true`: For editing a cell, just begin typing. Alternatively you can begin edit mode by pressing F2.
- `AutoExpand = [aeData, aeNavigation, aeDefault]`: Don't restrict usage of the grid to the predefined grid dimensions, for an Excel-like user-interface with an "infinite" worksheet.
- `EditorLineMode = elmMultiLine`: Activate cell editor supporting multiple lines during editing.
- `MouseWheelOption = mwGrid`: The mouse wheel scrolls the grid, not the selected cell.
- `Options`: add these flags to the standard options inherited from TCustomGrid:
  - `goColSizing`: the user can change a column width by dragging the vertical separating line between two column header cells
  - `goRowSizing`: the user can change a row height by dragging the horizontal separating line between two row header cells
  - `goDblClickAutosize`: a double-click on a separating line between two column or row header cells resizes the column width or row height to its optimum value.
  - `goEditing`: puts the grid into edit mode (same as `AutoEdit`)
  - `goThumbTracking`: immediate scrolling of the grid while the scrollbar is dragged with the mouse (if this option is off scrolling occurs at the moment when the mouse button is released).
- `TextOverflow = true`: allow long cell text flow into empty adjacent cells

See also

- FPSpreadsheet tutorial: Writing a mini spreadsheet application
- API documentation of FPSpreadsheet
- RPN formulas in FPSpreadsheet
- FPSpreadsheet: List of formulas
- Grids reference page

Online version

TsWorksheetChartSource

Overview

TsWorksheetChartSource is a charting component which is available in the source code of the FPSpreadsheet library. It is a chart data source component designed to work with the TChart component from the TAChartLazarusPkg package, which usually comes pre-installed with Lazarus.

Usage

At the moment TsWorksheetChartSource has the following properties to control its behavior:

```
  published
    property PointsNumber: Integer read FPointsNumber write SetPointsNumber de
    property XFirstCellCol: Integer read FXFirstCellCol write SetXFirstCellCol
    property XFirstCellRow: Integer read FXFirstCellRow write SetXFirstCellRow
    property YFirstCellCol: Integer read FYFirstCellCol write SetYFirstCellCol
    property YFirstCellRow: Integer read FYFirstCellRow write SetYFirstCellRow
    property XSelectionDirection: TsSelectionDirection read FXSelectionDirecti
    property YSelectionDirection: TsSelectionDirection read FYSelectionDirecti
  end;
```

The Chart will display an amount of PointsNumber points, being that on the coordinates of the X-Axis will be collected from the worksheet starting in the position (XFirstCellRow, XFirstCellCol). Subsequent X-Axis values will be taken from the cells directly to the right of the first cell, if XSelectionDirection has value fpsHorizontalSelection, or below the first cell, if XSelectionDirection has value fpsVerticalSelection. The values will be taken in sequence so if another layout of the data exists, such as each second cell or going up, it can be created by copying the relevant data to a new TsWorksheet.

The same is valid for the Y-Axis value, but applies to the properties YFirstCellCol, YFirstCellRow and YSelectionDirection. The property PointsNumber is valid for both axis.

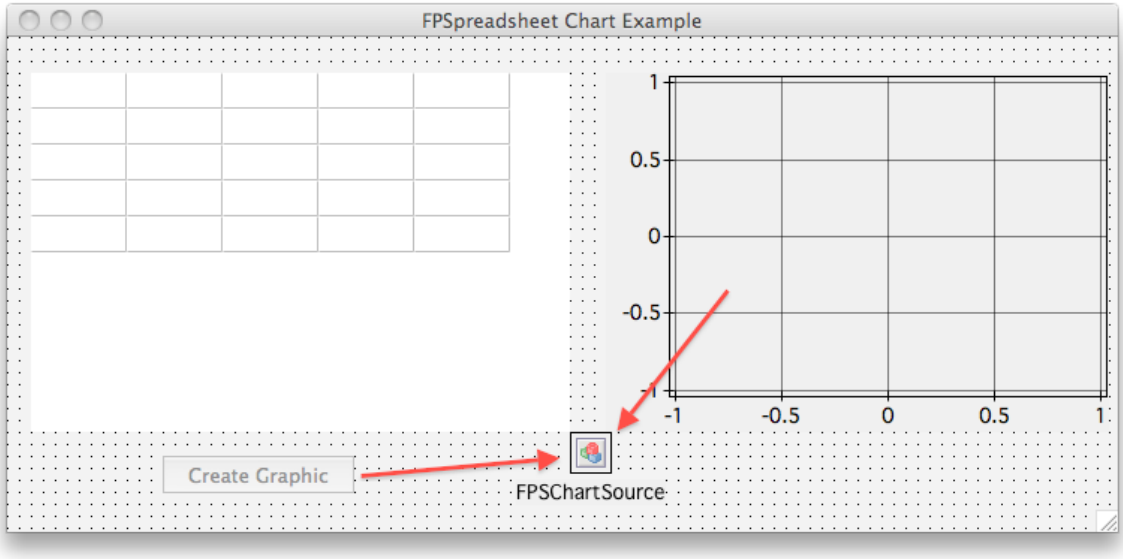
Be careful that if a value could not be read from the worksheet it's value will be considered zero.

The values for Worksheet coordinates use the same values as the FPSpreadsheet API, that is, they are zero-based. So, for example, to select the cell B2 as the starting position for X-Axis values the following values should be used: XFirstCellCol = 1 and XFirstCellRow = 1

Example

A simple example is present in the fpspreadsheet svn repository, located at fpspreadsheet/examples/fpchart/fpchart.lpi

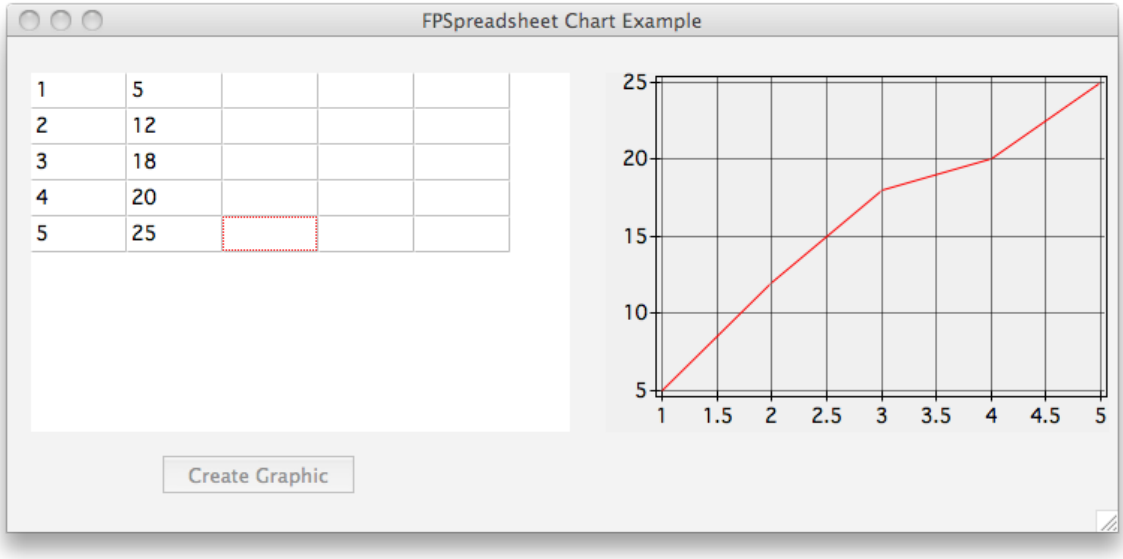
Please see the image below to see how the components are connected at design time. The TChart component can be edited by double-clicking. Then use the menus of the editor to add a new line series and connect this line series to our WorksheetChartSource by setting its Source property.



To load the data from the grid and into the chart, a button was created with the following code. Notice that we can't just use any Grid, it has to be an FPSpreadsheet Grid, which has the class TsWorksheetGrid:

```
type

  { TFPSChartForm }

  TFPSChartForm = class(TForm)
    btnCreateGraphic: TButton;
    MyChart: TChart;
    FPSChartSource: TsWorksheetChartSource;
    MyChartLineSeries: TLineSeries;
    WorksheetGrid: TsWorksheetGrid;
    procedure btnCreateGraphicClick(Sender: TObject);
  private
    { private declarations }
  public
    { public declarations }
  end;

...

procedure TFPSChartForm.btnCreateGraphicClick(Sender: TObject);
begin
  FPSChartSource.LoadFromWorksheetGrid(WorksheetGrid);
end;
```

And finally we can see how this works nicely in a running application in the picture below:



See also

- FPSpreadsheet

Online version

RPN Formulas in FPSpreadsheet

Introduction

FPSpreadsheet usually uses **string formulas**, just like the Office applications (Excel, OpenOffice/LibreOffice) do. In this syntax, formulas are expressed as human-readable strings showing the function names with arguments in brackets.

In addition, FPSpreadsheet supports also **RPN formulas** (RPN = "reverse polish notation) which are used internally in an intermediate stage after parsing a string formula. They are important for the binary Excel file format which stores formulas in this way.

This type of formulas will be discussed here.

If one wants to familiarize themself in PRN logic with quick toying platform one should look up some HPs programmable scientific pocket calculators (or one of those free emulators ie. free42 or still WIP newRPL OS for 50g HW (RPL = Reverse Polish Lisp)) from 1970 to 2016 as most of them are based on RPN notation (and stack)for maths and programming.

Understanding RPN formulas

RPN formulas consist of **tokens**, i.e. information on the constituents of the formula in a way that can be immediately used for calculation of the expression result. There are tokens for numbers, operations, functions etc. When parsing, the tokens are extracted from the expression string and pushed onto a stack. In FPSpreadsheet, this stack corresponds to the array `TsRPNFormula`, the array elements correspond to the tokens on the stack. When calculating the formula, Excel traverses the stack bottom-up (meaning in FPSpreadsheet: from low- to high-index array elements). Whenever it finds a token for an operation or function it removes this token from the stack, along with the tokens of the operands, and replaces them by the result of the calculation.

Here's an example: In a simple expression like `"=4+5"`, the stack contains the tokens for the number constants:

- the first argument: `[4]`
- the second argument: `[5]`
- the operation `[+]`.

The "+" operation is a binary operation, meaning that it needs two arguments. Therefore, when Excel reaches the `[+]` token, it removes the `[+]` and both operands from the stack and replaces them by the result of the calculation, the token with the value 9. Since there are no other elements on the stack, this is the final result of the calculation.

Now a more complex examples: `"=ROUND(2+4*3.141592, 2)"` which rounds the result of the calculation `2+4*3.141592` to two decimals. The function `"ROUND"` requires two parameters: the value to be rounded, and the number of decimal places. In total, the stack consists of these elements:

- `[2]`
- `[4]`
- `[3.141592]`

- [*]
- [+]
- [2]
- [ROUND]

Going from first to last, the first operation/function token met is [*]. As this is another binary operation, this requires two arguments. Therefore, [4], [3.141592] and [*] are removed from the stack and replaced by the result [12.56637].

Now the stack looks like:

- [2]
- [12.56637]
- [+]
- [2]
- [ROUND]

Now, the first operation token found is [+] replacing [2], [12.56637], [+] by [14.56637]. Finally, the stack is left with the tokens needed for the ROUND function:

- [14.56637]
- [2]
- [ROUND]

which immediately leads to the final result [14.57].

Using constant values in RPN formulas

For coding above formula "=4+5" in FPSpreadsheet the length of the RPNFormula array must be set to 3 (3 elements, "4", "5", "+"). The first and second elements are "numbers" which has to be indicated by setting ElementKind=fekNum for these array elements. The value of each number is specified as the DoubleValue of the formula element. The last element is the formula which is specified by the ElementKind of fekAdd.

In total, this results in the following code:

```
var
  MyRPNFormula: TsRPNFormula;
begin
  // Write the formula =4+5
  MyWorksheet.WriteUTF8Text(3, 0, '=4+5'); // A4
  // Write the RPN formula to the spreadsheet
  SetLength(MyRPNFormula, 3);
  MyRPNFormula[0].ElementKind := fekNum;
  MyRPNFormula[0].DoubleValue := 4.0;
  MyRPNFormula[1].ElementKind := fekNum;
  MyRPNFormula[1].DoubleValue := 5.0;
  MyRPNFormula[2].ElementKind := fekAdd;
  MyWorksheet.WriteRPNFormula(3, 2, MyRPNFormula);
end;
```

This requires quite some typing. For simplification a methodology of nested function calls has been added to FPSpreadsheet in which every element is specified by a function which links to the next element function via its last argument:

```
begin
  // Write the formula =4+5
  MyWorksheet.WriteUTF8Text(3, 0, '=4+5');
  // Write the RPN formula to the spreadsheet
  MyWorksheet.WriteRPNFormula(3, 2,   // Row and column of the formula ce
    CreateRPNFormula(                 // function to create a compact RPN
      RPNNumber(4,                    // 1st operand: a number with value
      RPNNumber(5,                    // 2nd operand: a number with value
      RPNFunc(fekAdd,                 // function to be performed: add
      nil)))));                       // end of list
end;
```

Using cells in RPN formulas

Of course, the formulas can also contain links to cells. For this purpose the `ElementType` needs to be `fekCellValue`. This instructs Excel to use the value of the cell in the calculation. There are, however, also functions which require other properties of the cell, like format or address. For this case, use `fekCellRef` for the `ElementKind`. Another specialty is the usage of absolute and relative cell addresses (`$A$1` vs. `A1`, respectively). Cell row and column addresses specified in the RPNFormula elements are absolute by default. If you want relative rows/columns add `rfRelRow` or `rfRelCol` to the element's `RelFlags` set. Or, if you prefer the nested function notation simply use the function `RPNCellValue` (or `RPNCellRef`) with the standard notation of the cell adress using the `$` sign.

Here, as an example, `=A1*$B$1` in array notation:

```
var
  MyRPNFormula: TsRPNFormula;
begin
  SetLength(MyRPNFormula, 3);
  MyRPNFormula[0].ElementKind := fekCellValue;
  MyRPNFormula[0].Row := 0;    // A1
  MyRPNFormula[0].Col := 0;
  MyRPNFormula[0].RelFlags := [rfRelRow, rfRelCol];  // relative!
  MyRPNFormula[1].ElementKind := fekCellValue;
  MyRPNFormula[1].Row := 1;
  MyRPNFormula[1].Col := 0;    // $B$1, RelFlags not needed since absolut
  MyRPNFormula[2].ElementKind := fekMul;
  MyWorksheet.WriteRPNFormula(3, 2, MyRPNFormula);
end;
```

And now in nested function notation:

```
  MyWorksheet.WriteRPNFormula(3, 2,   // Row and column of the formula ce
    CreateRPNFormula(                 // function to create a compact RPN
      RPNCellValue('A1',              // 1st operand: contents of cell "A
      RPNCellValue('$B$1',            // 2nd operand: contents of cell "$
      RPNFunc(fekMul,                 // function to be performed: multip
      nil)))));                       // end of list
```

Using ranges of cells

In spreadsheet applications like Excel, the notation `A1:C5` refers to a range of cells: the rectangle between (and including) cells `A1` and `C5`.

This feature is available in FPSpreadsheet as well: use the `ElementKind` `fekCellRange` and a second set of row/column indices (`Row2` and `Col2`, respectively). There are also flags `rfRelRow2` and `rfRelCol2` to mark the second corner cell as relative.

Using built-in operations and functions

Here is a list of the **basic operations** available in FPSpreadsheet RPN formulas:

ElementKind	Example	Meaning	Operands	Argument types	Argument function
fekAdd	`=A1+A2`	add numbers	2	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekSub	`=A1-A2`	subtract numbers	2	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekMul	`=A1*A2`	multiply numbers	2	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekDiv	`=A1/A2`	divide numbers	2	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekPercent	`=A1%`	divide a number by 100 and add "%" sign	1	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekPower	`=A1^2`	power of two numbers	2	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekUMinus	`=-A1`	unary minus	1	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekUPlus	`=+A1`	unary plus	1	fekNum, fekCellValue	RPNNumber(), RPNCellValue()
fekConcat	`="Hello "&A1`	string concatenation	2	fekString, fekCellValue	RPNString(), RPNCellValue()

Column "Operands" indicates how many operands are required on the stack before the function.

Beyond that, Excel provides a huge number of **functions**, many of which have been made available for FPSpreadsheet via the `fekFunc` symbol. To specify the formula you must pass the formula's name to the `FuncName` element of the `FormulaElement`. The formula name can be found in the 1st column of the table in this page.

Here is an example which calculates the sine function of the number in cell A1:

```
MyWorksheet.WriteRPNFormula(3, 2,   // Row and column of the formula ce
  CreateRPNFormula(                 // function to create a compact RPN
    RPNCellValue('A1',              // 1st operand: contents of cell "A
    RPNFunc('SIN',                  // function to be performed: 'SIN()
    nil))));                        // end of list
```

or, in array syntax:

```
var
  MyRPNFormula: TsRPNFormula;
begin
  SetLength(MyRPNFormula, 2);
  MyRPNFormula[0].ElementKind := fekCellValue;
  MyRPNFormula[0].Row := 0;    // A1
  MyRPNFormula[0].Col := 0;
  MyRPNFormula[0].RelFlags := [rfRelRow, rfRelCol];  // relative!
  MyRPNFormula[1].ElementKind := fekFunc;
  MyRPNFormula[1].FuncName := 'SIN';
  MyWorksheet.WriteRPNFormula(3, 2, MyRPNFormula);
end;
```

Please note that some functions allow a variable count of parameters. In this case, this value has to be specified as `ParamsNum` in the formula. The function `SUM`, for example, accepts up to 30 parameters. For calculating the sum of all numbers in the range A1:C10, therefore, we have to specify explicitly that a single parameter (the cell block A1:C10) is used:

```
  SetLength(MyRPNFormula, 2);
  MyRPNFormula[0].ElementKind := fekCellRange;
  MyRPNFormula[0].Row := 0;    // A1
  MyRPNFormula[0].Col := 0;
  MyRPNFormula[0].Row2 := 9;   // C10
  MyRPNFormula[0].Col2 := 2;
  MyRPNFormula[0].RelFlags := [rfRelRow, rfRelCol, rfRelRow2, rfRelCol2];
  MyRPNFormula[1].ElementKind := fekFUNC;
  MyRPNFormula[1].FuncName := 'SUM';
  MyRPNFormula[1].ParamsNum := 1;       // 1 argument used in SUM
  MyWorksheet.WriteRPNFormula(1, 2, MyRPNFormula);  // cell C2
```

or, shorter:

```
  MyRPNFormula.WriteRPNFormula(1, 2, CreateRPNFormula(
    RPNCellRange('A1:C10',
    RPNFunc(fekSUM, 1,         // SUM with 1 argument
    nil))));
```

Displaying RPN formulas

xls files store formulas in RPN notation internally. When such a file is read FPSpreadsheet reconstructs the string formula automatically.

Please note that the order of calculation is defined by the order of tokens in the RPN formula. The RPN formula by itself does not require parentheses as they would be needed for string formulas. However, this can cause problems when reconstructing string formulas from RPN formulas. For example, suppose the formula "`=(1+2)*(2+3)`". This is parsed to the token sequence `[1], [2], [+], [2], [3], [+], [*]` which makes sure that the correct order of operations is used. When the formula is reconstructed by `ReadRPNFormulaAsString`, however, it will be displayed as "`=1+2*2+3`" which obviously is not correct. To avoid this problem Excel provides a particular "parenthesis" token. In fpspreadsheet, add a `fekParen` token to the token array to put the preceding expression in parenthesis:

```
MyWorksheet.WriteRPNFormula(0, 0, CreateRPNFormula(
  RPNNumber(1,
  RPNNumber(2,
  RPNFunc(fekAdd,
  RPNParen,         // <--- this sets the parenthesis around the term (
  RPNNumber(2,
  RPNNumber(3,
  RPNFunc(fekAdd,
  RPNParen,         // <--- and this is the parenthesis around (2+3)
  RPNFunc(fekMul,
  nil))))))))));
```

It should be emphasized again that the parenthesis token does not have an effect on the calculation result, only on the reconstructed string formula.

Categories: Lazarus-CCR | FPSpreadsheet

Online version